

A Project Report

Submitted To

The Department of Computer Science and Engineering, Metropolitan University, in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science and Engineering.

On

Sarabo: An Electronics Repair Service Management Platform

Submitted By

Jahid Hasan

ID No: 222-115-116

Batch: 57 (C)

Under The Supervision of

Shrabanti Chowdhury

Senior Lecturer

Department of Computer Science and Engineering

Metropolitan University, Sylhet

Submission Date: 26 October 2026



STUDENT'S DECLARATION

I, the undersigned, solemnly declare that the project report entitled **"Sarabo: An Electronics Repair Service Management Platform"** is based on my original project work carried out during the course of study under the supervision of **Shrabanti Chowdhury**. The analysis, design, implementation, testing, and documentation presented in this report were completed as an individual final year project, except where the work of others has been properly cited.

I further declare that, to the best of my knowledge and belief, this report has not been submitted elsewhere in substantially the same form for the requirement of any other degree. All external literature, standards, documentation, libraries, and technologies used in the project have been acknowledged in the reference section.

Jahid Hasan

ID No: 222-115-116

Batch: 57 (C)



SUPERVISOR'S RECOMMENDATION

The project work entitled **"Sarabo: An Electronics Repair Service Management Platform"** submitted by **Jahid Hasan**, ID No: 222-115-116, Batch: 57 (C), is a record of project work carried out under my supervision. I hereby recommend that this project report is satisfactory in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science and Engineering and may be processed for further evaluation.

Shrabanti Chowdhury

Senior Lecturer

Department of Computer Science and Engineering

Metropolitan University, Sylhet

(Supervisor)



LETTER OF APPROVAL

This is to certify that the project work entitled **"Sarabo: An Electronics Repair Service Management Platform"** submitted by **Jahid Hasan**, ID No: 222-115-116, Batch: 57 (C), has been studied and approved by the project examination committee as satisfactory in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science and Engineering.

Shrabanti Chowdhury

Senior Lecturer, Department of Computer Science and Engineering

Supervisor

Nabila Zannat Rifa

Senior Lecturer

Reviewer 1

Md. Fahmidur Rahman Sakib

Senior Lecturer

Reviewer 2

Abstract

Most electronics repair in Bangladesh still happens informally. A customer hands over a device without knowing what it will cost or how the work is going, and independent technicians have no proper way to find jobs or get paid. In this project I built **Sarabo**, a web platform that turns a repair into a tracked service with clear prices and a full record of every step, used by three kinds of people. A *Customer* describes the problem (with optional AI help in Bangla, English or Banglish) and books a pickup time. They then compare ranked offers from qualified local technicians, pay an inspection fee that is later taken off the final price, approve an itemised quote, pay and follow the repair, and the finished repair comes with a 30-day warranty. A *Technician* finds work on an open jobs board, sends offers, inspects the device, quotes, repairs it and withdraws earnings through bKash. An *Admin* approves technicians, looks after a catalogue of 45 services, deals with refunds, disputes and stalled work, and reads reports that include an AI summary.

Sarabo is built with React 19, Node.js/Express 5 and MongoDB. It uses Firebase Authentication, Stripe (in test mode), Supabase storage and Google Gemini, and it is deployed on Vercel. The server checks every step that changes money or a request's state and runs it inside a database transaction, and no request is ever left waiting without a deadline or an admin alert. The system passes 2,761 automated server checks and 58 browser tests in continuous integration.

Keywords: Electronics repair, service marketplace, workflow management, React, Express, MongoDB, Stripe, generative AI.

Acknowledgements

First, I would like to thank my supervisor, **Shrabanti Chowdhury**, Senior Lecturer, Department of Computer Science and Engineering, Metropolitan University, for her guidance, feedback and encouragement throughout this final year project. Her advice helped me turn Sarabo from a rough application idea into a properly structured service management platform.

I am also thankful to the faculty members of the Department of Computer Science and Engineering for the academic foundation this work stands on, and to my reviewers, **Nabila Zannat Rifa** and **Md. Fahmidur Rahman Sakib**, for evaluating the project and sharing their feedback.

Finally, I owe a great deal to the open-source communities and documentation writers behind React, Express, MongoDB, Firebase, Stripe, Supabase, Google Gemini, TanStack Query, Tailwind CSS, Playwright and the other tools Sarabo uses. Without their work, building and testing a full-stack platform of this size on my own would not have been possible.

Contents

Abstract I
Acknowledgements II
Table of Contents III
List of Tables VII
List of Figures VIII
Listings IX

1 Introduction **1**
1.1 Project Overview 1
1.2 Problem Statement 2
1.3 Motivation 2
1.4 Objectives 3
1.5 Scope 3
1.6 Individual Contribution 4
1.7 Report Organization 4

2 Background and Related Work **5**
2.1 Web-Based Service Management 5
2.2 Single-Page Application Architecture 5
2.3 RESTful API and Express 5
2.4 Document Database and Transactions 6
2.5 Authentication and Authorization 6
2.6 Online Payment Processing 6
2.7 Private Object Storage 6
2.8 Large Language Models in Applications 7
2.9 Usability and Accessibility 7
2.10 Related Systems 7

3 Requirements Analysis **9**
3.1 Stakeholders and Roles 9
3.2 Functional Requirements 9
3.3 Non-Functional Requirements 11
3.4 Service Catalogue 11
3.5 Use-Case View 12

4 System Design and Architecture **14**
4.1 High-Level Architecture . . . 14
4.2 Client Architecture . . . 15
4.3 Server Architecture . . . 15
4.4 Data Collections . . . 15
4.5 Repair Request Document . . . 16
4.6 State Model . . . 17
4.7 The Service Spine . . . 17
4.8 API Design . . . 18

5 Implementation **19**
5.1 Technology Stack . . . 19
5.2 Development Process . . . 20
5.3 Frontend Implementation . . . 21
5.4 Backend Implementation . . . 22
5.5 Service Catalogue and Matching . . . 23
5.6 Pickup Scheduling . . . 23
5.7 Quotation Model . . . 23
5.8 Payment Implementation . . . 24
5.9 Technician Settlement and Wallet . . . 24
5.10 Damage and Evidence Images . . . 25
5.11 Public Tracking and Jobs Board . . . 25
5.12 Deployment and Continuous Integration . . . 25
5.13 Final Improvement Phase . . . 25

6 Core Workflows and Business Rules **27**
6.1 End-to-End Repair Workflow . . . 27
6.2 Request Creation . . . 27
6.3 Job Portal and Offers . . . 27
6.4 Offer Ranking (Smarter Matching) . . . 29
6.5 Inspection Fee and Booking . . . 29
6.6 Inspection, Quotation and Revision . . . 30
6.7 Payment, Repair and Receipt . . . 31
6.8 No Request Can Get Stuck . . . 31
6.9 Repair Warranty . . . 32
6.10 Settlement and Withdrawal Lifecycle . . . 33
6.11 Cancellation Rules . . . 34
6.12 Technician Profile Editing . . . 35
6.13 Service-Area Map . . . 35
6.14 Feedback and Moderation . . . 35
6.15 Admin Attention List and Reports . . . 35
6.16 Notifications and Reminders . . . 37
6.17 Damage Image Upload . . . 37

7 System Walkthrough **38**
7.1 Public Website . . . 38
7.2 Creating the Request . . . 42
7.3 Receiving an Offer . . . 45
7.4 Choosing and Booking . . . 47
7.5 Pickup, Inspection and Quote . . . 49
7.6 Payment . . . 52
7.7 Repair and Hand-Over . . . 54
7.8 Warranty . . . 57
7.9 The Technician's Account . . . 61
7.10 Administration . . . 63

8 AI-Assisted Features **71**
8.1 AI Customer Support Chat . . . 71
8.2 AI Request Helper . . . 71
8.3 Technician Price Guide and AI Offer Note . . . 72
8.4 AI Summary of Admin Reports . . . 72
8.5 Safeguards . . . 73

9 Security, Reliability and Data Integrity **74**
9.1 Security Principles . . . 74
9.2 Authentication . . . 74
9.3 Role- and State-Based Authorization . . . 74
9.4 Financial Integrity . . . 75
9.5 Privacy . . . 75
9.6 Private Image Security . . . 75
9.7 Input Validation and Safe Errors . . . 76
9.8 CORS, Rate Limiting and Abuse Control . . . 76
9.9 Transaction Safety . . . 76
9.10 Environment Isolation . . . 77
9.11 Legacy Compatibility Boundary . . . 77

10 Testing and Evaluation **78**
10.1 Testing Strategy . . . 78
10.2 Verification Results . . . 79
10.3 Representative Test Scenarios . . . 80
10.4 Manual End-to-End Acceptance . . . 81
10.5 Evaluation Against Objectives . . . 82
10.6 Validation Boundary . . . 82

11 Limitations and Future Work **83**
11.1 Current Limitations . . . 83
11.2 Future Enhancements . . . 84

12 Conclusion **85**

References 85

Appendices 88

A API Surface 89

B Data Model Summary 92

C Financial Rules 93

D Source and Build Notes 94

E Incident Record: Revised Quote Hang 96

List of Tables

1.1 Project objectives 3

2.1 Comparison of approaches to electronics repair 8

3.1 User roles 9
3.2 Functional requirements 10
3.3 Non-functional requirements 12
3.4 Service catalogue summary with sample services 12

4.1 MongoDB collections 16
4.2 Repair lifecycle states 17
4.3 Representative API endpoints 18

5.1 Implemented technology stack 19
5.2 Development phases (from the git history) 20
5.3 Final improvement phase log 26

6.1 Offer ranking factors 29
6.2 Inspection fee outcomes 30
6.3 Situations that used to block a request, and how they end now 32

8.1 AI risks and safeguards 73
8.2 Default AI usage limits 73

10.1 Verification results (14–16 October 2026) 79
10.2 Representative test scenarios 80
10.3 Evaluation against objectives 82

A.1 Sarabo API endpoints (main families) 89

B.1 Selected fields of a repair request 92

C.1 Worked example 93

D.1 Verified code baseline 94

List of Figures

3.1 Role-oriented use-case view 13

4.1 Sarabo high-level architecture 14
4.2 Conceptual structure of a V2 repair request 16
4.3 The four-stage service spine shown to users 18

6.1 End-to-end repair workflow 28
6.2 Rejected-quote recovery 31
6.3 Warranty claim lifecycle 33
6.4 Technician settlement and withdrawal lifecycle 33
6.5 Damage-image upload workflow 37

7.1 Home page 39
7.2 Services and prices 39
7.3 Service areas map 40
7.4 AI support chat 40
7.5 Public jobs board 41
7.6 Public tracking page 41
7.7 Home page and jobs board on a phone 42
7.8 Customer overview 43
7.9 AI request helper 43
7.10 Pickup address and time 44
7.11 Request created 44
7.12 Jobs board for a technician 45
7.13 Technician sends an offer 46
7.14 My repairs list 46
7.15 Offer seen by the customer 47
7.16 Choosing an offer 48
7.17 Stripe Checkout for the inspection fee 48
7.18 Technician booked 49
7.19 Technician's assigned jobs 50
7.20 Device condition at pickup 50
7.21 Inspection report 51
7.22 Writing the quote 51
7.23 Customer reviews the quote 52

7.24 Approving and paying the quote . . . 53
7.25 Payment successful . . . 53
7.26 Payment history . . . 54
7.27 Repair in progress . . . 55
7.28 Completing the repair . . . 55
7.29 Marking the device returned . . . 56
7.30 Confirming receipt . . . 56
7.31 Rating the technician . . . 57
7.32 Warranty claim . . . 58
7.33 Booking the free re-inspection . . . 58
7.34 The technician's record of the claim . . . 59
7.35 Resolved warranty claim . . . 59
7.36 Customer notifications . . . 60
7.37 Technician overview . . . 61
7.38 Technician wallet . . . 62
7.39 Editing the technician's details . . . 62
7.40 Completed jobs . . . 63
7.41 Admin operations overview . . . 64
7.42 All repair requests . . . 64
7.43 A request seen by the admin . . . 65
7.44 Assigning technicians . . . 65
7.45 Technicians and applications . . . 66
7.46 Services and prices catalogue . . . 66
7.47 Editing a service . . . 67
7.48 Admin reports with AI summary . . . 67
7.49 Requests over time and demand by region . . . 68
7.50 Devices, repairs and pickup times . . . 68
7.51 Recording a bKash payout . . . 69
7.52 Technician reviews . . . 69
7.53 User management . . . 70

10.1 Testing layers used in Sarabo . . . 78

Listings

5.1 Quote total and amount still to pay 23
5.2 Settlement rule (whole taka) 24
D.1 Development commands 94
E.1 Round-aware quote notification key 96

Chapter 1

Introduction

1.1 Project Overview

Sarabo is a web-based platform for managing electronics repair services, which I developed on my own as my final year project. It handles the whole life of a repair request, from the moment a customer describes a faulty device to the moment the technician gets paid, as one workflow that can be traced from start to finish. Three people are involved: a *Customer* who needs something repaired, a *Technician* who does the repair, and an *Admin* who keeps the service running.

The most important design decision was to treat a repair request as a *case that moves through states*, not as a simple database record. Every important activity moves the case from one clearly defined state to the next: creating the request, technician offers, booking, pickup, inspection, quotation, payment, repair, hand-over, settlement and feedback. For each move, the server checks that this user is allowed to make it at this moment. This is the main difference between Sarabo and a basic create-read-update-delete (CRUD) application. Whether a later action is correct depends on earlier actions having happened, and on them having happened only once.

The project was built in two main stages. In the first stage (July–August 2026) I built the core repair lifecycle, in which an admin assigned a technician to each request. An earlier version of this report described that stage. In the second stage (September–October 2026) Sarabo became a two-sided marketplace. It gained an open jobs board where technicians send offers, a paid inspection fee with automatic refunds, pickup scheduling, customer reviews, email notifications, operational reports, four AI-assisted features and a production deployment. A final phase (11–16 October

2026) then added offer ranking, a repair warranty, deadlines for every waiting state, a minimum inspection fee and faster hosting. This report describes the system as it stands after all of that work.

1.2 Problem Statement

A repair service needs both technical work and a lot of coordination. The customer has to describe the device and the problem, get an idea of the cost before handing the device over, choose someone they can trust, pay safely and find out whether the device has actually been repaired. The technician needs a steady supply of nearby jobs that match their skills, a clear way to record an inspection and a quote, and a reliable way to be paid. The administrator needs to see which requests are stuck, which payments need attention, and whether customers are being turned away.

When all of this is handled through phone calls, messaging apps and paper notes, things easily go wrong. A customer is charged before agreeing to a price. A technician is booked twice for the same time. An inspection fee is neither refunded nor taken off the bill. A technician's earning is paid out twice, or private photos of a customer's device end up in the wrong hands. Sarabo deals with this by modelling the repair process as a workflow that knows each user's role, with clear rules for every change of state, validation on the server and updates that run as transactions.

1.3 Motivation

I had five main reasons for choosing this project:

1. to work on a real, local problem, by making electronics repair in Bangladesh more transparent and trustworthy for customers and easier to access for independent technicians;

2. to build a realistic full-stack system that goes beyond simple CRUD, where roles and state changes are enforced on the server instead of being left to the browser;

3. to integrate real, production-grade services (identity, card payments and refunds, private file storage, email and generative AI) without exposing any privileged credentials;

4. to use generative AI where it actually helps people, while keeping it safe: it must not invent prices or leak personal data, and its usage must have hard limits; and

5. to give every user a clear interface in which the next useful action is always easy to see.

1.4 Objectives

The main objective was to build a reliable platform that manages the complete lifecycle of an electronics repair. Table 1.1 lists the specific objectives.

Table 1.1: Project objectives

No.	Objective
O1	Provide secure authentication and role-aware access for Customer, Technician and Admin.
O2	Let customers create structured repair requests (device, service, region, pickup window, optional damage photos), with optional AI assistance.
O3	Match requests only to approved technicians with the right expertise in the customer's region, through an open offer-based job portal.
O4	Support a paid inspection fee that books a technician, is credited toward the final price, and is refunded or split fairly when the repair does not go ahead.
O5	Provide an inspection and quotation workflow with customer approval, rejection and revision.
O6	Accept payment only for approved amounts, with a single authoritative paid state and downloadable receipts.
O7	Support repair progress, completion evidence, receipt confirmation and customer feedback.
O8	Keep customers and technicians informed through in-app notifications, email, reminders and a sanitized public tracking page.
O9	Provide a technician wallet with a 90/10 settlement and controlled bKash withdrawals.
O10	Give admins operational oversight: attention lists, reports, catalogue management and moderation.
O11	Validate the system through automated server tests, browser tests, static checks and continuous integration, and deploy it.
O12	Protect customers after the repair with a warranty, and make sure no request can get stuck in any state.

1.5 Scope

The finished system covers the following: registration and email verification, technician applications with structured expertise, a versioned service catalogue, AI-assisted request creation, damage-photo upload and pickup scheduling. It also covers an open jobs board with offers, booking with an inspection fee and refunds, inspection, quotation and revision, Stripe payment, repair

progress with completion evidence, receipt confirmation, and reviews and private reports. On top of that there are in-app and email notifications, public tracking, a technician wallet with bKash withdrawals, an AI customer-support chat, admin reports with an AI summary, offer ranking, a 30-day repair warranty, deadlines and admin alerts for stalled work, a record of the device's condition at pickup, and a production deployment.

The project does not try to do everything a commercial field-service company would. Automatic mobile-wallet payouts, cash payment, spare-parts inventory, live GPS tracking of technicians, native mobile apps and accounting for several branches are outside the current scope. Chapter 11 discusses them as future work.

1.6 Individual Contribution

Sarabo is a solo project, and I, Jahid Hasan, did all of the work on it. This covered requirements analysis, data modelling, user-interface design, frontend and backend development, the workflow and financial logic, integration of the external services, AI prompt and safety design, testing, deployment, debugging and writing this report.

1.7 Report Organization

Chapter 2 gives the technical background and looks at related systems. Chapter 3 sets out the stakeholders and requirements. Chapter 4 presents the architecture, the data model and the state model. Chapter 5 describes the implementation and how development was carried out. Chapter 6 explains the core workflows and business rules. Chapter 7 walks through the running system, with screenshots of every main step for each role. Chapter 8 covers the AI-assisted features and how they are kept safe. Chapter 9 discusses security, reliability and data integrity, and Chapter 10 presents testing and evaluation. Chapter 11 lists limitations and future work, and Chapter 12 concludes the report. The appendices summarise the API, the data model and the financial rules, give build notes, and record one notable incident from development.

Chapter 2

Background and Related Work

2.1 Web-Based Service Management

A static information system mostly stores and shows data. A service platform is different, because each request moves forward through decisions made by different people. Sarabo uses a resource-oriented HTTP API in the REST style [1]. The browser shows the workflow, but only the server decides whether a change of state is allowed.

2.2 Single-Page Application Architecture

The client is a React single-page application. React builds the interface out of stateful components [2], React Router maps the public and role-specific routes, and TanStack Query manages server state, caching and invalidation [3]. This is important here because a request's status, its offers, quotes, balances and notifications can all change while a user is in the middle of filling in a form. Vite [4] builds the app and Tailwind CSS [5] handles the styling.

2.3 RESTful API and Express

The server runs on Express 5 [6]. Its routes are grouped by domain: users, technicians, requests, offers, inspection, quotes, payments, repairs, warranty, wallet, feedback, notifications, tracking, catalogue, uploads, support and administration. Sensitive routes check who the caller is and what role they have before touching the database.

2.4 Document Database and Transactions

MongoDB [7] is a good fit for a repair request. The request keeps one identity for its whole life while collecting nested workflow data along the way, such as product and price snapshots, images, the inspection, the quote and its history, payment, evidence, assignment history and settlement. Things that have their own lifecycle, index or audit trail live in separate collections (offers, payments, notifications, tracking events, withdrawals, reviews and warranty claims). Multi-document transactions on a replica set make sure several of them change together, or not at all.

2.5 Authentication and Authorization

Firebase Authentication provides identity and signed ID tokens [8]. The server verifies each token with the Firebase Admin SDK before it trusts the caller's email. Authorization works on two levels: by role (Customer, Technician or Admin) and by state (the request has to be at the right stage). The Technician role is still stored as `rider`, a leftover from the original codebase, but users only ever see the word "Technician".

2.6 Online Payment Processing

Card payments go through Stripe Checkout [9], and the server's own record is the one that counts. Stripe calls a webhook that is verified with a shared secret [10], so a payment is still recorded even if the customer closes the browser halfway. Refunds use idempotency keys, which means the same refund can never be issued twice. For now, payments run in test mode.

2.7 Private Object Storage

Images are kept in a private Supabase Storage bucket [11]. The browser uploads each file directly through a short-lived signed URL, the server then checks the stored file's real type and size, and every time someone views an image a fresh signed URL is created. This way large files never pass through the API, and the storage key never reaches the browser.

2.8 Large Language Models in Applications

Large language models such as Google Gemini [12] can turn free text into structured data, summarise information and draft messages. They also have well-known weaknesses: they can be confidently wrong, they can be steered by instructions hidden inside user text (prompt injection), and their cost can be hard to predict. OWASP lists prompt injection and overreliance among the main risks [13]. For this reason Sarabo uses AI only as an *assistant*. Its answers must follow a JSON schema, they are checked against live data, they never set a price, and a person always reviews them (Chapter 8).

2.9 Usability and Accessibility

The interface is built around actions. The stage the repair has reached and the next thing the user needs to do are shown more prominently than secondary details. A "service spine" with four stages (Request, Inspect, Approve, Repaired) appears on the home page, the tracking page and the dashboards. Labels, focus order, keyboard access, colour contrast, reduced motion and layouts that work from 320 px wide all follow WCAG 2.2 [14].

2.10 Related Systems

Today, people in Bangladesh get repairs and home services in three broad ways:

- **Local repair shops and freelance technicians**, found through word of mouth or social media. The price is negotiated face to face, there is no tracking, and the customer has little to fall back on if something goes wrong.
- **Brand service centres**, which repair their own products reliably but cover only one brand and are mostly found in large cities.
- **General on-demand service marketplaces** such as Sheba.xyz in Bangladesh [15] and Urban Company in South Asia [16], where customers book many kinds of household services from listed providers.

International platforms such as Thumbtack [17] made a particular model popular: the customer describes a job and several professionals reply with quotes. Sarabo uses this offer model but adapts it to electronics repair, where the real price is usually unknown until someone has opened the device. Table 2.1 compares the approaches on the features that matter most for this project.

Table 2.1: Comparison of approaches to electronics repair

Capability	**Local shop**	**General marketplace**	**Sarabo**
Online request with device and problem details	–	✓	✓
Price estimate before hand-over	–	Partial	✓
Several competing offers per job	–	Varies	✓
Separate inspection fee credited to final price	–	–	✓
Formal inspection and itemised quote	–	Varies	✓
Online payment with receipt	–	✓	✓
Step-by-step public tracking	–	Partial	✓
Technician matched by repair-specific expertise	–	Partial	✓
Bangla/Banglish AI request assistance	–	–	✓

Sarabo does not introduce a new algorithm. Its value is in the engineering: it brings together patterns from ticketing systems (a trackable case with an ID and a history), field-service systems (assignment, inspection and completion), marketplaces (offers, payment, commission and payouts) and evidence-based workflows (private photos and completion records). These are combined into one consistent repair workflow, with clear rules for state, money and access.

Chapter 3

Requirements Analysis

3.1 Stakeholders and Roles

Sarabo has three roles for signed-in users, plus visitors who use it without an account. Table 3.1 summarises what each of them wants from the system.

Table 3.1: User roles

Role	Primary goal	Representative actions
Customer	Get a device repaired at a fair, known price	Create request (optionally with AI help), add photos, pick a pickup slot, compare offers, pay inspection fee, approve or reject quote, pay, track, confirm receipt, review or report the technician, claim warranty
Technician	Find suitable work and get paid	Apply with expertise and region, browse the jobs board, send offers, inspect, quote, revise, repair, complete, view wallet, withdraw via bKash
Admin	Keep the service healthy	Approve technicians, edit catalogue and prices, handle missed pickups, refunds, disputes and warranty reviews, process withdrawals, moderate reviews and reports, read reports
Visitor	Learn about the service	Browse services, service areas and open jobs, track a repair with a tracking code, ask the AI support chat

3.2 Functional Requirements

Table 3.2 lists the functional requirements. FR-21 to FR-32 were added during the second development stage, and FR-33 to FR-38 during the final phase.

Table 3.2: Functional requirements

ID	Requirement
FR-01	The system shall authenticate users (email/password and Google) and protect private routes.
FR-02	The system shall distinguish Customer, Technician and Admin capabilities on the server.
FR-03	A verified Customer shall be able to create a structured repair request for a catalogue service.
FR-04	The service catalogue shall provide active device and repair categories with BDT estimate ranges, inspection fee caps (never below the minimum fee) and required expertise level.
FR-05	Customers shall be able to attach up to three damage photos through a controlled private-storage workflow.
FR-06	Users shall be able to apply as Technicians with region, district and structured expertise; Admin shall approve or reject applications.
FR-07	The assigned Technician shall submit an inspection before a quote can be submitted.
FR-08	The Technician shall submit an itemised quote (labour, parts, additional charges).
FR-09	The Customer shall approve or reject the quote; after rejection the Technician may revise once more or close the request.
FR-10	Payment shall be possible only for an approved quote and never twice.
FR-11	The Technician shall record repair progress and completion with optional evidence photos.
FR-12	The Customer shall confirm receipt of the repaired device.
FR-13	The system shall send role-appropriate in-app notifications for workflow events.
FR-14	A public tracking page shall show a sanitized timeline by tracking code, without personal or payment data.
FR-15	A 90/10 Technician/platform settlement shall be frozen when the repair is paid.
FR-16	Technicians shall request withdrawals up to their server-computed available balance; Admin shall mark each as paid (with bKash TrxID) or rejected exactly once.
FR-17	A customer shall be able to cancel a request before a technician is booked.
FR-18	Admin shall be able to edit the service catalogue; price changes shall create a new version without changing existing requests.
FR-19	Technicians shall be able to edit their expertise; Admin shall see technician availability.
FR-20	Customers shall be able to download a PDF receipt for every payment.
FR-21	A request shall be accepted only if at least one approved Technician in the customer's region can perform it; refused demand shall be counted.
FR-22	Customers shall choose a two-hour pickup slot (Saturday–Thursday, up to 7 days ahead) subject to per-region capacity, and may reschedule before pickup.
FR-23	Open requests shall appear on a public jobs board; matching Technicians shall be notified and may send offers (estimate range, inspection fee, note).
FR-24	The Customer shall compare offers and choose one, or reject one (the job is reposted and the Technician may send one revised offer).

ID	Requirement
FR-25	Choosing an offer with a fee shall open a Stripe payment; the confirmed payment shall book the Technician atomically.
FR-26	The inspection fee shall be credited toward the final quote, refunded when the visit fails or is cancelled in time, and split 90/10 when kept.
FR-27	Customers shall rate a completed repair (1–5 with optional comment) or file a private report; Admin shall moderate both.
FR-28	The system shall email request confirmations, pickup reminders and new-job alerts, and post in-app pickup reminders daily.
FR-29	Admin shall see an attention list (overdue and missed pickups, unmatchable requests, no offers in 24 hours, refunds waiting) and period reports.
FR-30	An AI helper shall turn a free-text description and/or damage photo into a draft request that the customer reviews.
FR-31	Technicians shall see a price guide from past offers and may ask AI to draft their offer note.
FR-32	An AI support chat shall answer visitor questions from live catalogue data, and Admin reports shall include an AI summary checked against the real figures.
FR-33	Offers shall be ranked by rating, reliability, price, district and reply time, with the best match and its reasons shown; the customer still chooses.
FR-34	A finished repair shall carry a 30-day warranty: free re-inspection by the same technician, free re-repair if covered, and an admin refund if it cannot be fixed.
FR-35	Every waiting state shall have a deadline, an automatic step or an admin alert (unpaid quote, not repairable, repair that cannot be finished, receipt not confirmed, device not returned, stalled work).
FR-36	The technician shall record the device's condition and accessories at pickup, and may report a customer who is not home without it counting as a no-show.
FR-37	Every repair shall have an inspection fee of at least 100 BDT (200 BDT when the estimate exceeds 1,000 BDT).
FR-38	Technicians shall edit their own name, phone, address and service area (the area only while no repair is active).

3.3 Non-Functional Requirements

3.4 Service Catalogue

The catalogue has **45 services** spread over eight device categories (Table 3.4). Each service links a device category to a repair category. It also has an estimated price range in BDT, a maximum inspection fee, the expertise level a technician needs (beginner, intermediate, advanced or expert) and an estimated duration. These prices are only estimates. The amount the customer actually pays

Table 3.3: Non-functional requirements

Category	Requirement
Security	Identity, role, prices, fees, balances and refund decisions are derived or validated on the server. Private images are reachable only through short-lived signed URLs. Secrets never reach the browser.
Consistency	Steps that change several records (booking, approval, completion, settlement, refunds) run as database transactions.
Reliability	Repeated or concurrent actions return controlled conflicts instead of repeating money or state changes. Failed refunds stay pending and can be retried. Email or notification failures never fail the main action.
Privacy	Public pages (tracking, jobs board) expose no names, emails, addresses or payment data. AI providers receive no personal identifiers.
Cost control	Every AI feature has per-user and site-wide daily limits stored in the database, and is off unless explicitly enabled.
Usability	Each screen shows the current stage and the next action for that role; empty, loading and error states are truthful.
Accessibility	Keyboard access, visible focus, sufficient contrast, labelled controls, reduced-motion support, layouts from 320 px.
Performance	Cached server state on the client, code-split routes, direct-to-storage uploads, paginated lists.
Maintainability	Workflow rules, status presentation and query keys are centralised; CI runs lint, build and tests on every push.
Auditability	Tracking events, assignment history, quote history, payment rows, refund state, withdrawal processing and moderation history are kept.

is always the technician's written quote after inspection.

Table 3.4: Service catalogue summary with sample services

Device	Services	Sample service	Estimate (BDT)
Smartphone	8	Screen Replacement	1,500–6,000
Laptop / Computer	8	Motherboard Repair	2,500–12,000
Television	5	Screen / Panel Repair	4,000–18,000
Refrigerator	6	Compressor / Refrigerant Service	3,500–15,000
Washing Machine	6	Control Board Repair	2,000–7,000
Air Conditioner	6	Gas Refill / Compressor Repair	3,000–20,000
Microwave Oven	4	Electrical / Power Repair	1,500–5,000
Other Electronics	2	General Repair	1,000–5,000
Total	**45**		

3.5 Use-Case View

Figure 3.1 shows the main things each role does on the platform.

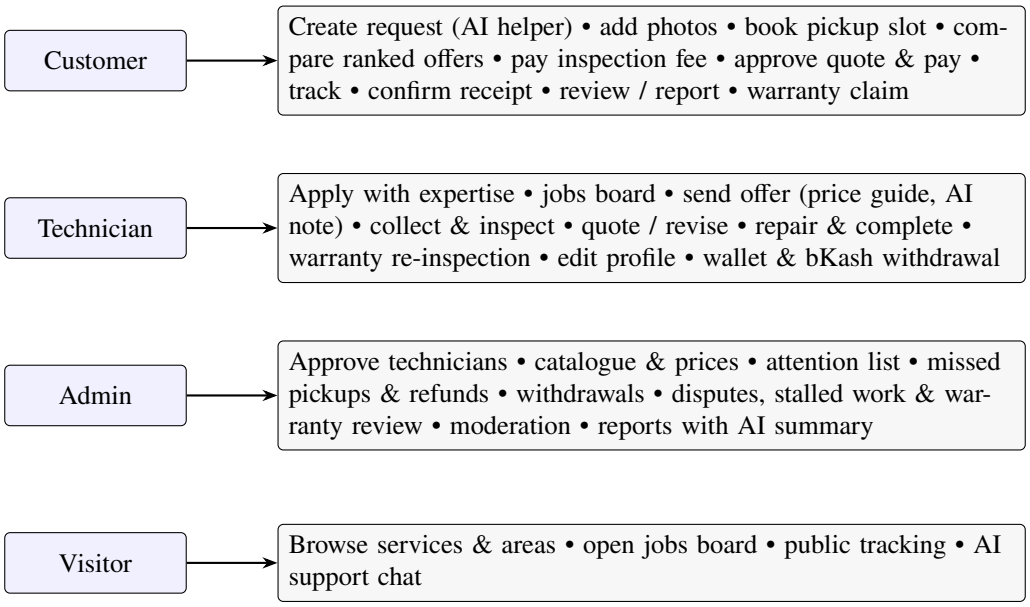


Figure 3.1: Role-oriented use-case view

Chapter 4

System Design and Architecture

4.1 High-Level Architecture

Sarabo has two parts: a client that runs in the browser and an API server. They rely on outside services for identity, payment, storage, email and AI (Figure 4.1). Both parts run on Vercel. The API is a serverless function in the Mumbai region, close to the MongoDB Atlas cluster that holds all the data, and Vercel also triggers a job once a day.

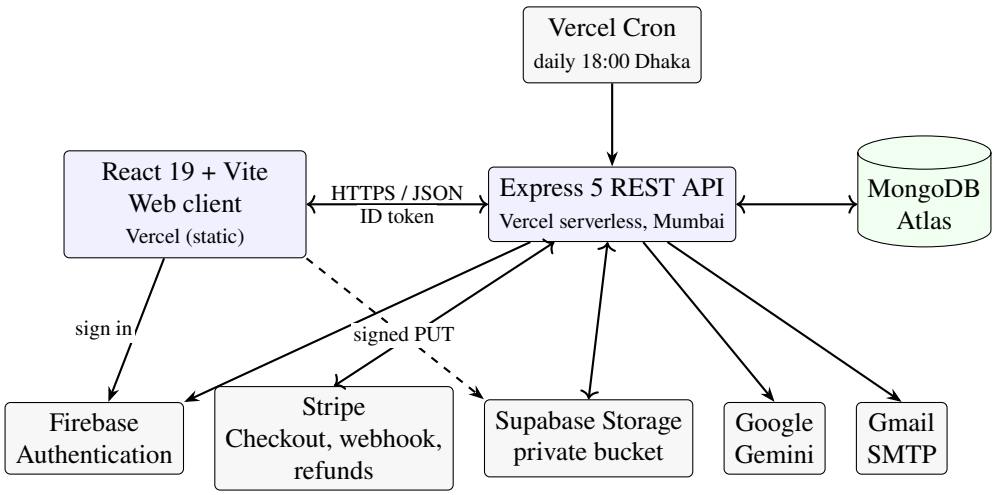


Figure 4.1: Sarabo high-level architecture

4.2 Client Architecture

The client has about 337 source files, organised into five areas:

1. **Routing and guards**: the public pages (home, services, areas, jobs, tracking, about, become a technician), the sign-in pages, and a dashboard that `CustomerRoute`, `TechnicianRoute` and `AdminRoute` divide by role. The server checks every rule again on its side.

2. **Feature components**: the request form and AI helper, photos, pickup slots, offers, inspection, quote, payment, repair, warranty, feedback, wallet, notifications, support chat and administration.

3. **Server-state hooks**: TanStack Query with query keys kept in one place, which handles caching, loading and error states, mutations and refreshing only the data that changed.

4. **Presentation utilities**: the "service spine" that maps each status to a stage, currency formatting, eligibility helpers and an error message for each action.

5. **Design system**: shared design tokens, light and dark themes, and Radix UI primitives.

4.3 Server Architecture

The server has about 191 JavaScript files, tests included. It is built in layers: *routes* → *middleware* → *controllers* → *services* / *utilities* → *database*. The middleware gives each request an ID, logs it safely, verifies the Firebase token and waits until the database is ready. Each controller is responsible for the state rules of one domain. Services hold the logic that several controllers share, such as eligibility, booking, refunds, settlement, notifications, matching, AI and email. External providers are wrapped in adapters, so the tests can swap them for fakes.

The Stripe webhook route is registered before the JSON parser, because checking the signature needs the raw request body. If the database goes down for a moment, the routes stay registered, so requests fail with a clear 503 error instead of a misleading 404.

4.4 Data Collections

The data is stored in 21 MongoDB collections, listed in Table 4.1.

Table 4.1: MongoDB collections

Collection	Purpose
`users`	Accounts and the authoritative role.
`technicians`	Application, approval status, expertise, region/district, work status, bKash details.
`repair_requests`	The repair case with all lifecycle sub-documents.
`job_applications`	Technician offers on open requests and their states.
`service_definitions`	Versioned catalogue of services and prices.
`payments`	Stripe payment rows for inspection fees and repairs, including refund state.
`checkout_sessions`	Checkout idempotency and expiry.
`technician_withdrawals`	Withdrawal requests and admin processing.
`technician_reviews`	Customer ratings with moderation history.
`technician_reports`	Private customer reports with status and admin notes.
`notifications`	In-app notifications with de-duplication keys.
`tracking_events`	Sanitized public timeline.
`damage_upload_sessions`	Customer photo upload sessions.
`repair_evidence_sessions`	Technician completion-evidence sessions.
`deletion_cleanups`	Retry records for storage clean-up after deletion.
`turned_away_requests`	Hashed count of refused demand (180-day expiry).
`email_log`	Sent/failed email record (90-day expiry).
`support_rate_limits`	Shared AI rate-limit counters (auto-expiring).
`ai_report_summaries`	Last AI summary per report period.
`warranty_claims`	Warranty claims with their full decision and refund history.
`warranty_photo_uploads`	Short-lived upload sessions for warranty photos.

4.5 Repair Request Document

Each request has `schemaVersion: 2`, a `product` snapshot (category, brand and model) and its own copy of the chosen service with its price rule. Because of this copy, editing the catalogue later never changes a request that already exists (Figure 4.2).

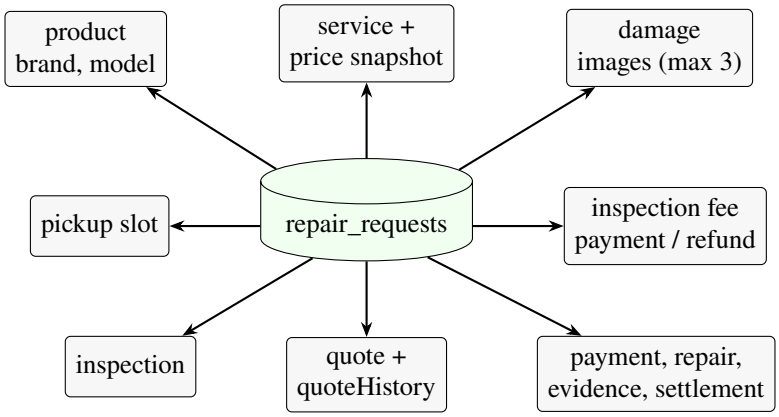


Figure 4.2: Conceptual structure of a V2 repair request

4.6 State Model

The status field, `deliveryStatus`, and several of its values come from the parcel-delivery codebase that Sarabo started from. I kept them as a fixed internal contract, and users only ever see repair terms (Table 4.2).

Table 4.2: Repair lifecycle states

Stored state	Meaning shown to users
`pending-pickup`	Waiting for a technician; open on the jobs board.
`assignment_pending`	Offered directly by an admin; technician to accept or reject.
`driver_assigned`	Technician booked.
`rider_arriving`	Technician on the way.
`parcel_picked_up`	Device collected.
`inspection_completed`	Inspection submitted.
`quote_submitted`	Final price waiting for the customer.
`quote_approved`	Customer accepted the price.
`quote_rejected`	Customer declined the price.
`payment_completed`	Customer paid.
`repair_in_progress`	Repair underway.
`repair_completed`	Repair finished; waiting for hand-over.
`parcel_delivered`	Customer confirmed receipt (terminal).
`cancelled`	Ended through an allowed path.

Other records have their own, smaller sets of states. A job application (an offer) can be `pending`, `accepted`, `not_chosen`, `withdrawn`, `no_show` or `declined`. A settlement goes from *pending* to *available*, or to *reversed* after a refund. A warranty claim starts as *submitted*, goes through re-inspection and a decision, and ends as *resolved* or *refunded*. A withdrawal goes from *requested* to either *paid* or *rejected*.

4.7 The Service Spine

That many states would be hard for a customer to follow, so the interface groups them into four stages that are easy to understand (Figure 4.3). These stages appear on the home page, the tracking page and every dashboard. Confirming receipt is not a fifth stage. It is the hand-over action that comes after “Repaired”.

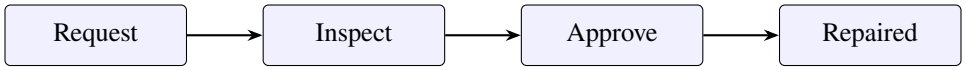


Figure 4.3: The four-stage service spine shown to users

4.8 API Design

The API has more than 120 endpoints, grouped into families by resource. Table 4.3 shows some typical ones, and Appendix A gives a longer list.

Table 4.3: Representative API endpoints

Method	Route	Purpose
POST	`/repair-requests`	Create a request
GET	`/jobs`	Public open-jobs board
POST	`/repair-requests/:id/applications`	Technician sends an offer
POST	`/repair-requests/:id/applications/:aid/accept`	Customer chooses an offer
POST	`/repair-requests/:id/inspection`	Submit inspection
POST	`/repair-requests/:id/quote/decision`	Approve or reject quote
POST	`/repair-requests/:id/checkout-session`	Start repair payment
POST	`/stripe-webhook`	Signed payment events
POST	`/repair-requests/:id/repair/complete`	Complete repair
POST	`/repair-requests/:id/review`	Rate the technician
POST	`/ai/request-draft`	AI request helper
GET	`/admin/attention`	Items needing admin action
GET	`/public/trackings/:code`	Sanitized tracking

Chapter 5

Implementation

5.1 Technology Stack

Table 5.1: Implemented technology stack

Layer	Technology	Role in Sarabo
Frontend	React 19, Vite 7	Component-based single-page app and build tooling
Routing	React Router 7	Public, authenticated and role-specific routes
Server state	TanStack Query 5, TanStack Table	Caching, mutations, invalidation; admin data tables
UI	Tailwind CSS 4, Radix UI, Motion, Recharts, Leaflet	Design system, accessible primitives, animation, charts, service-area map
Forms	react-hook-form	Validated multi-step forms
Backend	Node.js, Express 5	REST API, middleware, controllers, services
Database	MongoDB 7 driver, MongoDB Atlas	Persistence and multi-document transactions
Identity	Firebase Authentication + Admin SDK	Sign-in (email, Google) and token verification
Payments	Stripe 20	Checkout, signed webhooks, refunds
Storage	Supabase Storage	Private images with signed URLs
AI	Google Gemini (`@google/genai`)	Support chat, request helper, offer note, report summary
Email	Nodemailer (Gmail)	Request, reminder and job-offer emails
Documents	pdfkit	PDF payment receipts
Testing	Custom Node test harness, Playwright	Server integration tests; browser tests
Delivery	GitHub Actions, Vercel	CI, serverless API (Mumbai region), cron, static hosting

5.2 Development Process

I built Sarabo in small phases that were each easy to review, from 26 July to 4 October 2026, followed by a final improvement phase from 11 to 16 October. Altogether this came to 163 commits on the client and 120 on the server. Every phase followed the same loop. First I defined the rule, then implemented it on the server together with its tests, then built the interface, ran lint, the build and the tests, and finally reviewed the result. Larger features were developed on their own branches and merged through a pull request once CI had passed (Table 5.2).

Table 5.2: Development phases (from the git history)

Phase	Dates	Main work
1. Foundation and hardening	26–28 Jul	Imported the base codebase; secured authentication and role routing; secure Stripe checkout and webhook; validated status transitions; public sanitized tracking; transactional assignment, approval and completion.
2. Admin and notifications	29 Jul – 4 Aug	Admin request management; in-app notifications; production database isolation; health monitoring; double-booking prevention.
3. Repair domain (V2)	5–8 Aug	Service taxonomy and priced catalogue; technician expertise; V2 request schema; expertise-based matching; damage uploads; prices in BDT; inspection, quote, payment and repair workflows; safe deletion; verified email.
4. Rename to Sarabo	10–12 Aug	Migrated database, collections, fields, modules, API routes and UI from parcel to repair vocabulary; assignment accept/reject; receipt confirmation.
5. Money and UI redesign	13–19 Aug	90/10 settlement; wallet and withdrawals; storage moved to Supabase; complete "service spine" redesign; accessibility pass. *(Earlier report submitted 20 Aug.)*
6. AI support and feedback	18–24 Sep	Gemini support chat; reviews and private reports with moderation; product tour; second redesign; admin catalogue editor; CI on GitHub Actions.
7. Operations	24–25 Sep	Pickup scheduling; region rule; email updates; Playwright tests; admin reports and attention list; in-app reminders; refusing unmatchable requests.
8. Job portal and inspection fee	25 Sep – 2 Oct	Technician offers and customer choice; inspection-fee payment, credit, refunds and split; reject-and-repost; public jobs board; job notifications; PDF receipts; one-step approve-and-pay.
9. AI helpers and payouts	2–4 Oct	AI request helper (text and photo); price guide and AI offer note; AI report summary; bKash withdrawals; hosting configuration.

Phase	Dates	Main work
10. Final improvements	11–16 Oct	Map search, final payout on removal, faster API and AI, offer ranking, warranty, deadlines for every waiting state, pickup condition, minimum inspection fee, technician profile editing (Section 5.13).

5.3 Frontend Implementation

5.3.1 Role-Aware Routing

The dashboard is behind a route that requires sign-in, with guards for each role. Customers see Overview, Create Request, My Requests and Payment History. Technicians see Overview, Find jobs, Assigned jobs, Completed jobs, Warranty claims and Wallet. Admins see Overview, Repair requests, Assign technicians, Technicians, Withdrawals, Warranty claims, Users, Services & prices, Reports, Technician reports and Technician reviews. Every role also has Notifications and Profile, and on phones these pages move to a tab bar at the bottom. Deep links such as `?category=smartphone` are kept through sign-in. The routing is only there for convenience, though. Every protected endpoint checks authentication and authorization again on the server.

5.3.2 Server-State Hooks

Each feature has its own query-key factory (for example `quoteKeys` or `walletKeys`) and hooks for reading and changing data. When something changes, only the queries that are now out of date are refreshed. Approving a quote, for instance, refreshes the quote, the request, the request list and the payment eligibility. This keeps what all three roles see in step, without anyone having to reload the page.

5.3.3 Action-First Workspace

Every request page (see Figure 7.23 for an example) starts with the service spine. Below it is a “next step” panel that tells the signed-in user what to do, such as “Choose a technician” or “Approve and pay BDT 3,800”, and the supporting details come after that. Shared components for empty, loading, error and confirmation states and for forms make every page behave the same way.

A product tour introduces each dashboard, and a command menu lets users move around with the keyboard.

5.4 Backend Implementation

5.4.1 Controllers, Services and Adapters

Each domain has a controller for its endpoints, for example `jobPortalController`, `quoteController` and `paymentController`. Logic that is shared goes into services such as `jobBooking`, `inspectionFeeRefunds`, `paymentEligibility`, `technicianMatchGate` and `notificationService`. The external providers (Supabase, Gemini and Stripe) are wrapped in adapters, which the tests replace with fakes.

5.4.2 Database Transactions

Whenever several records depend on each other, they are updated inside a MongoDB transaction. Take booking after an inspection-fee payment. In one transaction (`services/jobBooking.js`) the server records the payment, sets the request to `driver_assigned`, marks the technician as busy, accepts the chosen offer and marks all the other offers `not_chosen`. If any of these steps fails, none of them is saved.

5.4.3 Notifications, Email and Scheduled Jobs

All in-app notifications are created through one service. Each one has a de-duplication key, so retrying an action never sends the same notification twice. Emails (request received, pickup reminder and new job) are sent through Gmail after the main action has finished. If an email fails, it is logged in `email_log`, and the request itself still succeeds.

Once a day a Vercel Cron job calls `GET /cron/pickup-reminders` at 12:00 UTC, which is 18:00 in Bangladesh. The call is protected by a secret. The job sends the pickup reminders for the next day and applies every deadline described in Section 6.8. It closes declined and unpaid quotes that nobody acted on, confirms receipts that the customer has left unanswered for three days, and warns admins about late warranty steps and stalled work. Each of these steps runs separately, so

if one fails the others still run. Email and the daily job are both turned on in production, which means the live site sends reminders by email as well as in the app.

5.5 Service Catalogue and Matching

Admins edit services and prices from the dashboard. Changing a price increases `pricingRule.version`, while requests that already exist keep their own copy of the old price. Every service has an inspection fee of at least 100 BDT, or 200 BDT if the top of its estimate is above 1,000 BDT. A single rule, `minimumInspectionFee`, is used for requests, offers, the catalogue editor and the public price list. A one-time migration raised the fee for the 16 live services that had none.

When a request is created, `technicianMatchGate` checks that there is an approved technician in the region who covers the device and the repair at the required level. If there is not, the API answers `409 NO_TECHNICIAN_AVAILABLE`, and the refused request is counted once per customer, service, region and day, with the customer stored only as a hash. These counts show admins where they need to recruit more technicians.

5.6 Pickup Scheduling

Customers choose one of four two-hour windows between 10:00 and 18:00 Bangladesh time, from Saturday to Thursday, anywhere from one hour to seven days ahead. Each region accepts a set number of pickups per slot. This number can be configured and is three by default, and a full slot returns `409 PICKUP_SLOT_FULL`. The customer can change the window until the device is collected, and if the pickup moves, the reminder is sent again.

5.7 Quotation Model

A quote is made up of labour, parts and additional charges, all in whole taka, plus a total, a currency, a version and a status. When a quote is rejected, it is moved to `quoteHistory` before the live quote is cleared. That way there is still a record of it, and it can never be paid by mistake.

Listing 5.1: Quote total and amount still to pay

```
repairSubtotal = laborAmount + partsAmount + additionalCharges
quote.totalAmount = repairSubtotal (currency "BDT")
amountDue = max(quote.totalAmount - inspectionFeePaid, 0)
```

5.8 Payment Implementation

Every Stripe Checkout session is created from data the server has stored, either the approved quote or the fee of the chosen offer. It never uses an amount sent by the browser. The session's metadata records what it is paying for, either `repair` or `inspection_fee`. The signed webhook and, as a backup, the success page both call the same idempotent processor, so each payment is recorded exactly once. A single server function decides whether a request counts as paid, and the eligibility checks, admin filters and status badges all use it. Every payment also has a PDF receipt showing the Stripe ID, the date in Dhaka time, the device, the repair, the technician, the amount and the refund status.

5.9 Technician Settlement and Wallet

As soon as a repair is paid, the server takes a snapshot of the settlement from the approved quote, and this snapshot does not change afterwards:

Listing 5.2: Settlement rule (whole taka)

```
commission = round(repairSubtotal * 10 / 100) # Sarabo
receivable = repairSubtotal - commission # Technician
# invariant: commission + receivable == repairSubtotal
```

The settlement stays *pending* until the customer confirms they have received the device, and then it becomes *available*. Balances are never stored. They are worked out each time from the settlements and withdrawals, so they cannot slowly drift away from the truth. A technician can withdraw up to their available balance to a bKash number. An admin then sends the money with bKash Send Money and records the TrxID. A unique partial index makes sure each technician can only have one open withdrawal at a time.

5.10 Damage and Evidence Images

Images go straight from the browser to a private Supabase bucket through a signed upload URL. They must be JPEG, PNG or WebP files of up to 8 MB, with at most three per request. Before attaching an image, the server checks the real type and size of the stored file. Signed links for viewing it last five minutes and are given only to the owner, an admin or the assigned technician. When something is deleted, the storage clean-up is retried until it succeeds.

5.11 Public Tracking and Jobs Board

Every request gets a tracking code of the form `SRB-<random>`, with 128 bits of randomness. `GET /public/trackings/:code` only accepts exact codes, is rate limited, and returns nothing but the status timeline. The jobs board (`GET /jobs`, limited to 60 requests per minute per address) shows the device, the repair, the area, the pickup day, the usual price and the number of offers. It never shows the problem description, photos, the tracking code or any personal details.

5.12 Deployment and Continuous Integration

The API is a Vercel serverless function (Node.js 24) at `https://sarabo-server.vercel.app`, and the client is a static Vercel site at `https://sarabo-client-six.vercel.app`. Both are deployed from the `main` branch. The API runs in Vercel's Mumbai region (`bom1`), close to the MongoDB Atlas cluster. Moving it there brought the first request after an idle period down from 10.7 s to 2.9 s, and a normal request from 0.75 s to about 0.2 s. Production has a database of its own, and the server refuses to start there unless its secrets and an `https` client origin are set.

GitHub Actions runs on every push. It runs the server suites on a throwaway MongoDB replica set, never with real credentials, and it runs lint, the build and the browser tests for the client. I only push to `main` after all of these checks have passed.

5.13 Final Improvement Phase

The features were finished on 4 October. After that, a final phase from 11 to 16 October 2026 got the project ready for submission. It began with a review of both codebases and a fresh run

of every check (Chapter 10). I recorded each change in the project work log, with the problem it solved, the files it touched and how it was verified. Table 5.3 summarises these changes.

Table 5.3: Final improvement phase log

#	Date	Change and verification
0	11 Oct 2026	Codebase and documentation review; re-ran server suites against an isolated in-memory MongoDB replica set, client lint, production build and Playwright browser tests.
1	11 Oct 2026	Service-area map: OpenStreetMap coordinates for 226 of the 236 covered sub-areas, each kept only if it lies inside its own district. Searching a sub-area (e.g. Ambarkhana) now zooms to that spot instead of the district centre. Verified with two new browser tests on desktop and mobile.
2	11 Oct 2026	Final payout on technician removal: removing a technician queues their whole available balance as a bKash withdrawal in the same transaction (Section 6.10). Verified with 15 new server checks.
3	11 Oct 2026	Homepage hero: the photo moved to the right side (top band on phones) so the headline no longer sits over it. Checked with screenshots at three screen widths.
4	11 Oct 2026	Final verification of the committed code and CI on both repositories (Section 10.2).
5	14 Oct 2026	Slow first load: the API was moved to Vercel's Mumbai region, next to the database (10.7 s → 2.9 s cold, 0.75 s → 0.2 s warm, measured live).
6	14 Oct 2026	AI support timeouts: faster Gemini model with an automatic fallback model; live answers now take 1.4–1.9 s. 4 new tests.
7	14 Oct 2026	Smarter matching: offers ranked by rating, reliability, price, district and reply time, with a "Best match" badge and reasons (Section 6.4). 22 new tests.
8	14 Oct 2026	30-day repair warranty with free re-inspection, re-repair, admin review and refunds (Section 6.9). 45 new tests.
9	14 Oct 2026	Whole-workflow review, then fixes so no request can get stuck: unpaid quotes, not-repairable devices, repairs that cannot be finished, unconfirmed receipt, warranty deadlines (Section 6.8). 42 new tests.
10	14 Oct 2026	Remaining gaps: late payment after cancellation refunded, device return after cancellation, stalled-work alerts, pickup condition, customer not home. 30 new tests.
11	14 Oct 2026	Photos upload only when the customer presses "Upload"; technicians edit their own profile; minimum inspection fee for every repair, applied to old offers and migrated on production. 22 new tests.
12	14 Oct 2026	All of the above pushed and deployed; full verification (Section 10.2) and screenshots of the running system (Chapter 7).
13	16 Oct 2026	Stripe's repair-payment page named the item "Repair request: undefined"; it now names the device ("Repair: Samsung Galaxy A54"). 1 new test; pushed and deployed.

Chapter 6

Core Workflows and Business Rules

6.1 End-to-End Repair Workflow

Figure 6.1 shows the main path a repair takes, from the first request to the technician's settlement.

6.2 Request Creation

The customer chooses a device and a repair from the live catalogue, or lets the AI helper suggest them. They add the brand, the model and a description, and can also add up to three photos, which are sent only when they press "Upload". Finally they pick a pickup window. The estimate shown is the price range from the catalogue. While the customer fills in the form, it checks `GET /service-availability`, and the server checks once more when the form is submitted. If no approved technician in the region can do the repair at the required level, the request is refused and the customer is told why. An accepted request gets a tracking code and a confirmation email (Figures 7.9 and 7.10).

6.3 Job Portal and Offers

While a request is waiting for a technician, it is *open*. It appears on the public jobs board as a summary with nothing private in it, and every approved technician who matches it gets one `job_posted` notification. A technician who only starts to match later, after being approved or

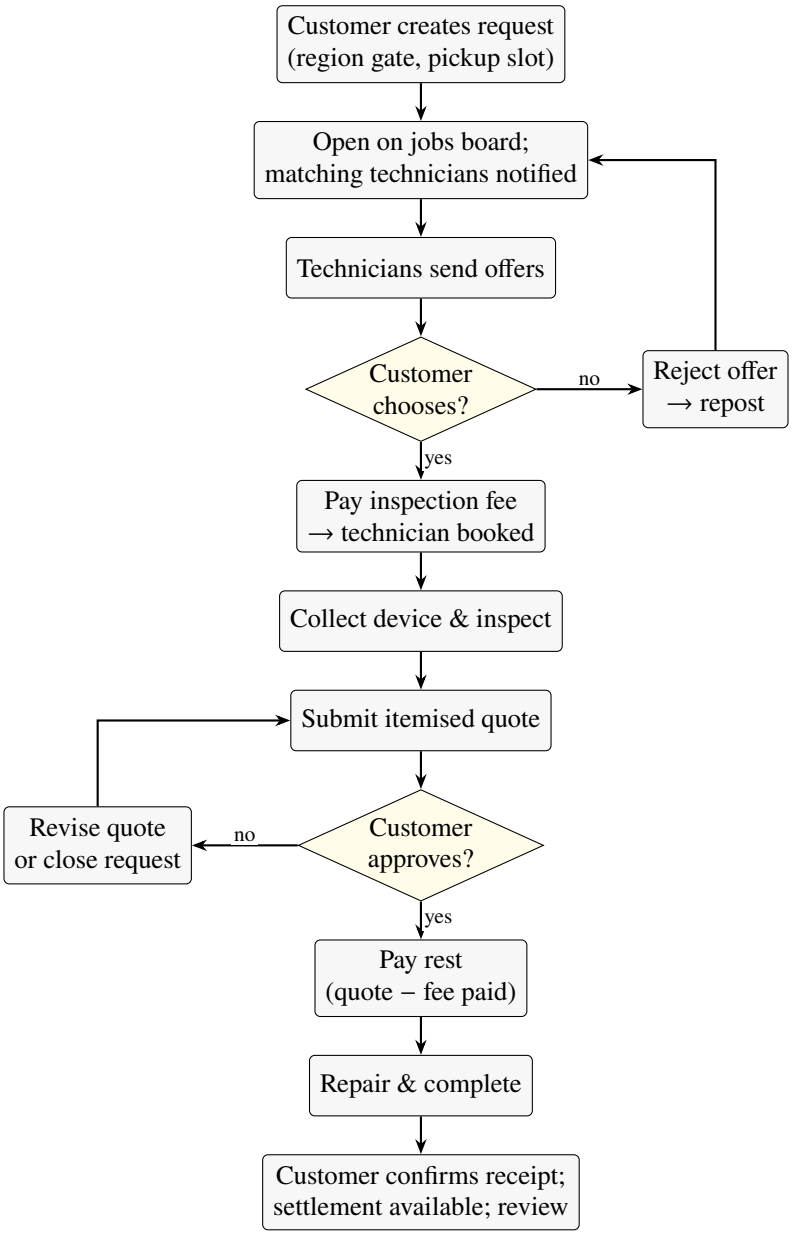


Figure 6.1: End-to-end repair workflow

adding a new skill, is notified then. Technicians can see the district, the pickup time, the problem and the photos, but never the customer's name, email or street address.

An *offer* contains an estimated price range, an inspection fee (somewhere between the minimum fee and the catalogue fee) and a note. The customer can choose an offer, or reject one and give a reason (the price, the fee or something else). After a rejection the job is *reposted*. The rejected technician may send one revised offer, and the other matching technicians are notified again. If a job still has no waiting offer 24 hours after it was last posted, it goes on the admin attention list. Admins can also invite a technician to apply, or step in and assign one directly (Figures 7.13 and 7.15).

6.4 Offer Ranking (Smarter Matching)

Offers are not simply listed in the order they arrive. They are ranked. A scoring module with no side effects (`services/offerRanking.js`) combines five factors, all taken from data Sarabo already stores, so it needs no paid service, map API or AI (Table 6.1). If a technician has no history for a factor yet, they get a neutral score for it. When there are two or more offers, the top one gets a "Best match" badge. The reasons behind it, such as "Rated 4.8 ★ from 12 reviews" or "In your district", are shown only when they are actually true. The customer can also sort the offers by price or date. The final choice is always theirs, and nothing is ever booked automatically.

Table 6.1: Offer ranking factors

Factor	Weight	How it is measured
Rating	30%	Average review, smoothed toward 3 ★ so one 5 ★ review does not beat forty 4.8 ★ reviews
Reliability	25%	Repairs completed compared with missed pickups
Price	20%	The offer's estimate compared with the other offers on the same job
District	15%	Same district as the customer
Reply time	10%	Median minutes from the job being posted to the technician's offer (last 20 offers, at least 3)

6.5 Inspection Fee and Booking

When the customer chooses an offer, a Stripe Checkout opens for exactly that offer's fee. Once Stripe confirms the payment, a single transaction records it and books the technician. Sometimes

booking is no longer possible by then, because the technician is busy, the request was cancelled or another offer was chosen. In that case the payment is refunded automatically and the customer can choose again. The fee is stored separately from the repair payment and is taken off the final price (Figures 7.16 and 7.23). Table 6.2 shows what happens to the fee when the repair does not go ahead.

Table 6.2: Inspection fee outcomes

Situation	Fee
Repair approved and paid	Credited toward the final price
Technician does not come (admin records a missed pickup)	Full refund; other offers can be chosen
Customer cancels 2 hours or more before the pickup window	Full refund
Customer cancels less than 2 hours before	Kept: 90% technician, 10% Sarabo
Customer declines the final price and the request ends	Kept: 90% technician, 10% Sarabo
Booking fails after payment	Automatic full refund

A refund happens in two steps. First, the transaction that causes the refund marks the fee and its payment row as *refund pending*. After that transaction has been committed, Stripe is called with the idempotency key `inspection-refund-<session id>`, so no more than one refund can ever be made. If the refund fails, it stays pending on the admin home page, where it can be retried with a single click. It is never quietly lost.

6.6 Inspection, Quotation and Revision

Only the booked technician can submit the inspection (the findings and the issues found), and only after collecting the device. A quote can only be submitted from `inspection_completed`. The customer sees the quote together with the fee they have already paid and the amount still due. The button “Approve and pay BDT *X*” takes them straight to payment.

If the customer rejects the quote, the technician can either revise it or close the request (Figure 6.2). Revising archives the old quote, and the cycle of submitting and deciding starts again. If nobody acts for three days, the daily job closes the request and records who closed it (`technician`, `customer` or `auto`). The inspection fee is then kept and split.

Each round of quoting has its own notification identity, `quoteHistory.length + 1`, which the server works out. Because of this, a retried submission does no harm, while a real revision does send a new notification (Appendix E).

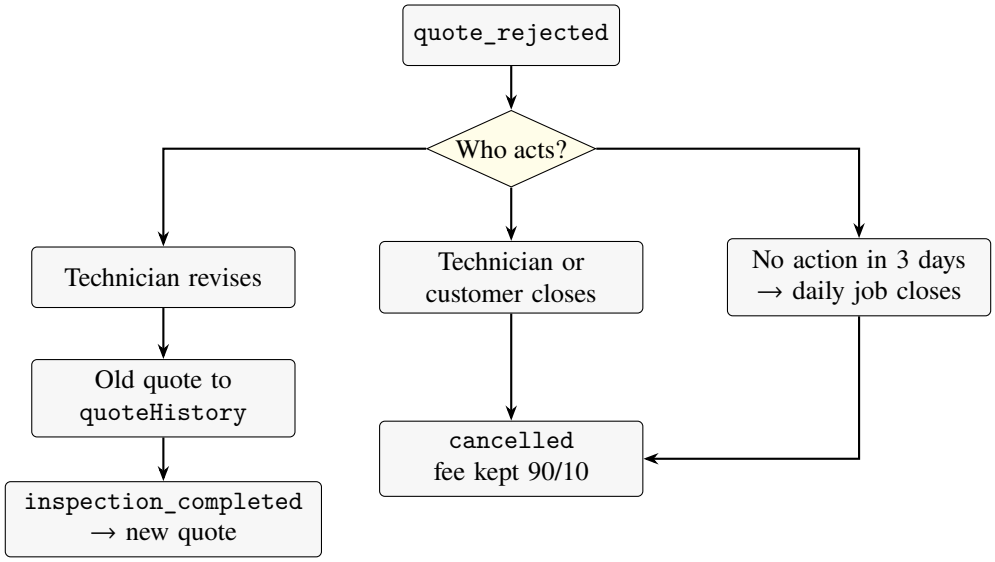


Figure 6.2: Rejected-quote recovery

6.7 Payment, Repair and Receipt

Only a quote that is approved and not yet paid can be paid. After payment the technician starts the repair, posts progress updates and marks it complete, with evidence photos if they want. Completing the repair frees the technician for new work and asks the customer to confirm that they have received the device. Only this confirmation ends the request and makes the technician's earning available. So no technician can withdraw money for a device the customer has not got back.

6.8 No Request Can Get Stuck

Before the final release I went through the whole workflow looking for situations in which a request could end up waiting forever. Each of them now has a way out, an automatic step once a deadline passes, or an alert for an admin (Table 6.3). A daily job applies the deadlines. One set of rules (`services/stalledWork.js`) puts any work that has not moved for three days on the admin overview, or fourteen days in the case of a repair that is in progress.

Table 6.3: Situations that used to block a request, and how they end now

Situation	What happens now
Approved quote never paid	Customer can cancel ("I don't want the repair"); otherwise closed automatically after 3 days. The inspection fee is kept 90/10 and the technician is freed.
Device not repairable or not worth it	Technician or customer closes the request; same fee outcome.
Paid repair cannot be finished	Technician reports why; an admin cancels and refunds the whole amount, and the technician's settlement is reversed.
Customer never confirms receipt	Technician marks the device returned; receipt is confirmed automatically after 3 days unless the customer says "I haven't received it", which goes to an admin.
Device must go back after a cancellation	The return is recorded, confirmed by the customer or automatically after 3 days; disputes go to an admin.
Customer not at home at pickup	Technician reports it; the customer is asked for a new time. It is not counted as a technician no-show.
Payment arrives after cancellation	Recorded and refunded at once; a failed refund stays on the admin refund list.
Work not moving (no inspection, quote, answer or repair)	Shown on the admin overview with one alert per request and stage.

Pickup condition. When collecting the device, the technician writes down its condition and the accessories that come with it, such as a charger, a case or a SIM. The customer can see this record and dispute it, in which case an admin reviews it. This protects both sides if there is a disagreement later (Figure 7.20).

6.9 Repair Warranty

Every finished repair is covered for 30 days from the day the customer confirms receipt. The warranty covers the same fault coming back or a fitted part failing. It does not cover new damage, water damage or drops (Figure 6.3).

A customer can have one open claim at a time, and two claims per repair. Every step is a guarded database update, so no step can be skipped, and if two people act at the same moment only one of them succeeds. If a deadline is missed, the claim is marked overdue and admins can take it over. A technician who still has open claims cannot be removed. A refund pays back the whole quote through Stripe, meaning the repair payment plus the fee that was credited, with a separate

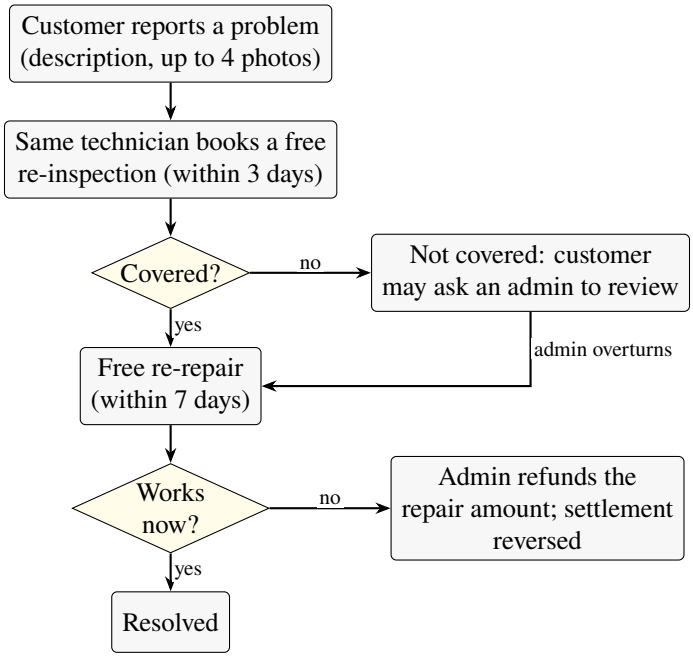


Figure 6.3: Warranty claim lifecycle

idempotency key for each part. The settlement then becomes *reversed*. If the technician has already withdrawn that money, it is shown as owed and taken out of their future earnings (Figures 7.32 and 7.35).

6.10 Settlement and Withdrawal Lifecycle

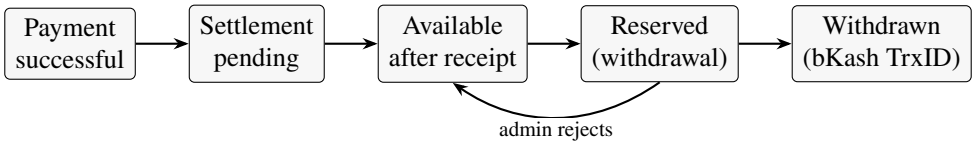


Figure 6.4: Technician settlement and withdrawal lifecycle

The wallet is never stored. It is calculated every time from the following sums:

$$\text{Pending} = \sum \text{paid settlements not yet receipt-confirmed} \tag{6.1}$$

$$\text{Gross available} = \sum \text{receipt-confirmed settlements} + \sum \text{kept inspection-fee shares} \tag{6.2}$$

$$\text{Reserved} = \sum \text{open withdrawal requests} \tag{6.3}$$

$$\text{Withdrawn} = \sum \text{paid withdrawals} \tag{6.4}$$

$$\text{Available} = \text{Gross available} - \text{Reserved} - \text{Withdrawn} \tag{6.5}$$

When an admin rejects a withdrawal, the reservation simply disappears, so there is never a need to write a correcting entry to the balance.

Final payout when a technician is removed. When an admin removes an approved technician, the technician's whole available balance is queued as a bKash withdrawal in the same transaction, so either both happen or neither does. The bKash number is taken from the first of these that exists: the number the admin enters, a withdrawal request that is already open (which is topped up rather than duplicated), the last withdrawal, or the phone number from the technician's application. If money is owed and there is no valid number, the removal is refused. Money that is still pending is not included. The payout is tagged "final", and it is sent and recorded like any other withdrawal, by hand through bKash Send Money, because Sarabo has no bKash API.

6.11 Cancellation Rules

Until a technician is booked, the customer can cancel at no cost and later use "Request again", which fills in the old details. Once a technician is booked and the fee is paid, `GET /repair-requests/:id/booking-cancellation` tells the customer in advance whether cancelling now would refund the fee or keep it. Cancelling is possible only until the device has been collected.

6.12 Technician Profile Editing

Technicians can edit their own name, phone, address and service area (the region and district together). They cannot change their email, national ID, approval status or availability. The service area is locked while they have an active repair, because that repair's pickup was booked in the old area. After a change, the technician is told about the open jobs in their new area.

6.13 Service-Area Map

The Service Areas page shows every district on an OpenStreetMap map (using Leaflet). It reads the same `serviceAreas.json` file that the request form uses. I looked up the coordinates of the sub-areas once, offline, and kept a point only if it really lies inside its own district, which was true for 226 of the 236. The search lists sub-areas before districts, so searching "Ambarkhana" opens the map at street level instead of at the city centre. A sub-area without a point falls back to its district's pin, and no geocoding happens in the browser.

6.14 Feedback and Moderation

Once the customer has confirmed receipt, they can leave one review for the repair: 1 to 5 stars and, if they like, a plain-text comment of up to 1,000 characters. Admins can hide a review or bring it back, giving a reason, and every change is kept in a moderation history. A customer can also send a private report about a technician, for example about unprofessional behaviour, damage to the device or a payment problem. Only admins see these reports. A report moves through *open*, *under review*, *resolved* or *dismissed*, and it is never deleted along with the request. Technicians can see their rating and their visible reviews, but never the reports or who wrote a review.

6.15 Admin Attention List and Reports

Attention list. The admin home page shows the work that needs a person first (Figure 7.41). This includes missed pickups (a technician was booked but did not collect the device), overdue pickups (the window passed and no technician was found), requests with no offer 24 hours after posting, stalled repairs, requests that no technician in the region can take, and inspection-fee refunds still

waiting on Stripe. The items are sorted by urgency and then by age. Clicking a count opens its list with the filter already applied, for example "Assign technicians" showing only overdue or unmatched requests.

Reports page. `GET /admin/reports?days=7|30|90` builds a single read-only report for the last 7, 30 or 90 days, counted in Bangladesh time (Figures 7.48–7.50). The report has seven parts:

- **Overview**: requests made, repairs finished, requests cancelled, the success rate (finished ÷ (finished + cancelled)), the average number of days from request to finished repair, and the requests turned away because no local technician offers that repair. Each figure is shown next to the figure for the previous period of the same length.
- **Trend**: a line chart of requests made and repairs finished per day. "Show as a table" gives the same numbers as text.
- **Regions**: for each division, the number of requests, how many are waiting, how many of those no local technician can take, how many were turned away, and how many technicians are approved and how many are free. Regions that need more technicians stand out.
- **Devices and repairs**: the eight most requested devices and repair types.
- **Pickup times**: a heat map of booked pickups by weekday and two-hour window (Friday is closed), so admins can see when demand is highest.
- **Technician workload**: how many technicians are busy, free or have never been assigned, and the ten who finished the most repairs. These are shown only with name, region and counts, never with contact details.
- **Inspection fees**: fees paid and refunded, fees kept (split into the technician's 90% and Sarabo's 10%, and by reason, either a declined quote or a late cancellation), and refunds that are still pending.

I checked the chart colours for contrast in both light and dark mode. This same report is the only input given to the optional AI summary (Section 8.4), so the summary can never mention a number that the page itself does not show.

6.16 Notifications and Reminders

Sarabo has forty-seven notification types, and each of them has a de-duplication key. They are all defined in a single module, `utils/notificationEvents.js`. Together they cover offers, booking, refunds, inspection, quotes, payment, repair, returns and disputes, warranty, reviews and withdrawals. For each request and pickup slot, the daily job sends one “pickup tomorrow” reminder to the customer and to the assigned technician, both in the app and by email. If the pickup is moved, the reminder is sent again. On the live site this runs every evening at 18:00.

6.17 Damage Image Upload

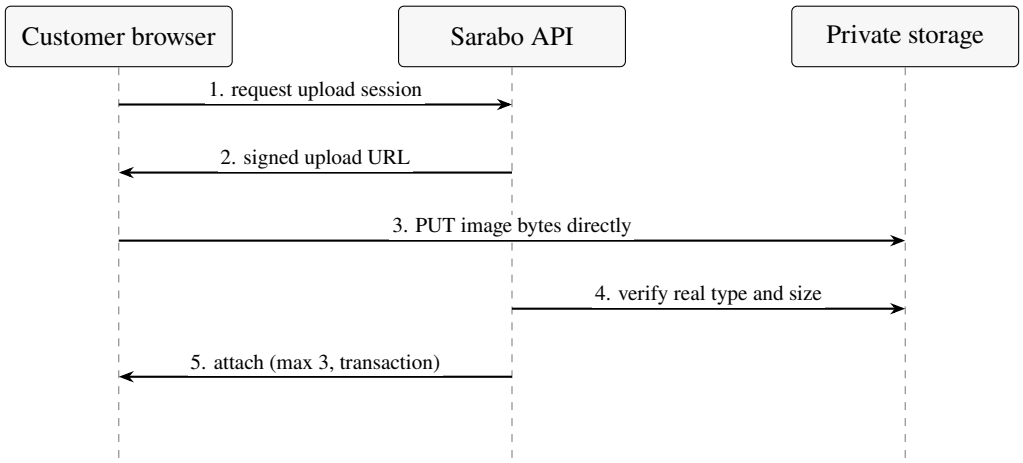


Figure 6.5: Damage-image upload workflow

Chapter 7

System Walkthrough

This chapter shows the finished system in 53 figures (54 screenshots). I took them on 14 October 2026 from the final code, with real Firebase sign-in, real Gemini answers and Stripe in test mode. Section 7.1 shows the public pages. From Section 7.2 onwards, the screenshots follow *one* repair, the cracked Samsung Galaxy A54 used as the example throughout this report, through every state in Figure 6.1. They switch between the customer's and the technician's screens as the case moves along. The last two sections show the technician's own account and the admin pages. Each picture shows the whole browser window exactly as the user saw it, without cropping, two to a page. Personal email addresses are blurred, and any other requests, people and figures that appear in lists come from the seeded development data.

7.1 Public Website

Anyone can use these pages without an account. A visitor can see what Sarabo repairs and what it usually costs, check that their area is covered, ask the AI chat a question, browse the open jobs that technicians can take, and follow a repair with its tracking code. Every page also works on a phone.

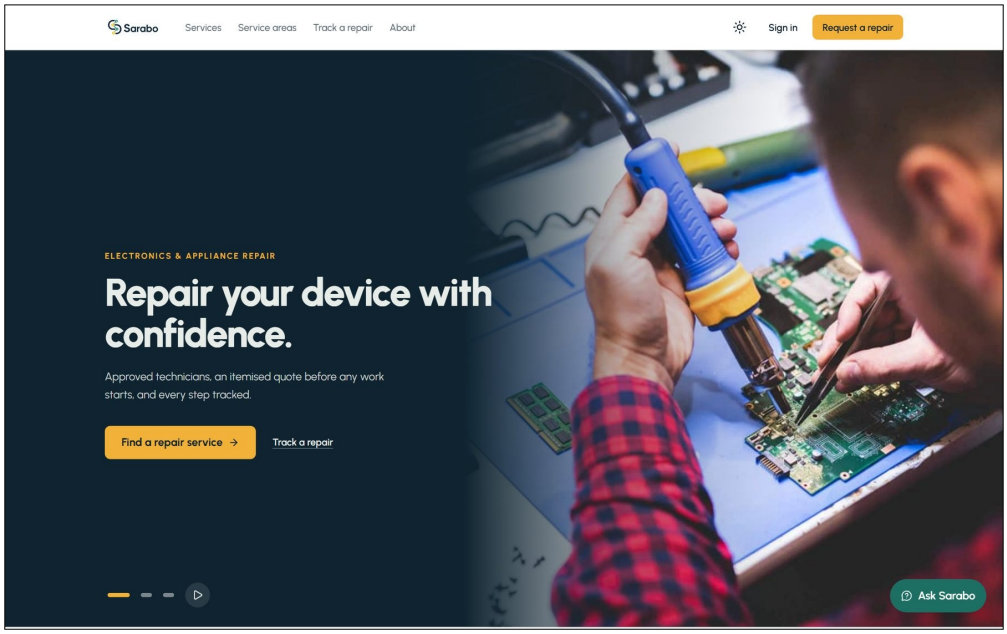


Figure 7.1: The public home page, which is the first screen a visitor sees. The banner sums up what the service promises (approved technicians, an itemised quote before any work starts, and every step tracked) and offers the two main actions, “Find a repair service” and “Track a repair”. The top bar links to services, service areas, tracking and the about page. On every page, the “Ask Sarabo” button in the corner opens the AI support chat.

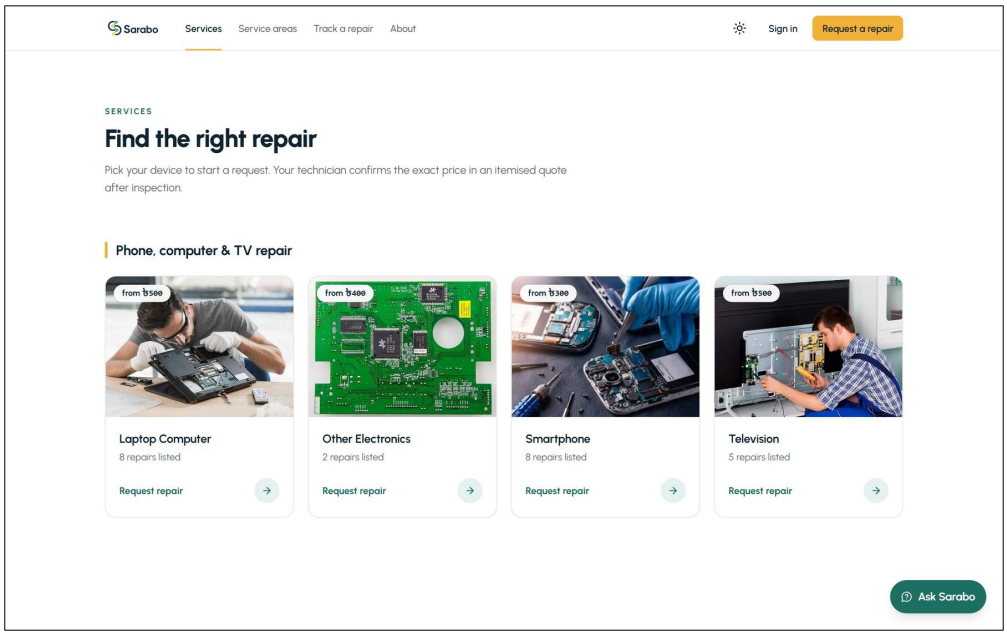


Figure 7.2: The Services page, which anyone can open without an account. Repairs are grouped by device (laptop, other electronics, smartphone, television and more below), and each card shows the lowest catalogue price and how many repairs are listed for that device. The prices come from the live catalogue of 45 services that admins maintain, and the exact price is always confirmed in an itemised quote after inspection.

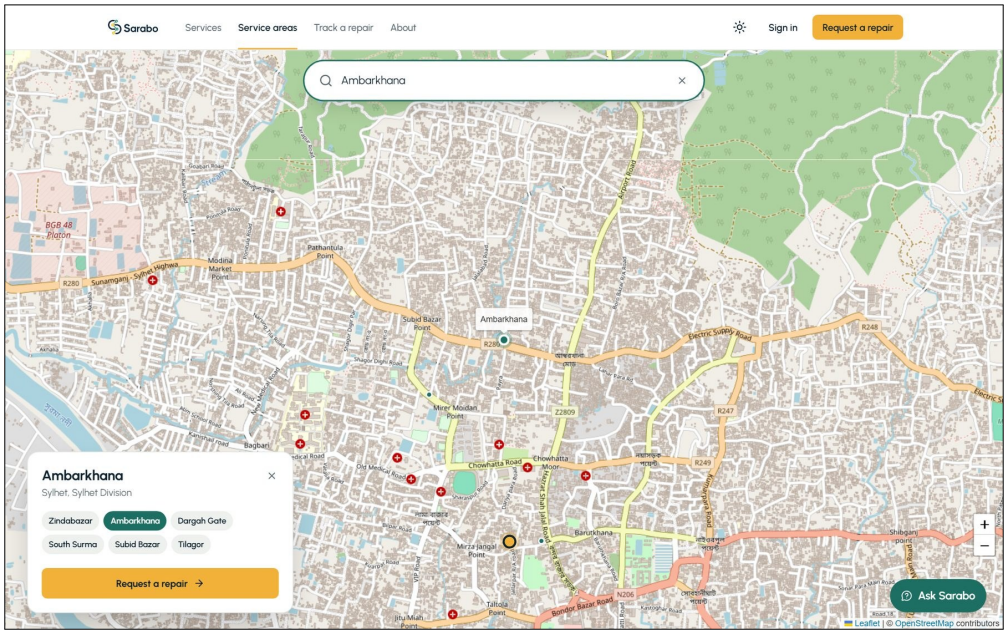


Figure 7.3: The Service areas page. Searching "Ambarkhana" moves the OpenStreetMap map to that part of Sylhet, and the card at the bottom left lists the other covered sub-areas of the city with a direct "Request a repair" button. A visitor can check that Sarabo serves their address before creating an account (Section 6.13).

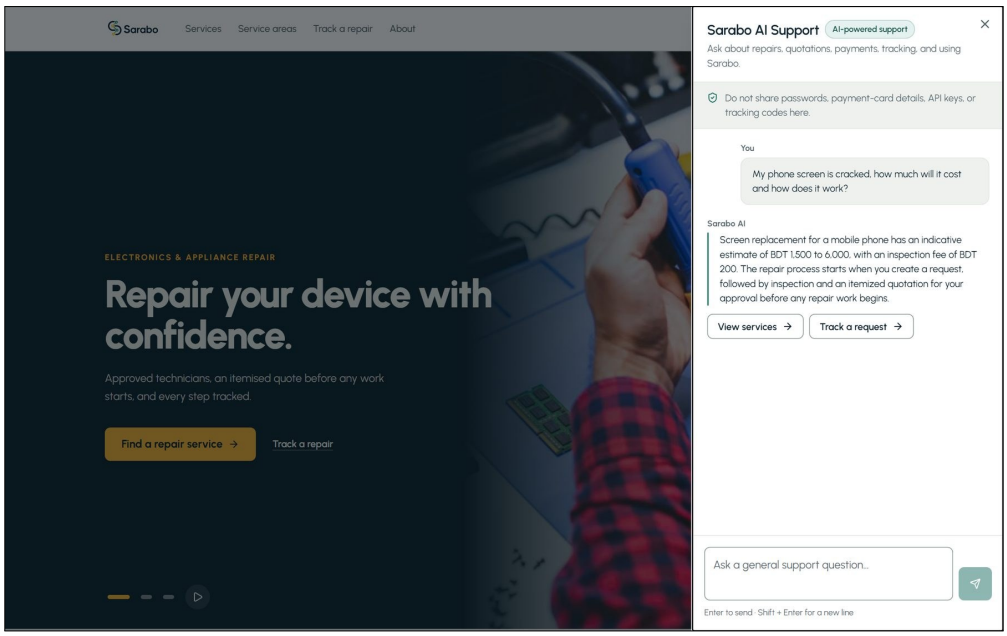


Figure 7.4: The AI support chat ("Ask Sarabo"), opened over the home page. A visitor asks what it costs to fix a cracked phone screen. The answer gives the 1,500–6,000 BDT estimate and the 200 BDT inspection fee from the live catalogue and explains the steps. The two buttons under the answer come from a fixed list, and a notice warns users not to share passwords or card details (Chapter 8).

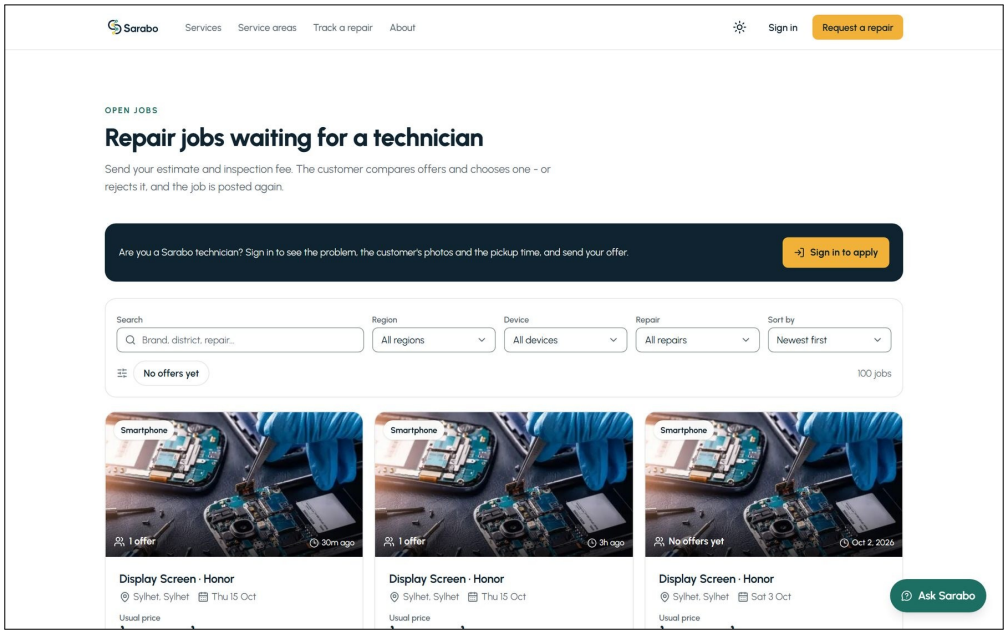


Figure 7.5: The public jobs board lists every open repair job, with search, region, device and repair filters and a "No offers yet" shortcut. Each card shows only a summary with nothing private in it (device, repair, district, pickup day, usual price and number of offers), never the customer's name, address or contact details. Technicians sign in from the banner to see the whole job and send an offer.

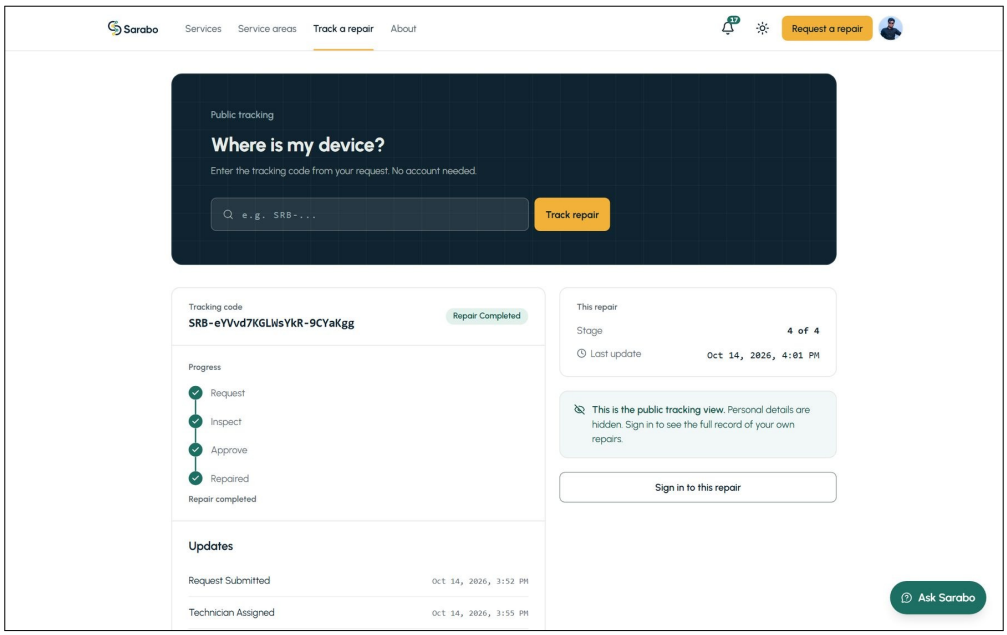


Figure 7.6: The public tracking page for the walkthrough repair, opened with just its tracking code and no account. It shows only the four stages (all complete here), the current status, the last update time and a time-stamped list of updates. Personal and payment details are hidden, and the owner can sign in from the button on the right to see the full record.

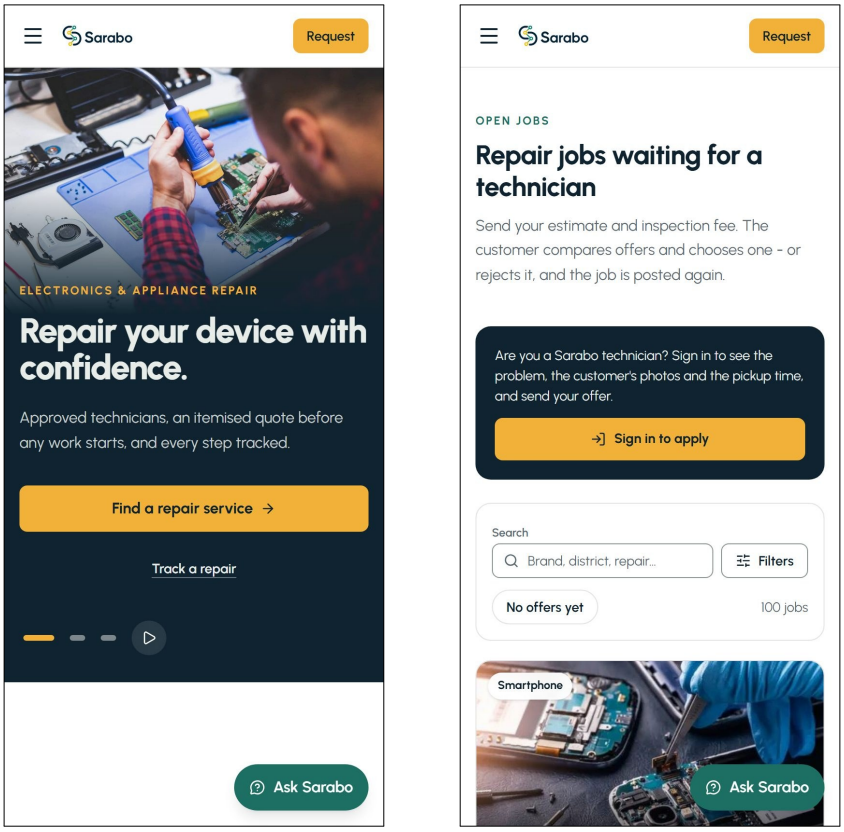


Figure 7.7: The same public pages on a phone, 393 pixels wide: the home page (left) and the jobs board (right). The layout changes for small screens: the menu folds into the button at the top left, the main actions become full-width buttons, and the job filters move behind a "Filters" button. Every page in this chapter works the same way on a phone.

7.2 Creating the Request

From here the chapter follows one repair. Rahim, a customer in Sylhet, dropped his Samsung Galaxy A54: the screen cracked, a green line appeared and the touch stopped working. He signs in, describes the problem in his own words and lets the AI fill in the form, then chooses where and when the phone should be collected.

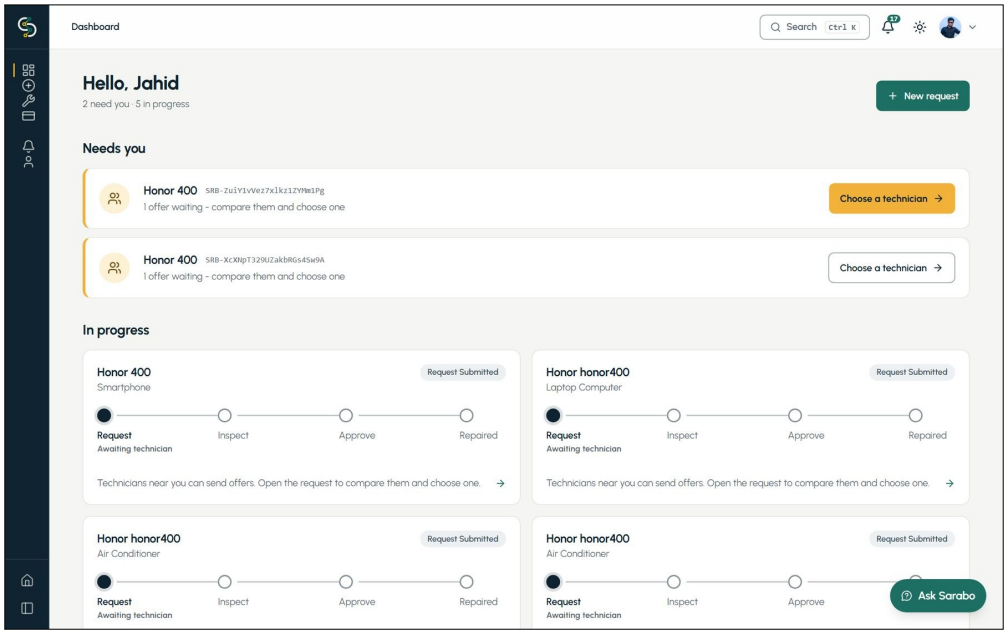


Figure 7.8: The customer's dashboard after signing in. "Needs you" comes first and lists the requests that are waiting on the customer (here, two requests with an offer waiting), each with a button for the next action. Below it, "In progress" shows every other open request with the four-stage progress bar. "New request" at the top right starts the request form.

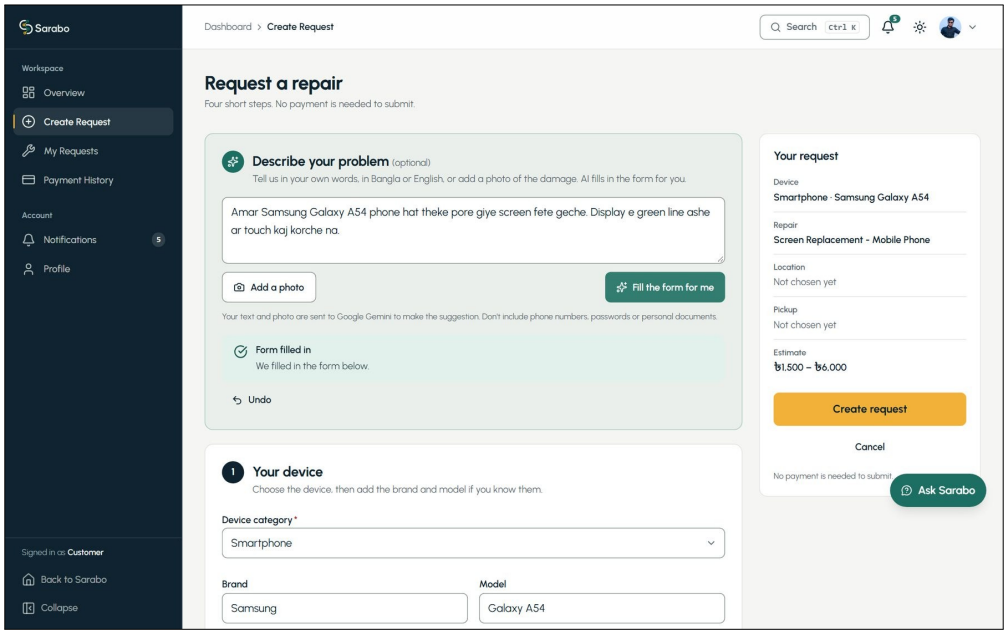


Figure 7.9: Step one of the request form with the AI request helper. Rahim typed the problem in Banglish ("the phone fell from my hand, the screen cracked, a green line, touch not working"). Pressing "Fill the form for me" set the device, the catalogue service "Screen Replacement – Mobile Phone", the brand, the model and an English description. The summary on the right already shows the 1,500–6,000 BDT estimate, and "Undo" restores the previous values.

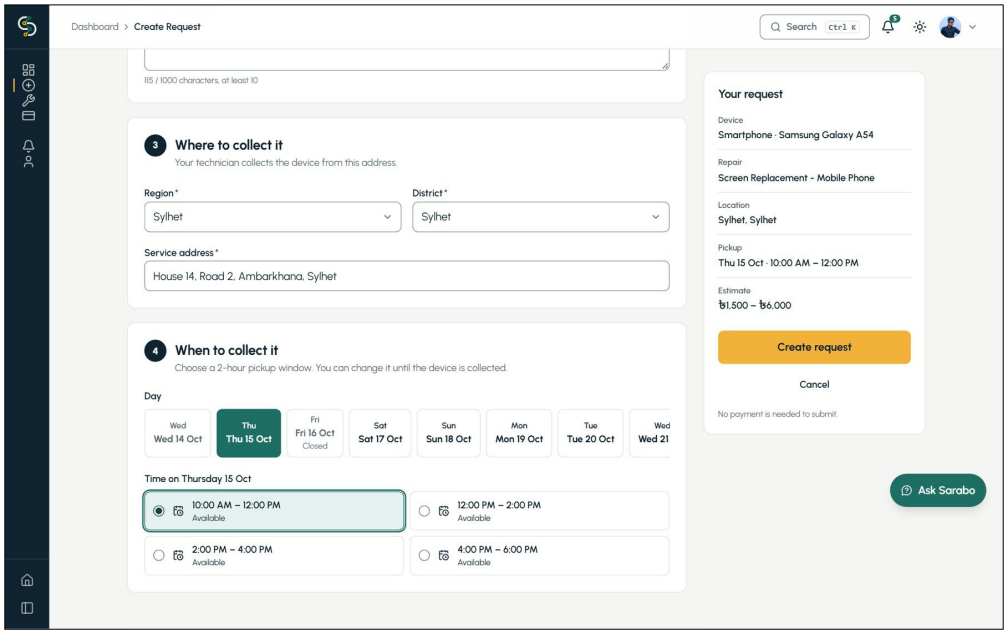


Figure 7.10: Steps three and four of the request form: the pickup address (region, district and street address) and a two-hour pickup window. Friday is shown as closed, and each window shows whether it is still available. The summary on the right now holds every detail of the request, and no payment is needed to submit it.

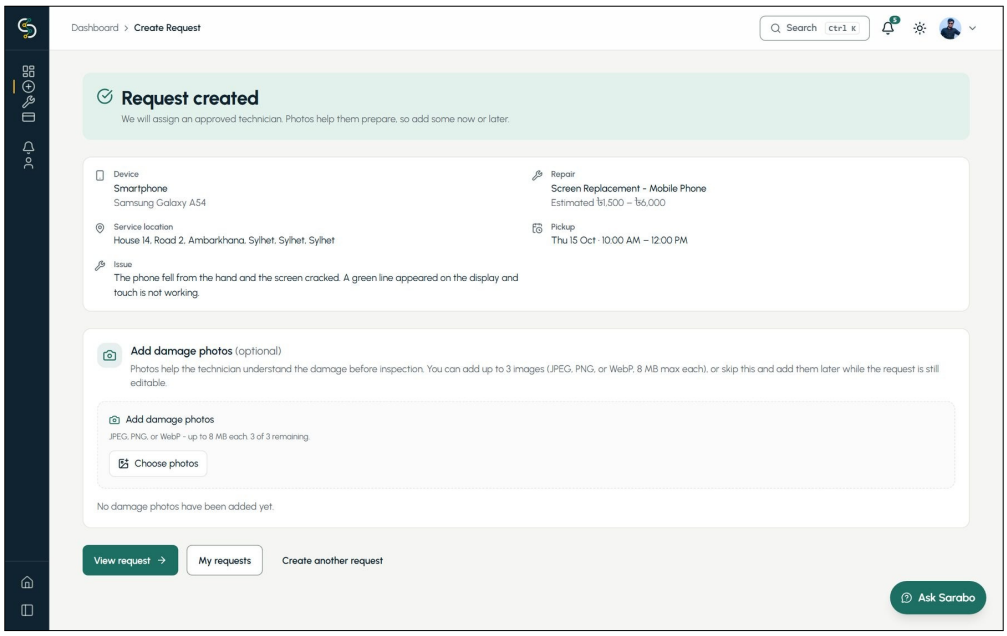


Figure 7.11: The confirmation after the request is submitted. It repeats the device, repair and estimate, the service address, the pickup window and the problem description, and offers to add up to three damage photos now or later. The request goes out to matching technicians straight away. Photos are optional, and they are sent only when the customer presses “Upload”.

7.3 Receiving an Offer

The request is posted on the jobs board, and technicians in the same region who can do the repair are notified. One of them sends an offer with an estimate, an inspection fee and a note, and Rahim sees it on his request with the reasons it is ranked where it is.

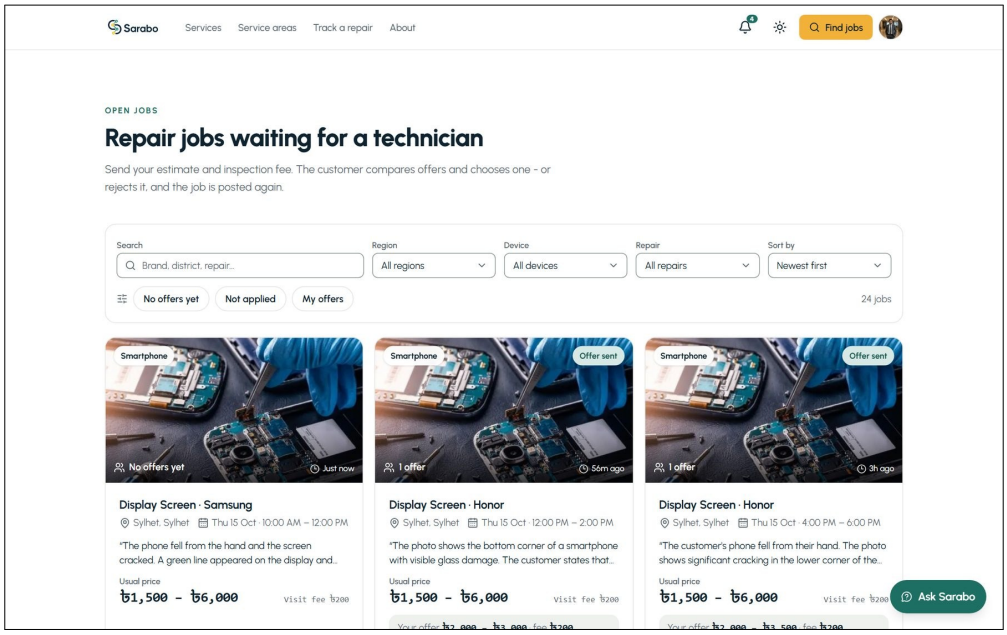


Figure 7.12: The jobs board as a signed-in technician sees it. Beside the search and the region, device and repair filters, the shortcuts “No offers yet”, “Not applied” and “My offers” help the technician find work. Jobs they have already made an offer on carry an “Offer sent” badge and show that offer. Rahim’s new Samsung job is first, posted “Just now”, with its pickup window and usual price.

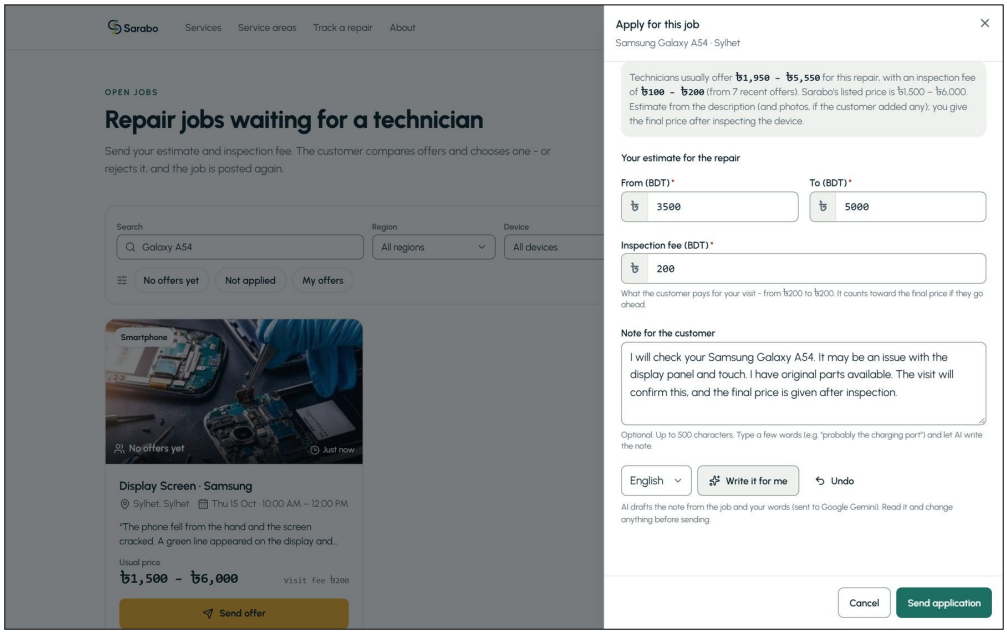


Figure 7.13: A technician applying for Rahim's job. The grey box is the price guide, built from 7 recent offers for the same repair and shown next to Sarabo's listed price. The technician enters an estimate of 3,500–5,000 BDT and a 200 BDT inspection fee, which is within the allowed range. The AI wrote the note from a few words ("Write it for me"), and the technician can edit it before sending.

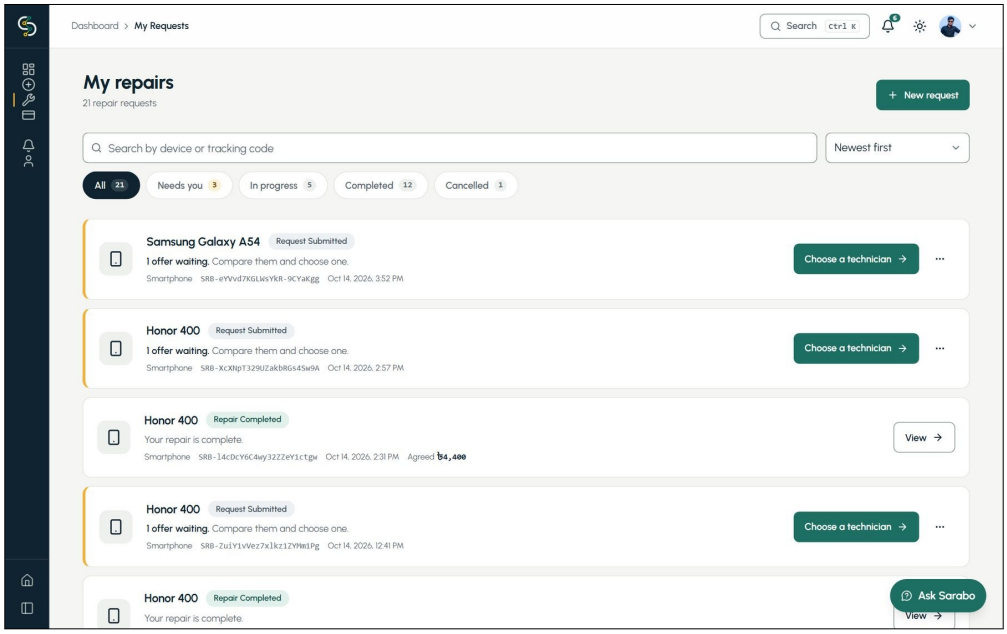


Figure 7.14: The customer's "My repairs" page, with search, sorting and status tabs (All, Needs you, In progress, Completed, Cancelled) and a count on each. Rahim's Samsung Galaxy A54 is at the top, marked "1 offer waiting" and with a "Choose a technician" button. Completed repairs show the agreed price instead.

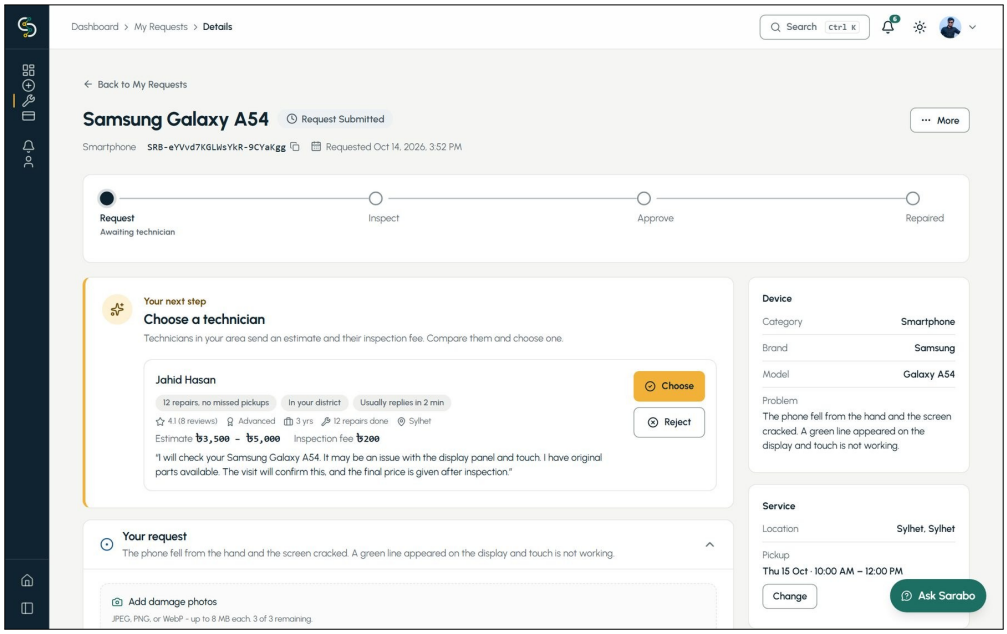


Figure 7.15: Rahim's request page after the offer arrives. The progress bar at the top shows the request is still at the first stage, and the "Your next step" panel asks him to choose a technician. The offer shows why it is ranked where it is (12 repairs with no missed pickups, in the same district, usually replies in 2 minutes), along with the rating, level, experience, estimate, fee and note, and "Choose" and "Reject" buttons (Section 6.4).

7.4 Choosing and Booking

Rahim chooses the offer. Paying the inspection fee through Stripe books the technician for the pickup window and closes the other offers. The fee is taken off the final price later.

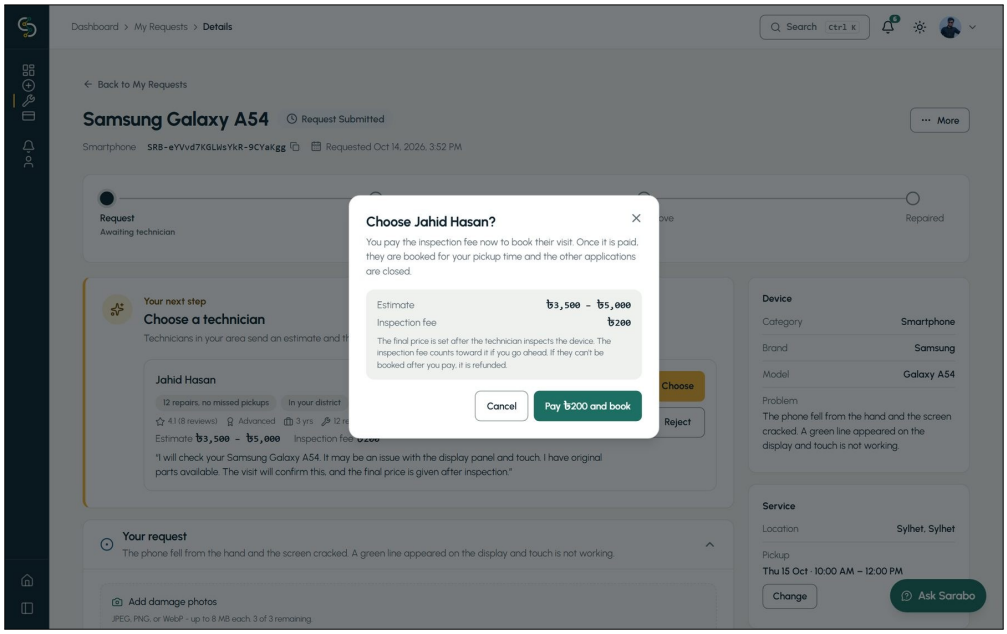


Figure 7.16: The confirmation Rahim sees when he chooses the offer. It repeats the estimate and the 200 BDT inspection fee and explains that paying the fee books the visit and closes the other offers. "Pay 200 BDT and book" opens Stripe Checkout. If the booking fails after he has paid, the fee is refunded automatically.

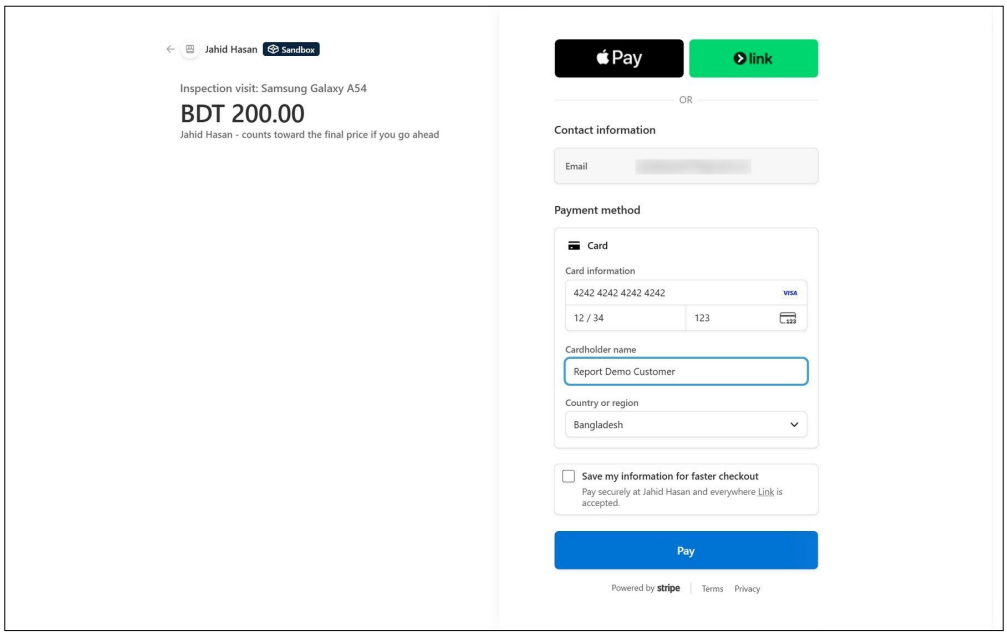


Figure 7.17: Stripe's hosted checkout page for the 200 BDT inspection fee, in test mode (the "Sandbox" badge). The left side names the item ("Inspection visit: Samsung Galaxy A54") and says the fee counts toward the final price. The card shown is Stripe's public test card. Card details are typed only on Stripe's page and never reach Sarabo's server.

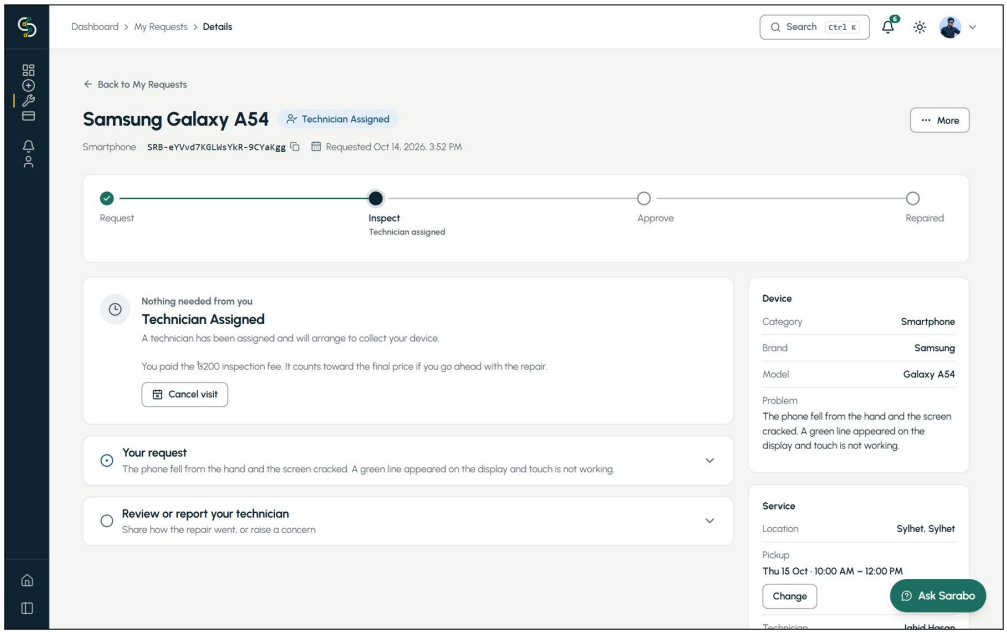


Figure 7.18: Rahim's request after the fee is paid. The progress bar has moved to "Inspect" with the status "Technician assigned", and the panel says nothing is needed from him while the technician arranges the pickup. It also confirms that the 200 BDT fee counts toward the final price. "Cancel visit" follows the cancellation rules in Table 6.2, and "Change" on the right moves the pickup window.

7.5 Pickup, Inspection and Quote

The technician collects the phone, records its condition, inspects it and writes an itemised quote. Nothing is charged until Rahim has seen the findings and the price and approved them.

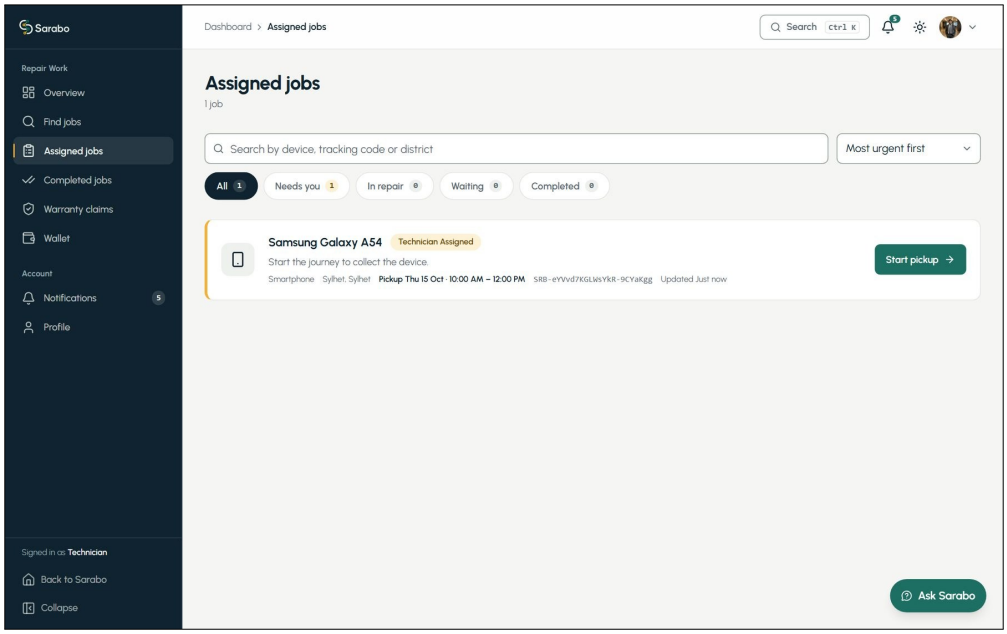


Figure 7.19: The technician's "Assigned jobs" list, with search, "Most urgent first" sorting and tabs for jobs that need the technician, are in repair, are waiting or are completed. Rahim's job shows the pickup window and the tracking code, and the next action sits on the job itself. "Start pickup" tells the customer that the technician is on the way.

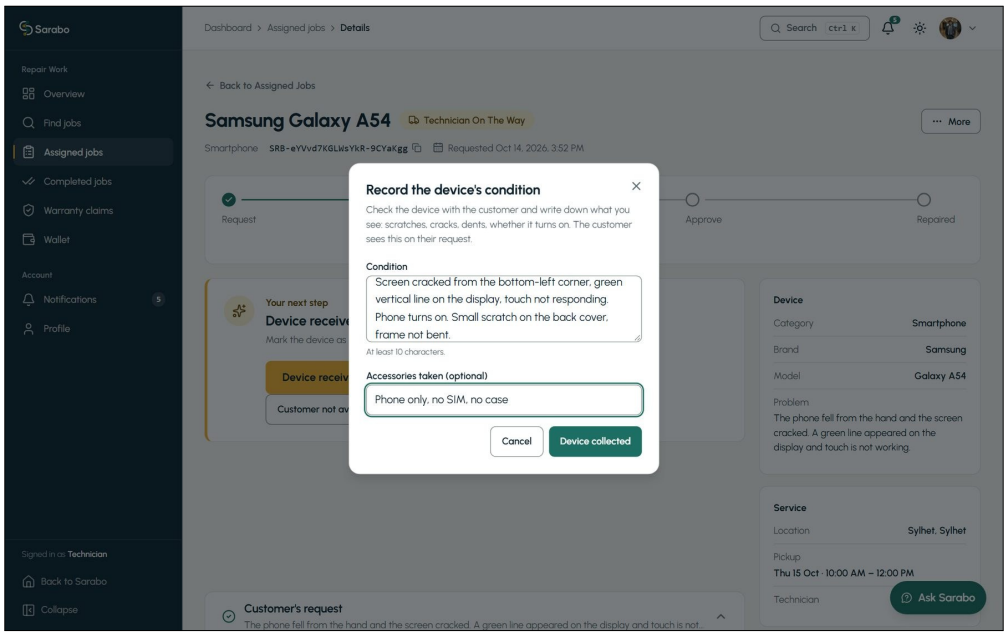


Figure 7.20: The technician's view at pickup. Before taking the phone, the technician writes down its condition (a cracked screen, a green line, touch not responding, a small scratch on the back) and the accessories taken (phone only). The customer sees this record on his request and can dispute it, which protects both sides if there is a disagreement later (Section 6.8).

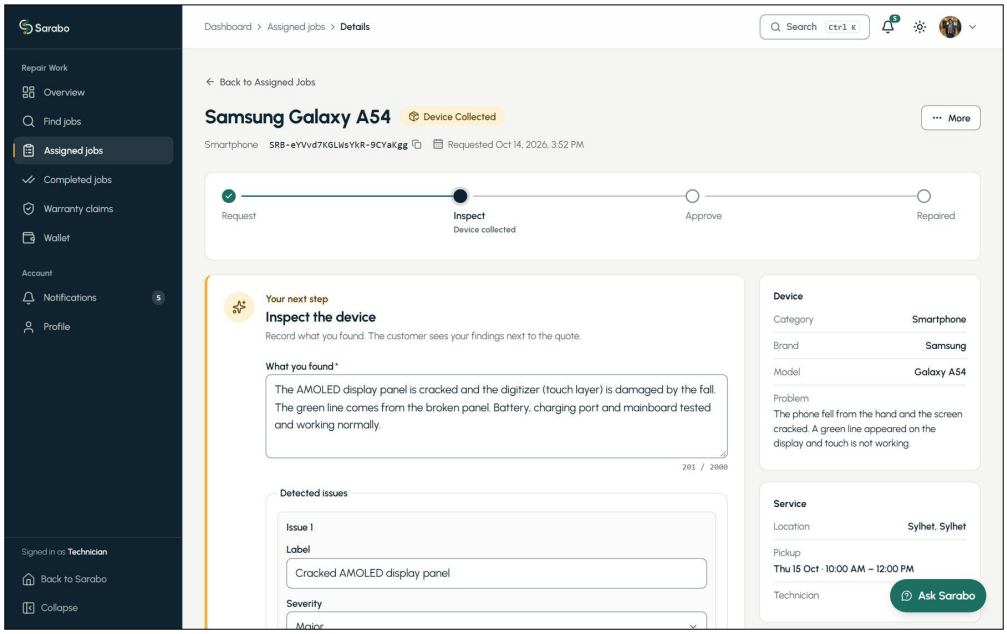


Figure 7.21: The inspection step on the technician's job page, after the device was collected. The technician records what was found (a cracked AMOLED panel and a damaged touch layer, with the battery, charging port and mainboard working) and lists each issue with a severity. The customer sees these findings next to the quote. If the result had been "not repairable" or "not economical", either side could have closed the request.

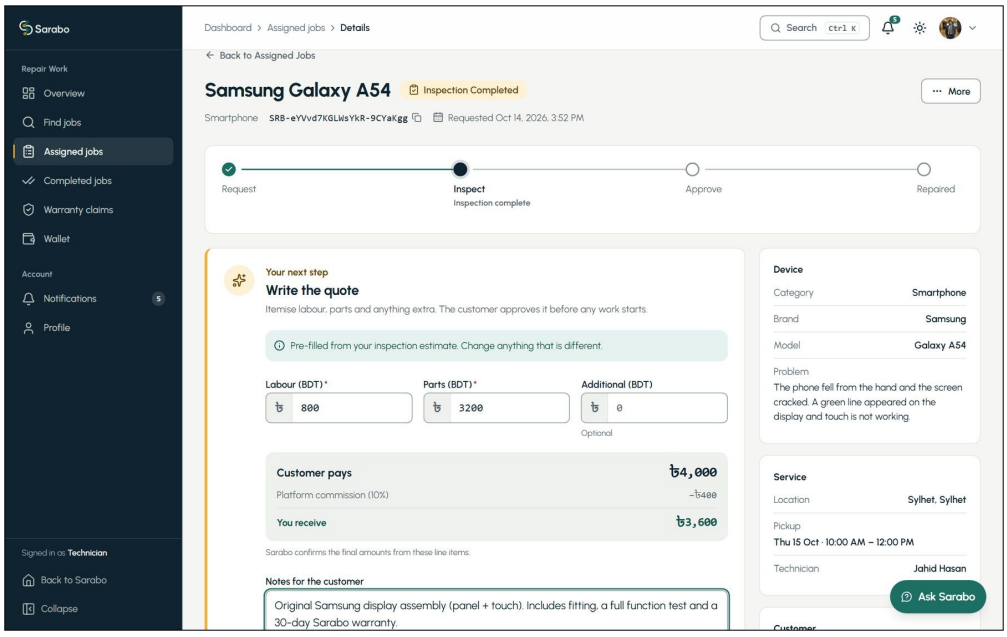


Figure 7.22: The technician writes an itemised quote: 800 BDT labour and 3,200 BDT parts, 4,000 BDT in total. The box below shows the 10% platform commission (400 BDT) and the 3,600 BDT the technician will receive, and the note tells the customer what the price includes. The server works out the final amounts from these line items itself.

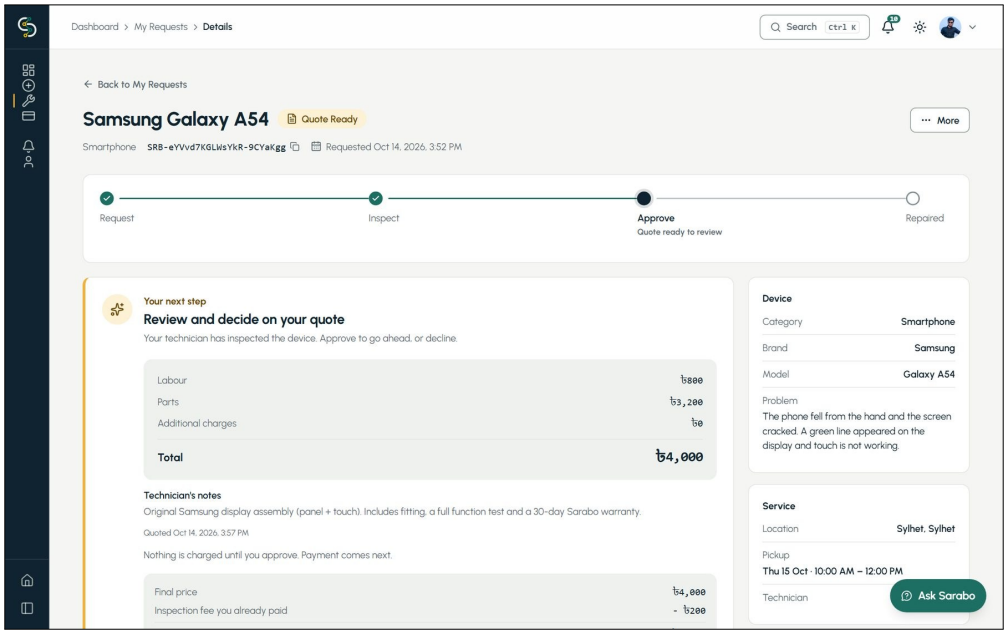


Figure 7.23: Rahim's view of the quote. The progress bar has moved to "Approve", and the panel lists labour, parts and extra charges with the 4,000 BDT total and the technician's notes. Below that, the 200 BDT inspection fee he already paid is taken off the final price. Nothing is charged until he approves.

7.6 Payment

Rahim approves the quote and pays the rest by card. Each payment gets a PDF receipt and appears in his payment history.

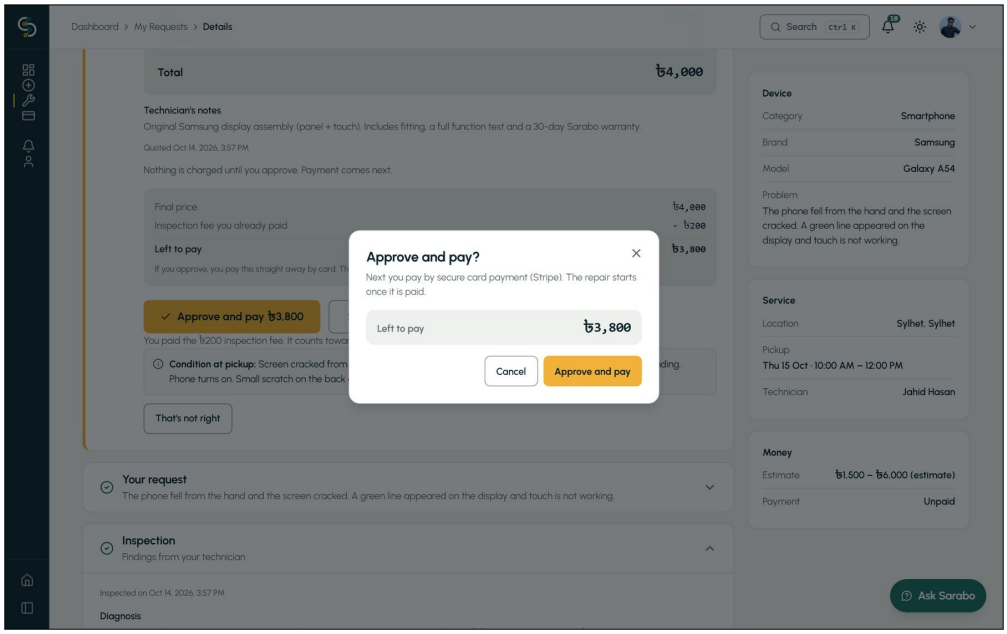


Figure 7.24: Rahim approves the quote. Behind the dialog, the price summary shows the 4,000 BDT final price minus the 200 BDT already paid, which leaves 3,800 BDT. The dialog repeats that amount and says the repair will start once it is paid. "Approve and pay" goes straight to a Stripe card payment, and "That's not right" on the page lets him dispute the pickup record instead.

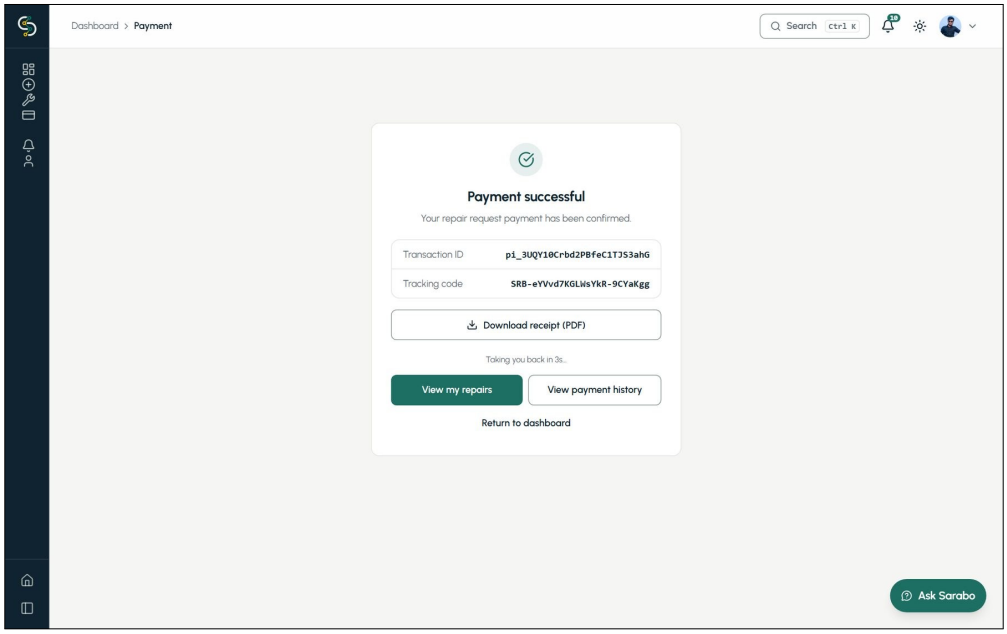


Figure 7.25: The page shown after Rahim pays the remaining 3,800 BDT through Stripe. It shows the Stripe transaction ID and the request's tracking code, and "Download receipt (PDF)" gives him a receipt generated by the server. The buttons take him to his repairs, his payment history or the dashboard.

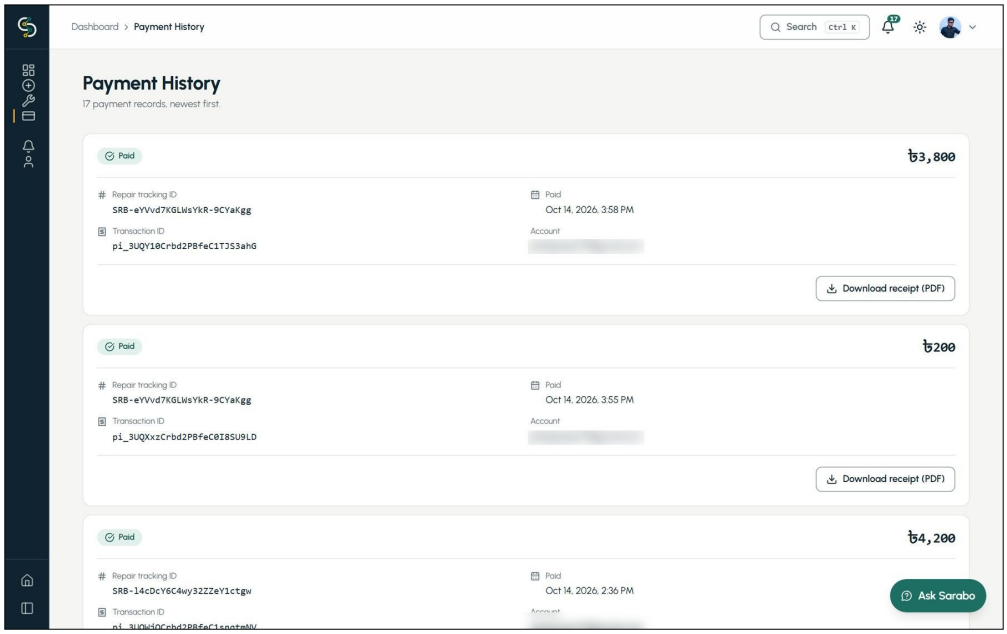


Figure 7.26: The customer's Payment History, newest first. The walkthrough repair has two rows: the 3,800 BDT repair payment and, below it, the 200 BDT inspection fee, each with the tracking code, the Stripe transaction ID, the time and a PDF receipt. The other rows are from earlier test repairs.

7.7 Repair and Hand-Over

The technician repairs the phone, posting updates that Rahim can see, then returns it. Rahim confirms he has it back, which releases the technician's earning and starts the warranty, and rates the technician.

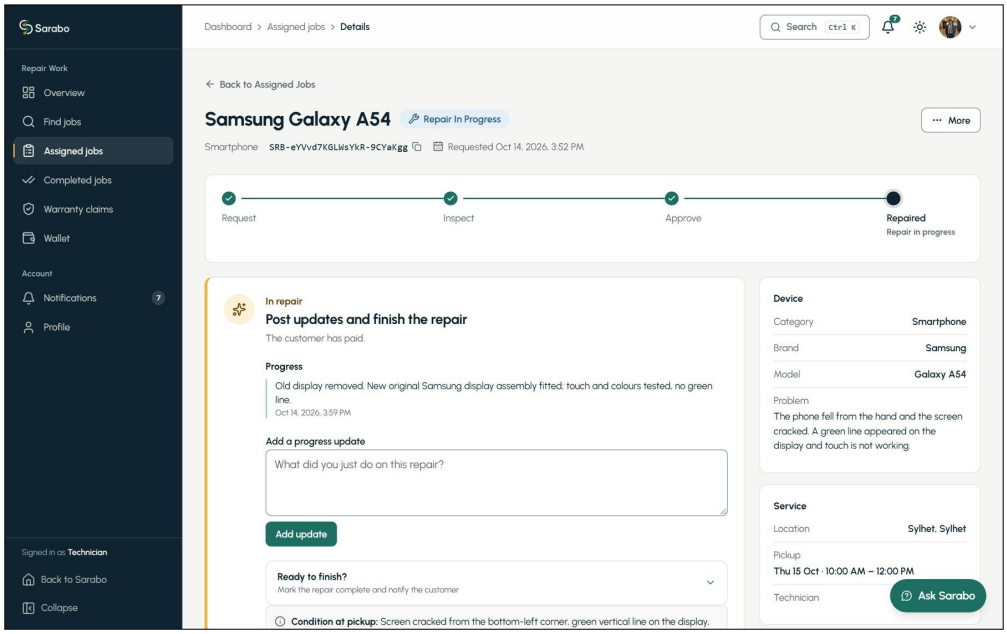


Figure 7.27: The technician's job page once the customer has paid. The progress bar is at the last stage, "Repaired – repair in progress", and the panel asks the technician to post updates and finish the repair. Rahim can already see the first update (old display removed, new original display fitted and tested) with its time.

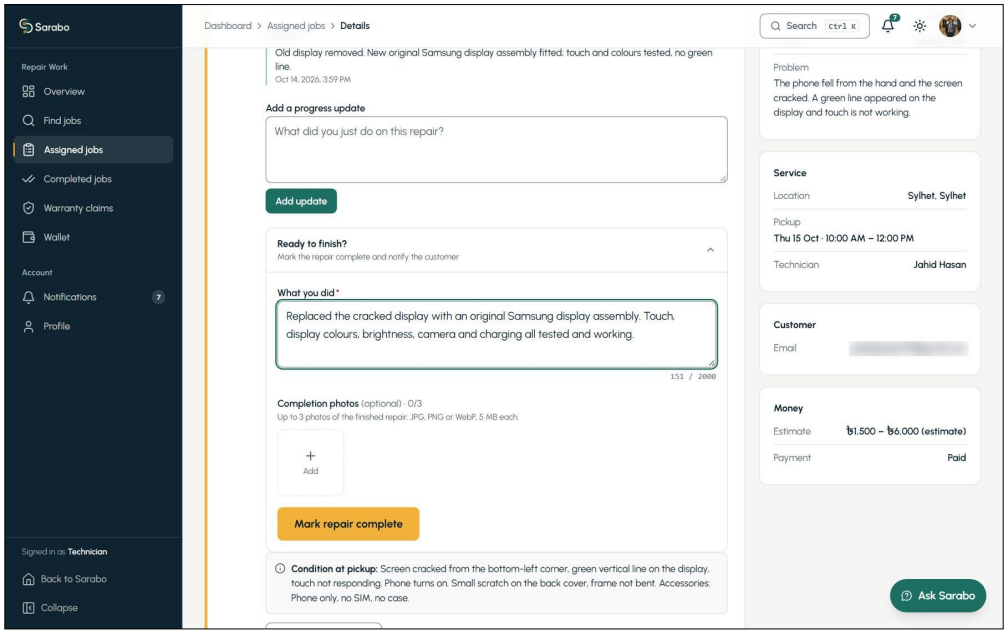


Figure 7.28: The same page further down, when the work is done. Under "Ready to finish?" the technician describes the work done and can add up to three completion photos before pressing "Mark repair complete". The condition recorded at pickup stays visible at the bottom, and the right-hand panel shows the request is paid.

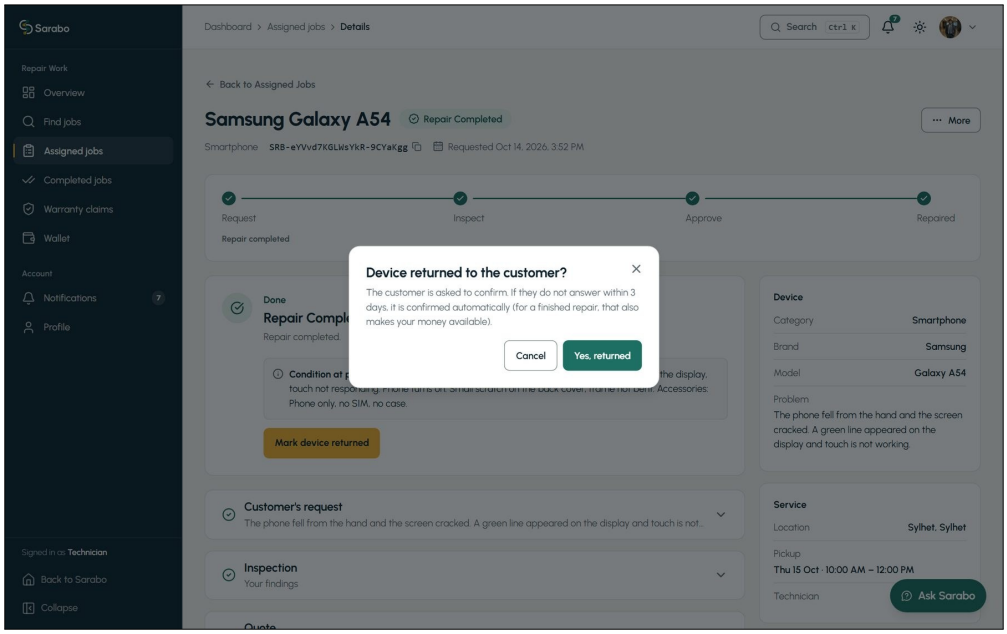


Figure 7.29: After returning the phone, the technician marks it returned. The dialog explains that the customer is asked to confirm, and that if he does not answer within 3 days it is confirmed automatically, which also makes the technician's money available. This way, no step of the repair can be left stuck waiting for one person.

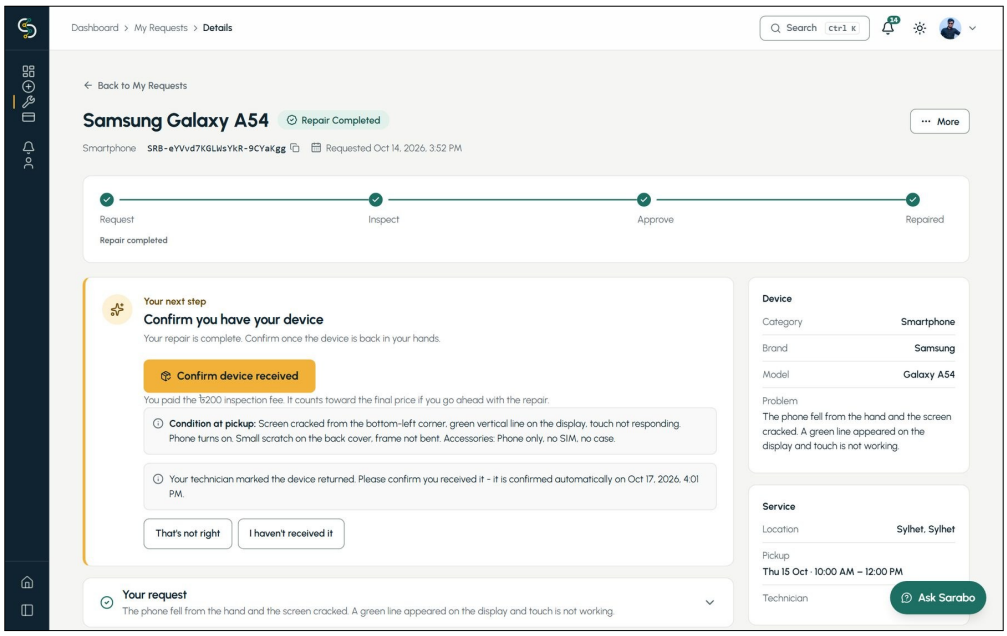


Figure 7.30: Rahim's request page after the technician marked the phone as returned. All four stages are complete, and he is asked to confirm that he has the device. If he does nothing, it is confirmed automatically after 3 days (17 October). "That's not right" and "I haven't received it" send the case to an admin. The technician's earning only becomes available after confirmation.

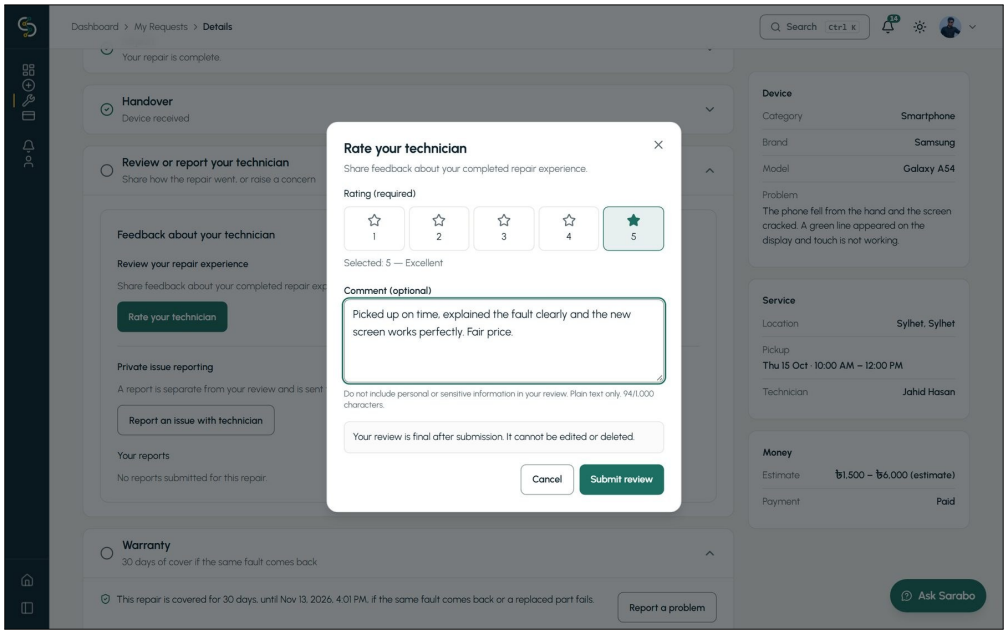


Figure 7.31: Once the device is back, Rahim rates the technician from 1 to 5 and can add a comment (here 5, “Excellent”). The dialog warns that a review is final and cannot be edited or deleted, which keeps ratings honest. Sending the admins a private report about the technician is a separate action, visible behind the dialog.

7.8 Warranty

Ten days later the touch fails again. Rahim reports it, the technician books a free re-inspection, finds that the part he fitted is faulty and repairs it again for free. Rahim then confirms that it works. Every step has a deadline, and Rahim is notified each time.

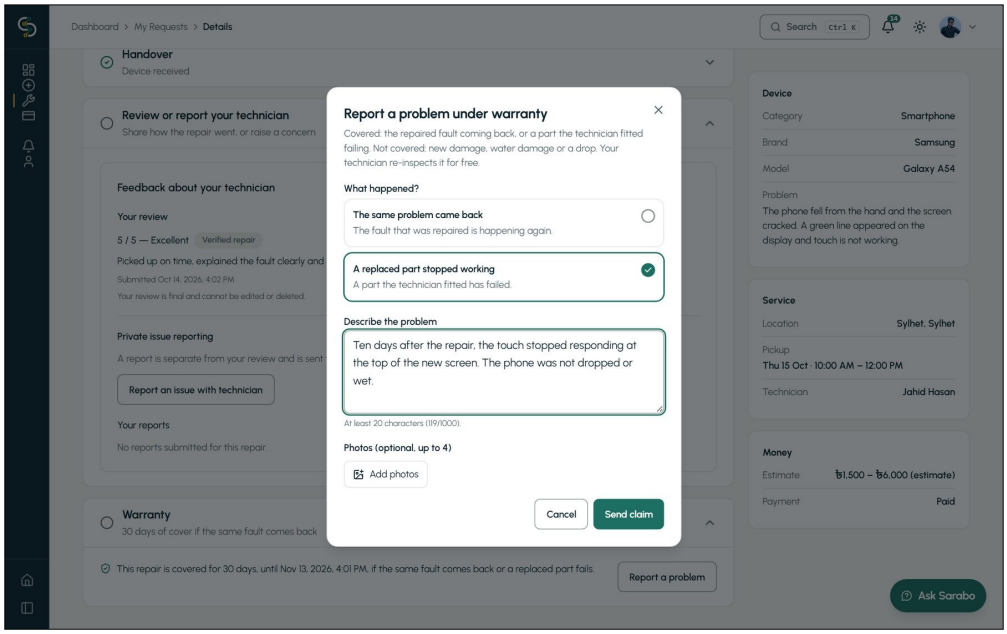


Figure 7.32: Ten days later the touch fails again, so Rahim reports a problem under the 30-day warranty. The dialog explains what is covered (the same fault coming back or a fitted part failing) and what is not (new damage, water or a drop). He chooses "A replaced part stopped working", describes the problem and can add up to four photos. The warranty card behind the dialog shows that the cover lasts until 13 November.

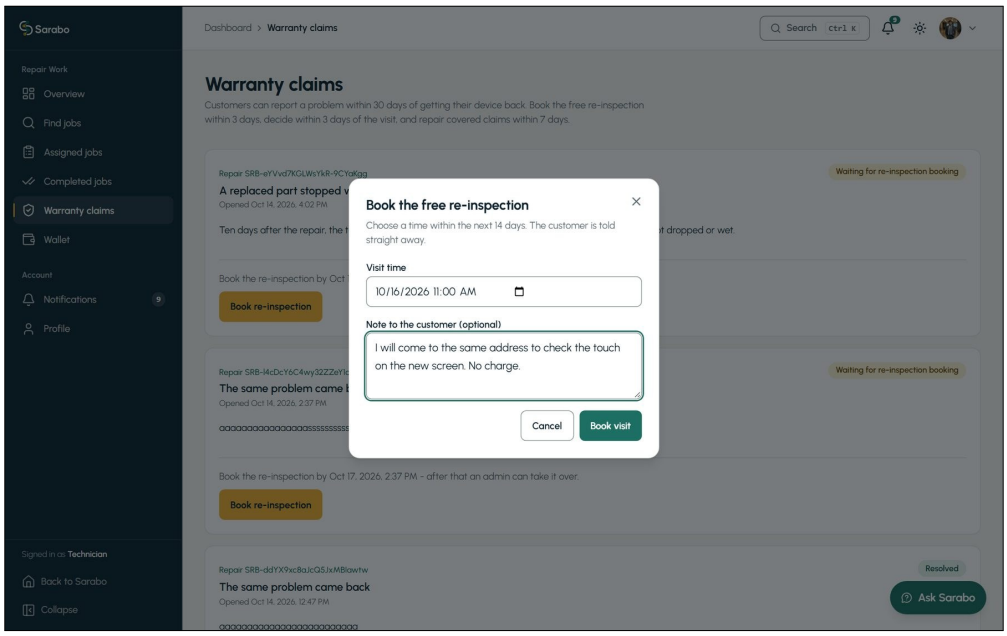


Figure 7.33: The technician's Warranty claims page. The header lists the deadlines: book the free re-inspection within 3 days, decide within 3 days of the visit, and finish a covered repair within 7 days. If a deadline is missed, an admin can take the claim over. Here the technician books a visit for 16 October at 11:00 with a note for Rahim (the other claims in the list are test data).

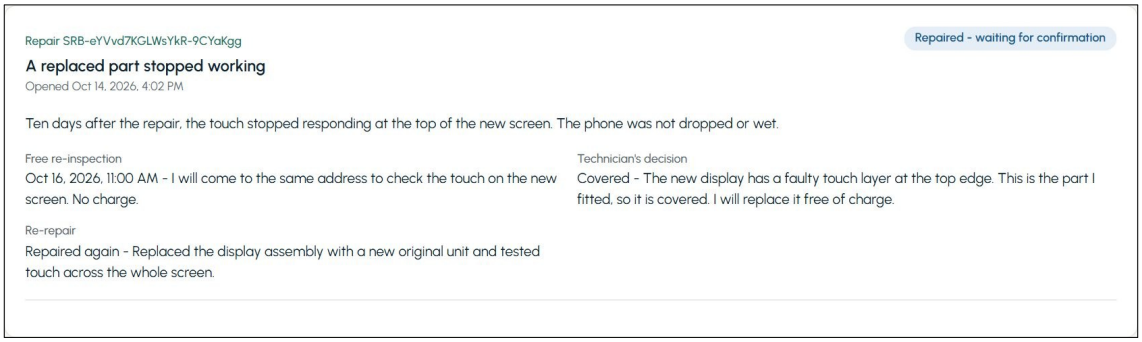


Figure 7.34: The claim card on the technician's side after the visit. It records the free re-inspection appointment, the decision "Covered" with its reason (the faulty touch layer is in the part he fitted) and the re-repair. The status "Repaired – waiting for confirmation" means it is now up to Rahim to say whether it works.

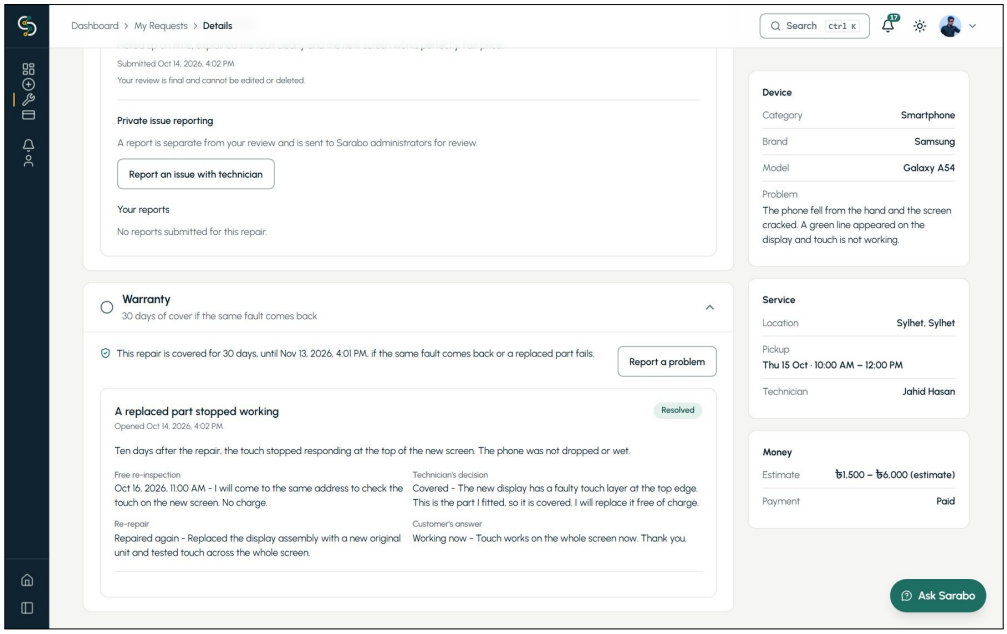


Figure 7.35: The warranty claim after it was resolved, as Rahim sees it. It records each step in order: the free re-inspection appointment, the technician's decision ("Covered"), the re-repair, and Rahim's answer "Working now". If the re-repair had not worked, an admin would have refunded the repair.

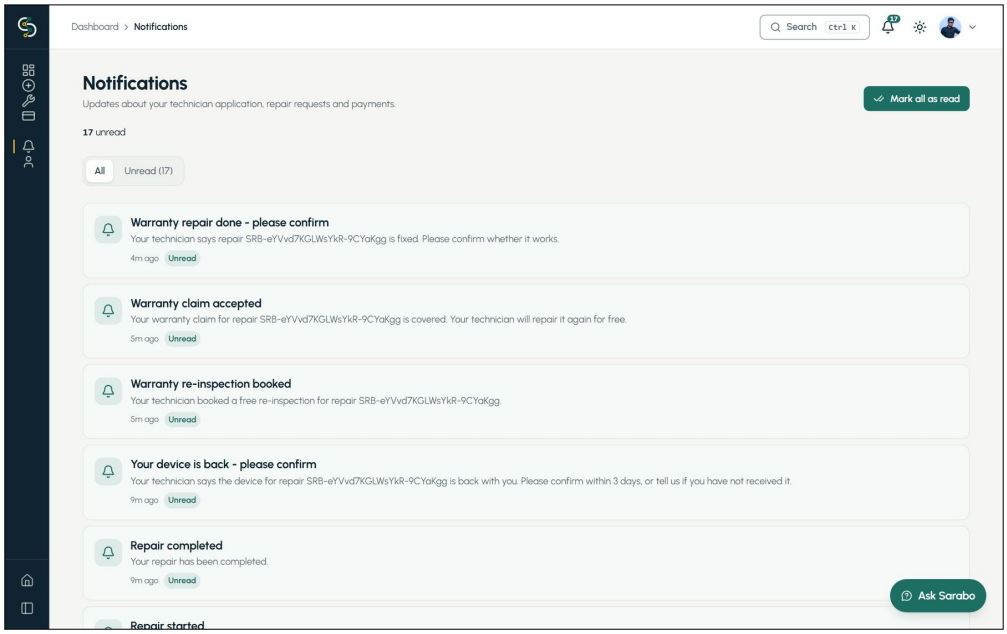


Figure 7.36: Rahim's notifications page. He got a notification at every step of the repair, listed newest first: here the warranty repair, the accepted claim and the booked re-inspection, then "Your device is back" and "Repair completed". Each one names the tracking code, and "Mark all as read" clears the unread count.

7.9 The Technician's Account

Besides the job pages, a technician has an overview, a wallet for withdrawing earnings, a profile and a list of completed jobs.

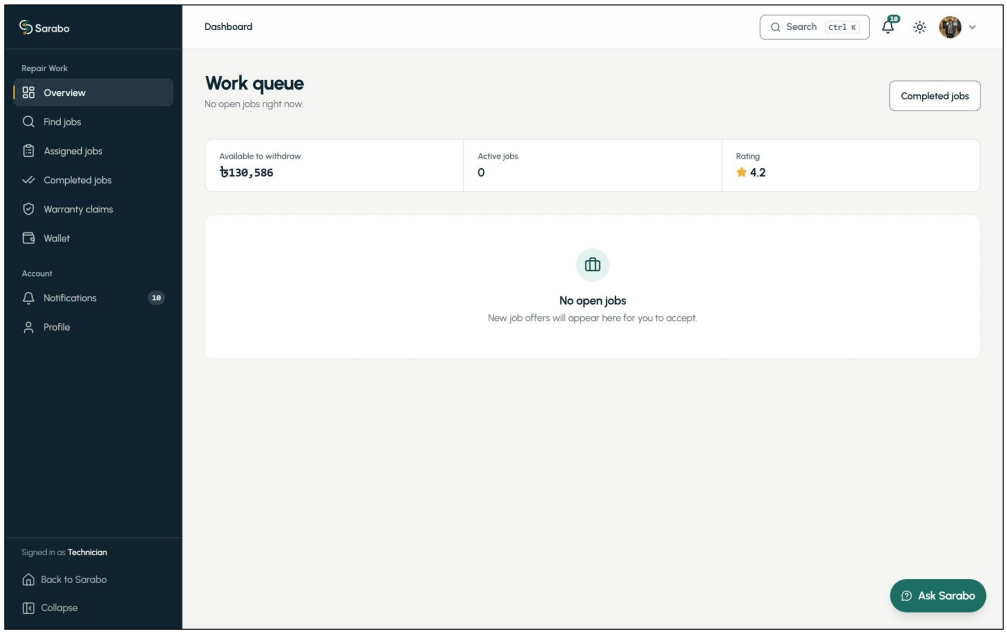


Figure 7.37: The technician's "Work queue", the first page after signing in. Three figures summarise the account: the money available to withdraw, the number of active jobs and the average rating (4.2). The queue below is empty because the walkthrough job is finished. New jobs that need an action will appear here.

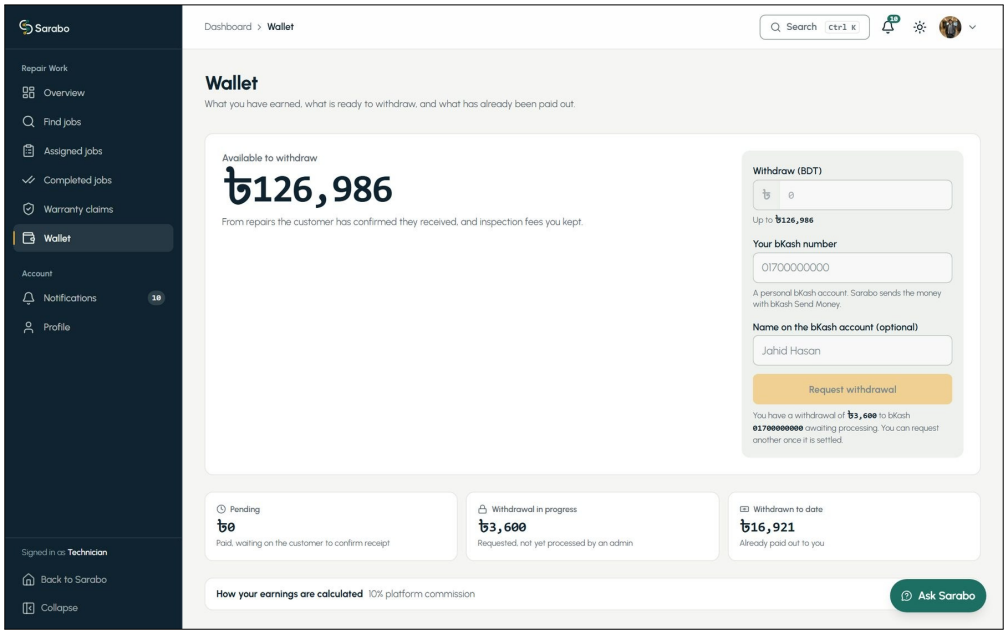


Figure 7.38: The technician's wallet. The large number is the money available to withdraw, made up of earnings from repairs the customer has confirmed plus kept inspection fees. The cards below show pending money, the withdrawal in progress and the total paid out so far. The 3,600 BDT from this repair has been requested as a bKash withdrawal. Balances are always calculated from settlements and withdrawals and are never stored.

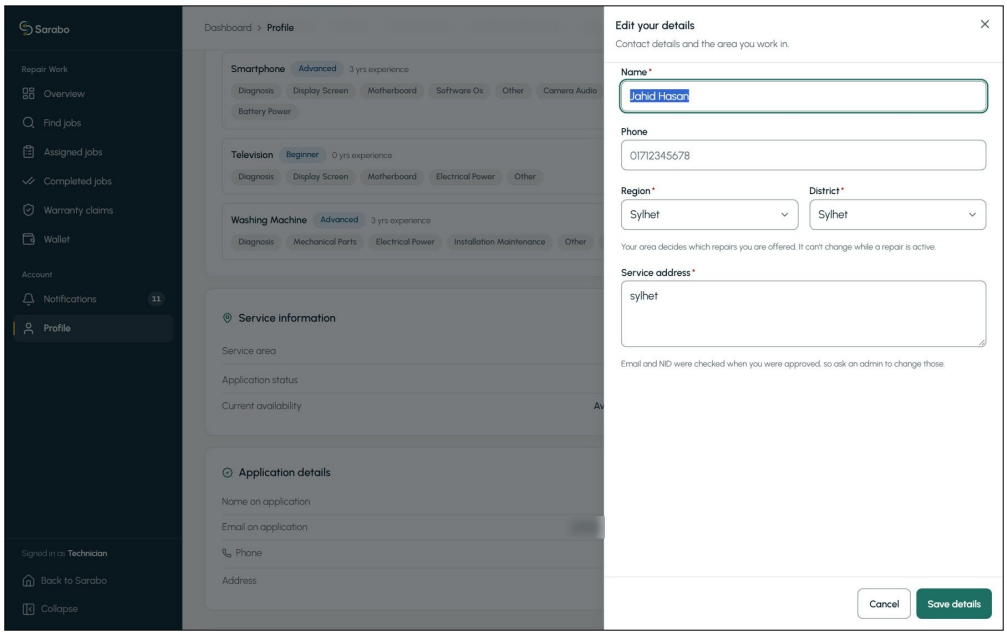


Figure 7.39: A technician editing their own details: name, phone, region, district and service address. The area decides which repairs the technician is offered, so it cannot be changed while a repair is active. The email and NID were checked at approval, so only an admin can change them. Behind the panel, the profile lists the technician's expertise by device, level and experience.

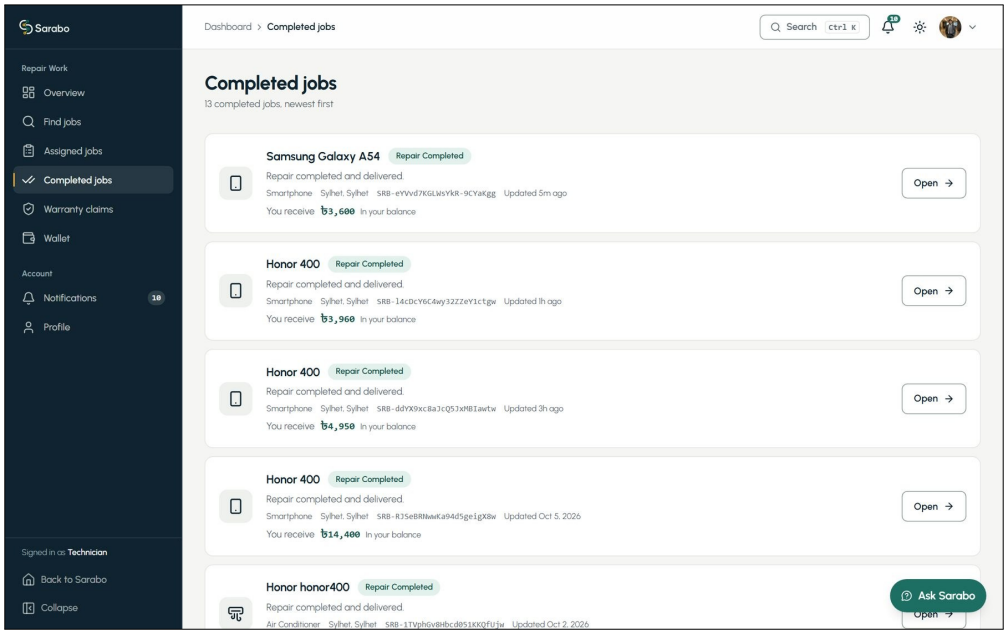


Figure 7.40: The technician's completed jobs, newest first. Each job shows the device, the area, the tracking code and "You receive", the amount after the 10% commission (3,600 BDT for the walkthrough repair), with "In your balance" once the customer has confirmed receipt.

7.10 Administration

Admins run the platform. Their overview shows the problems that need a person first, and from there they manage requests and technicians, the service catalogue, reports, payouts, reviews and user roles. Most of the numbers in these screens come from the seeded development data.

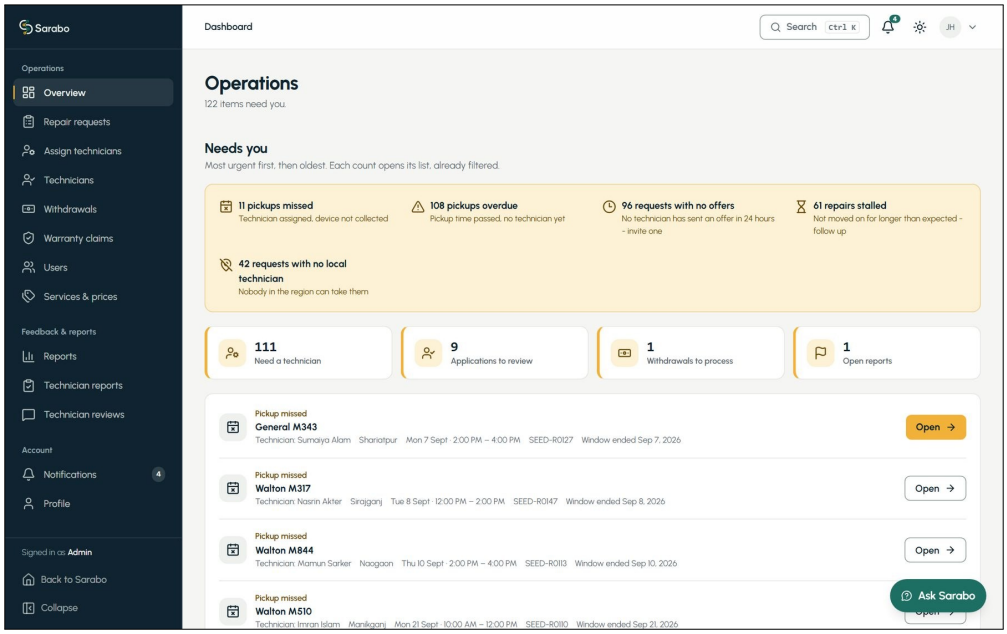


Figure 7.41: The admin's home page, "Operations". The yellow "Needs you" box comes first and counts the urgent problems: missed pickups, overdue pickups, requests with no offers, stalled repairs and requests no local technician can take. Each count opens its own list, already filtered. Below are the counts of waiting work and the most urgent items, oldest first. Most of the numbers come from the seeded development data.

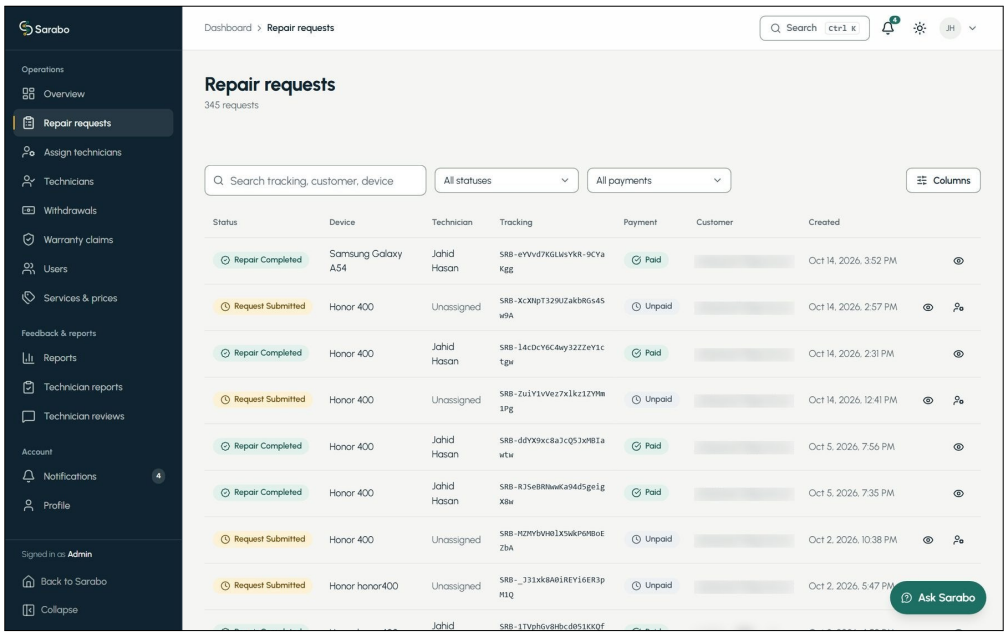


Figure 7.42: The admin's list of all 345 repair requests, with search by tracking code, customer or device, status and payment filters, and selectable columns. Each row shows the status, device, technician, tracking code, payment state and creation time, and the icons open the request or find a technician for it. Customer emails are blurred in this report.

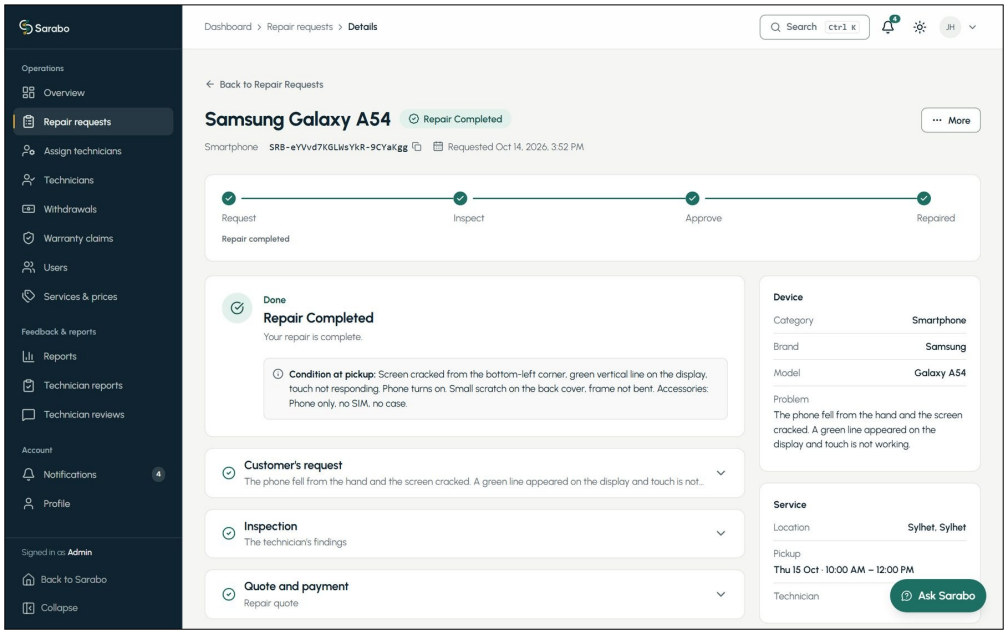


Figure 7.43: The walkthrough repair from the admin's side, with the same layout as the customer's and technician's pages. The admin sees the full record: the pickup condition, the customer's request, the inspection, the quote and payment, and the device and service details. "More" holds the admin actions, such as reassigning the technician or handling a dispute.

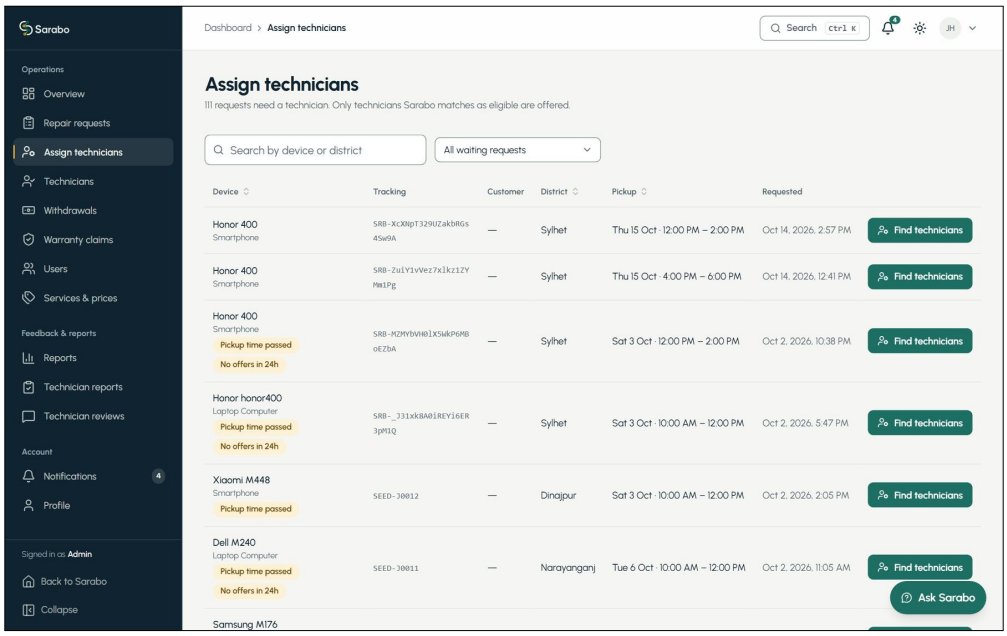


Figure 7.44: "Assign technicians" lists the 111 requests still waiting for a technician. Flags such as "Pickup time passed" and "No offers in 24h" mark the ones that need help, and "Find technicians" suggests only the technicians that the matching rules allow. The admin can invite one of them to apply, or step in and assign one directly.

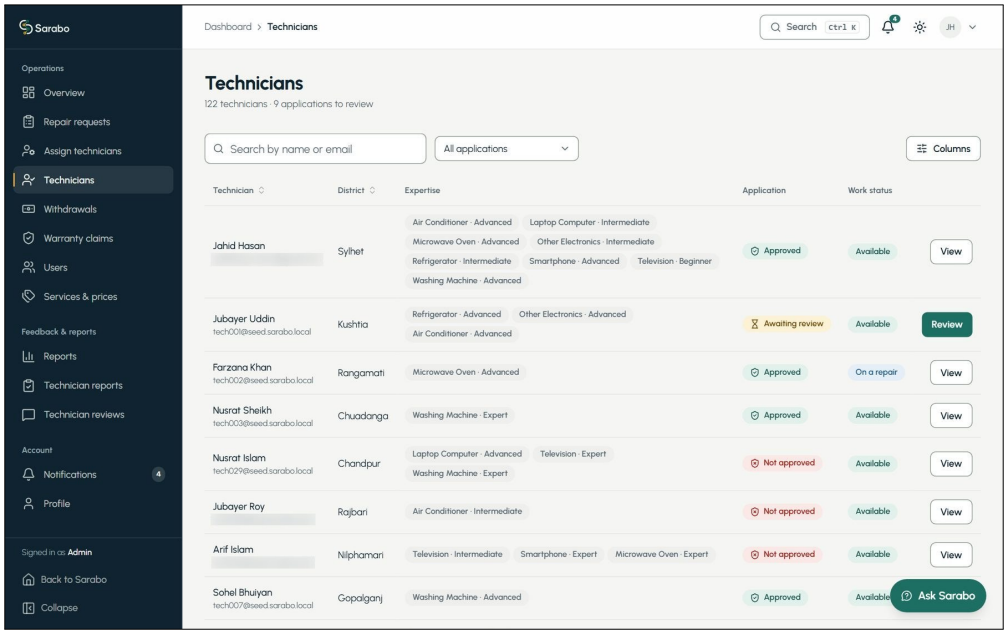


Figure 7.45: The Technicians page lists all 122 technicians and the 9 applications waiting for review. Each row shows the district, the expertise (device and level), the application status (approved, awaiting review or not approved) and whether the technician is free or on a repair. “Review” opens an application with its documents for approval.

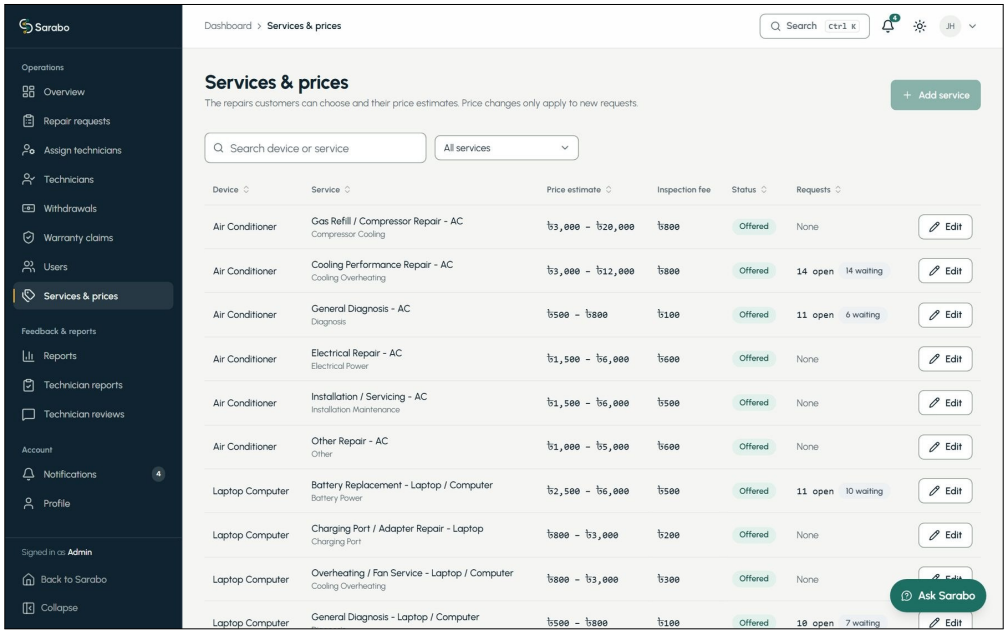


Figure 7.46: The catalogue editor, “Services & prices”. Each service has a device, a name, a price estimate, an inspection fee, a status and the number of open and waiting requests that use it. The note at the top says price changes apply only to new requests.

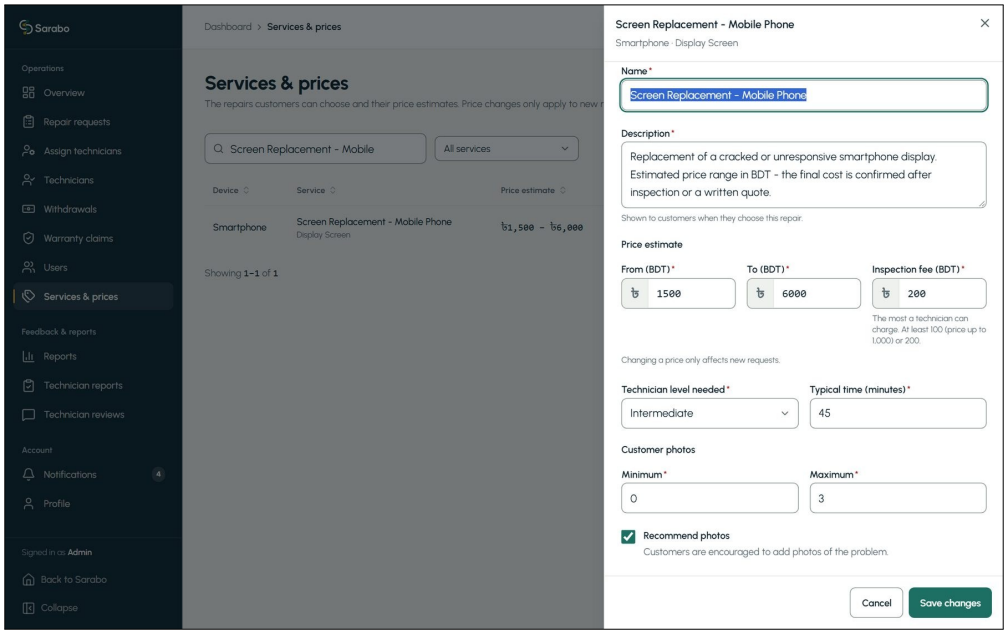


Figure 7.47: Editing the "Screen Replacement – Mobile Phone" service used in the walkthrough. The admin sets the description shown to customers, the 1,500–6,000 BDT estimate, the 200 BDT inspection fee (not below the minimum), the technician level needed, the typical time and how many photos customers may add. Saving creates a new pricing version, so existing requests keep their old price.

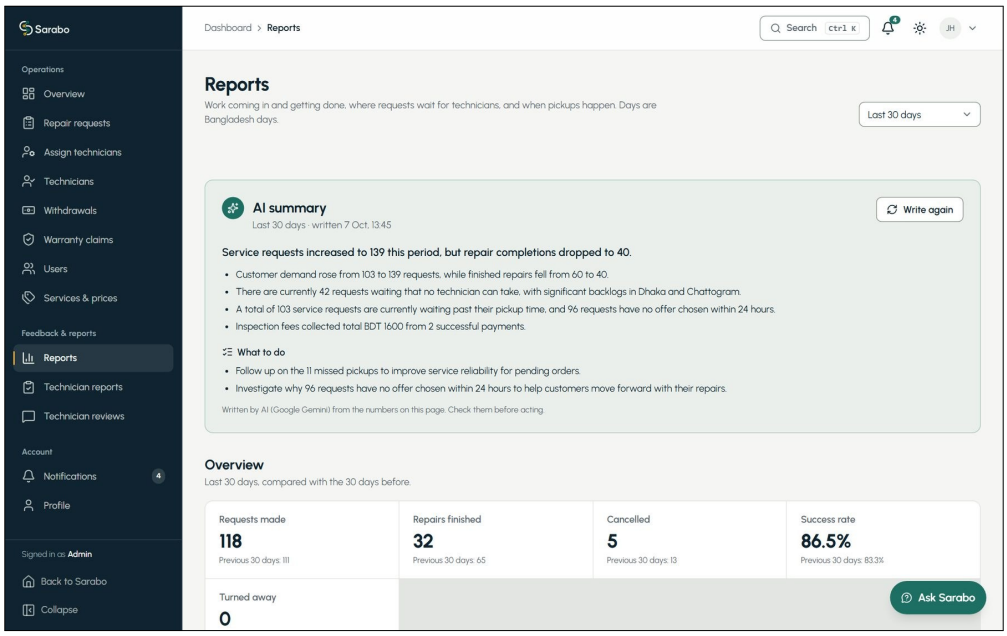


Figure 7.48: The admin Reports page for the last 30 days. The AI summary at the top (a headline, highlights and "What to do") is written by Google Gemini only from the numbers on the page, and any sentence with a number that is not in the data is removed. Below it, the overview compares this period with the previous one.

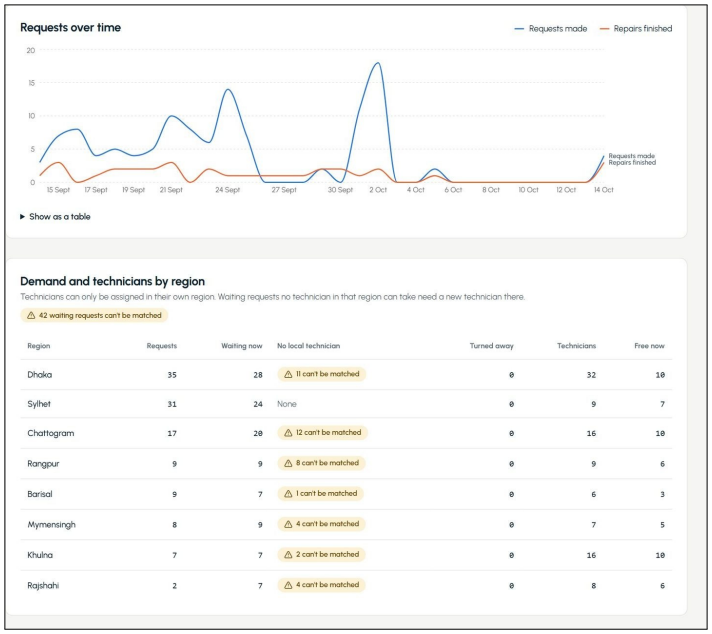


Figure 7.49: Further down the Reports page. The line chart compares requests made with repairs finished each day, and "Show as a table" gives the same numbers as text. The region table shows, for each division, the requests, how many are waiting, how many no local technician can take, how many were turned away, and how many technicians there are and are free.



Figure 7.50: The last part of the Reports page. The bars rank the most requested devices and repairs. The heat map shows how many pickups are booked in each two-hour window of each day, with darker cells for busier slots and Friday closed. Admins can use it to see where and when more technicians are needed.

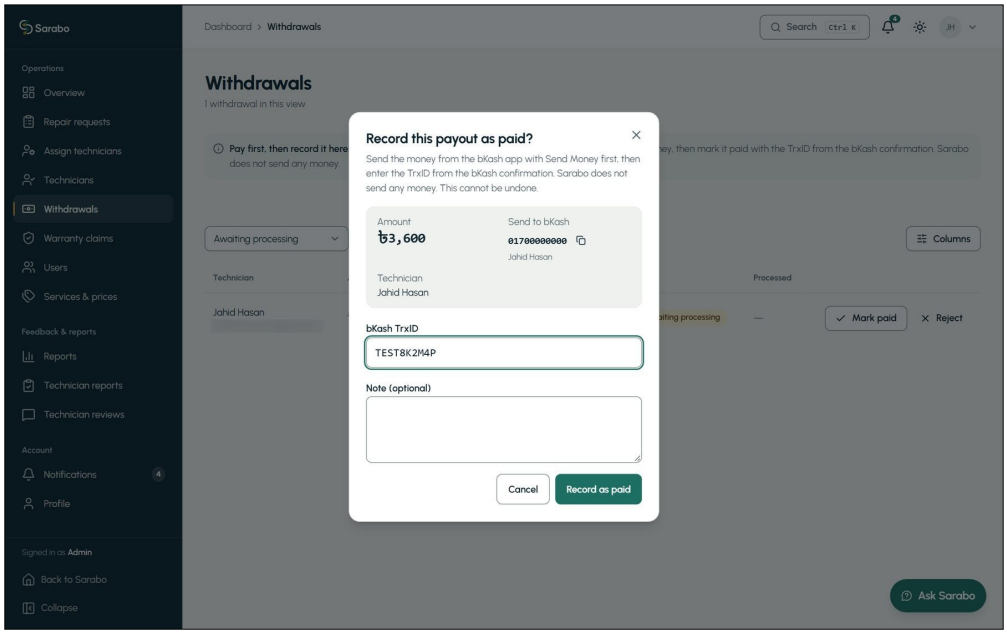


Figure 7.51: An admin recording the technician's withdrawal as paid. Sarabo does not send the money itself. Instead, the admin sends 3,600 BDT to the technician's bKash number with the bKash app, then enters the TrxID from the bKash confirmation (here a test value). A TrxID can be used only once, and the record cannot be undone.

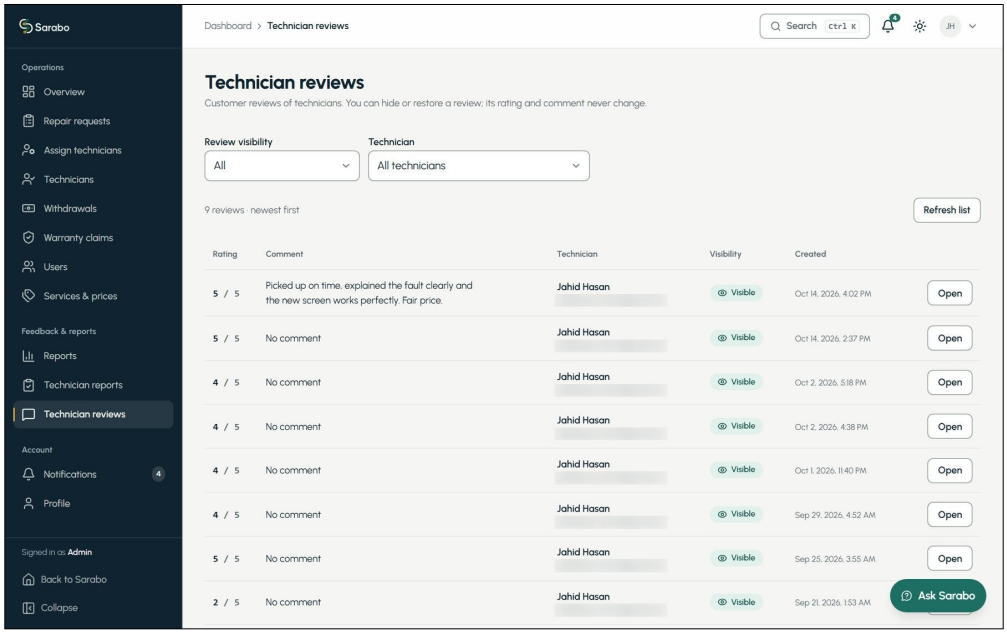


Figure 7.52: The admin's list of technician reviews, filtered by visibility and technician, with Rahim's 5/5 review at the top. An admin can hide or restore a review with a reason, but the rating and comment themselves never change, and every change is kept. Technician emails are blurred in this report.

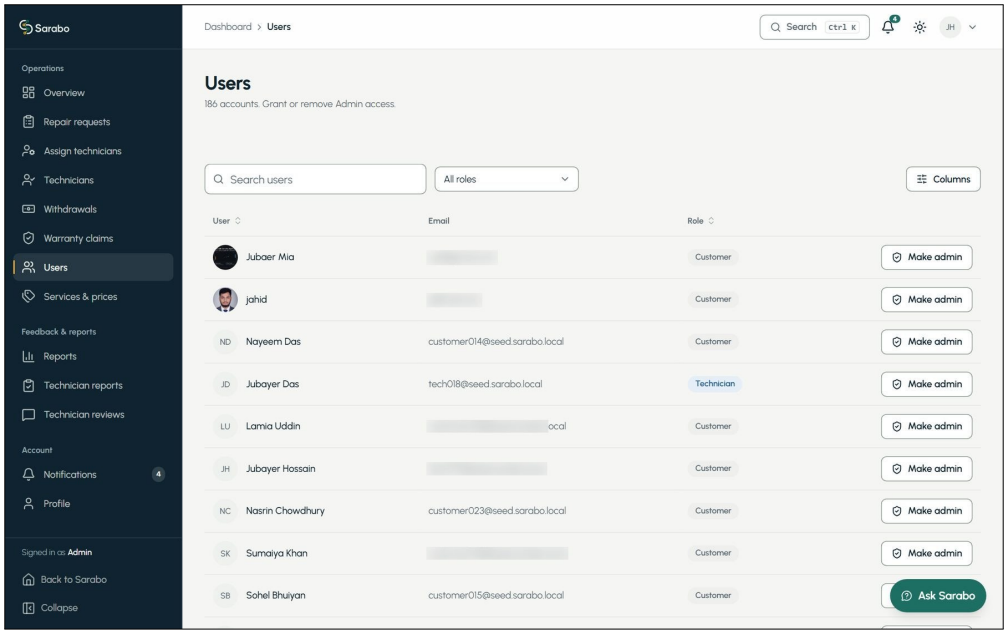


Figure 7.53: The Users page lists all 186 accounts with their role (Customer, Technician or Admin). Only an admin can give or remove admin access, using “Make admin”, and customers can only become technicians through an approved application. Personal emails are blurred, and the addresses that are still visible belong to seeded test accounts.

Chapter 8

AI-Assisted Features

Sarabo uses Google Gemini in four places. In all of them the same idea applies: the AI *suggests*, and a person *decides*. Nothing the AI returns can change a price, a payment or the state of a request unless a person has reviewed it. All four features are off by default and only work once they are turned on in the settings and an API key is supplied. The client calls `GET /ai/status` and hides any feature that is not available. The main model is `gemini-3.5-flash-lite`. When it is slow, overloaded or rate-limited, `gemini-3.1-flash-lite` is used instead.

8.1 AI Customer Support Chat

Every page has a chat that answers questions about how Sarabo works, what services it offers and what they usually cost. The prompt is built from the *live* catalogue (`supportKnowledgeService`), so the prices it mentions are always up to date. The length of the history, of each message and of each answer is capped. The follow-up buttons (View services, Track a request, Log in and Create an account) come from a fixed list, so the AI never makes up links (Figure 7.4).

8.2 AI Request Helper

On the request form, the customer can describe the problem in Bangla, in English or in a mix of the two (“Banglish”), add a photo of the damage, or do both. The AI then drafts the device,

the catalogue service, the brand, the model and an English description. It also gives a confidence level, may ask a follow-up question, and for a photo says whether it actually shows a device. The form fills itself in, the customer can change anything, and *Undo* brings back the previous values (Figure 7.9). If the photo shows a device, it is kept with the request's other photos and sent when the customer presses "Upload", so they do not have to pick it a second time.

8.3 Technician Price Guide and AI Offer Note

The price guide does not use AI at all. It shows the usual price range from real offers made for the same service over the last 180 days, running from the 25th percentile of the "from" prices to the 75th percentile of the "to" prices, and it does the same for fees. If there are fewer than three offers, it uses the catalogue range instead. An offer that falls far outside the guide gets a warning, but the technician can still send it.

The technician can also ask the AI to write the offer note, in English or in Bangla. The model is given only what the technician can see: the device, the repair, the problem description and whether there are photos. It writes in the technician's voice, describes the likely issue only as a possibility, says that the final price will be set after inspection, and uses no numbers. Phone numbers (Bangla digits included), emails, links and social media handles are removed from the note, so all contact stays on the platform (Figure 7.13).

8.4 AI Summary of Admin Reports

The Reports page (for 7, 30 or 90 days, see Section 6.15) can show an AI summary made up of a headline, two to four highlights and one to three suggested actions. The model only receives totals: requests made, finished and cancelled, the success rate, the average number of days, refused requests by region, the top devices and repairs, technician workload, inspection-fee money and the attention counts. It never receives names, emails or IDs. The most recent summary for each period is saved, so opening it again costs nothing (Figure 7.48).

8.5 Safeguards

Table 8.1 sums up how Sarabo deals with the main risks of features built on LLMs [13], and Table 8.2 gives the default usage limits.

Table 8.1: AI risks and safeguards

Risk	Safeguard in Sarabo
Invented catalogue items	The request helper must answer with IDs from an enum built from the live catalogue (JSON schema); the server re-checks them and drops unknown or inactive IDs, or a repair belonging to another device.
Invented prices	No AI feature ever sets a price. Estimates come from the catalogue; the offer note contains no numbers.
Invented statistics	Every sentence of the report summary is checked: each number must appear in the facts, be a difference or rounded percentage between periods, a small count up to 10, or a fixed context number. Failing sentences are dropped; if nothing remains, the request fails with 502.
Prompt injection	Customer text is marked as untrusted data in the prompt, and output is forced into a schema.
Personal data leakage	Text containing a card number, password, token or tracking code is refused before any AI call. Models receive no names, emails or addresses. Contact details are stripped from generated notes.
Unbounded cost	Per-user 10-minute and daily limits plus a site-wide daily budget per feature, stored in MongoDB so all serverless instances share them. If the database is unreachable, the feature fails closed.
Bad photos	JPEG/PNG/WebP only, at most 1.5 MB, first bytes must match the type; the client shrinks photos to 1024 px first. The helper's photo is not stored by the AI endpoint.
Provider failure	If the first model is slow (over 5 s for support), overloaded or rate-limited, the request goes straight to the fallback model; otherwise controlled errors (503 unavailable, 504 timeout, 502 invalid answer) and the rest of the form keeps working. Live support answers take 1.4–1.9 s.

Table 8.2: Default AI usage limits

Feature	Per user / 10 min	Per user / day	Site / day
Support chat (signed in)	20	100	300
Support chat (anonymous)	8	30	(shared)
Request helper	10	30	200
Offer note	10	40	200
Report summary	5	20	50

All of the AI tests use fake providers, so the automated test suites never call Gemini and never cost any money.

Chapter 9

Security, Reliability and Data Integrity

9.1 Security Principles

In Sarabo the server has the final word. The client decides what to show, but only the server decides what is allowed, and it checks authorization on every operation on every resource [18]. The secrets for the database, Stripe, Supabase, Gemini, Gmail and the cron job are kept only in the server's environment variables. They never appear in the repository or in the browser.

9.2 Authentication

Every protected request carries a Firebase ID token, which the server verifies with Firebase Admin. The caller's email is taken only from this token. This is why endpoints such as `/technicians/me` or the wallet never accept someone else's email as input. A user also needs a verified email address before they can create requests or make payments.

9.3 Role- and State-Based Authorization

Every endpoint that changes data checks two things: who the caller is, and what state the request is in.

- Only Admins can approve technicians, edit the catalogue, retry refunds, process withdrawals and moderate feedback.

- Only an approved Technician who matches the job can see its details or send an offer, and only the booked Technician can inspect, quote, revise and repair.
- Only the Customer who owns the request can choose or reject offers, decide on a quote, pay, cancel, confirm receipt and leave a review.
- A quote can only be submitted after the inspection, payment can only happen after approval, and a revision can only start from `quote_rejected`. A paid repair can never go back into the rejected-quote path.

If a request does not belong to the caller, the server gives the same "not found" answer as it would for a request that does not exist, so it does not reveal that the request is there.

9.4 Financial Integrity

Prices, fees, credits, commissions, balances and withdrawal limits are all calculated on the server from stored data, in whole taka. Stripe sessions use amounts the server has stored, and the webhook signature is checked against the raw request body [10]. Payments and refunds are idempotent, and a settlement is fixed only once. A withdrawal moves to *paid* or *rejected* through an update that only one caller can win, so it can never be paid twice.

9.5 Privacy

The public endpoints (tracking, the jobs board and availability) return data shaped specially for them, with no names, emails, addresses, phone numbers, payment data or database IDs. Until they are booked, technicians see only the district and the problem. The counts of refused requests store a hash instead of the customer's email, and reports about technicians are visible to admins only.

9.6 Private Image Security

The storage bucket is private. Each signed URL is for one object only and expires quickly. The server checks the real type and size of each stored file, and it never sends out the permanent identifier of an object.

9.7 Input Validation and Safe Errors

The server validates IDs, state changes, amounts, text lengths, file types and the fields each update is allowed to touch, and it rejects anything unexpected. Text written by users is stored and shown as plain text. Errors use fixed codes such as `PICKUP_SLOT_FULL` instead of passing on raw messages from providers. The logs keep only the request ID, the route, the status and a category, never request bodies or secrets.

9.8 CORS, Rate Limiting and Abuse Control

The API accepts calls from browsers only if they come from the configured `https` client origin. Tracking, the jobs board and all of the AI features are rate limited. The AI limits are stored in MongoDB, so they still apply when the API runs on several serverless instances at once. The scheduled job will not run without its secret.

9.9 Transaction Safety

MongoDB transactions protect the rules that matter most:

- a request's state and its technician's availability always agree, so a technician cannot be booked twice;
- a payment, the booking it leads to and the offers it closes all change together;
- a change to a quote, its tracking event and its notification are applied together or not at all;
- a fee refund is marked as pending in the same transaction as the event that causes it.

A bug found late in development (Appendix E) taught me a less obvious rule. When a duplicate-key error happens inside a transaction, the transaction has already been aborted. The error therefore has to be passed on so the driver can retry, and must never be swallowed.

9.10 Environment Isolation

Development, testing and production each use their own database, and the tests refuse to run against the development or production database. Seed and migration scripts do a dry run unless told otherwise, and running them on production needs explicit one-time flags. CI uses a throwaway database and no real credentials.

9.11 Legacy Compatibility Boundary

Some stored values still come from the parcel-delivery codebase Sarabo started from, such as the role `rider` and statuses like `driver_assigned` or `parcel_delivered`. They are fixed contract values hidden behind a presentation layer, so users only ever see repair terms. Renaming them would need a data migration, which is left as future work.

Chapter 10

Testing and Evaluation

10.1 Testing Strategy

The testing mainly checks that the business rules are right. In other words, can the right person do the right thing at the right time, and only once? I used four layers of testing (Figure 10.1).

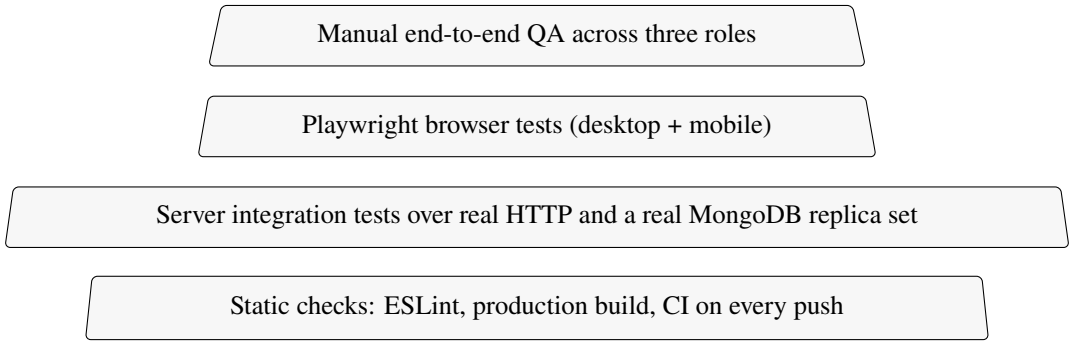


Figure 10.1: Testing layers used in Sarabo

Server integration tests. A custom test harness (`test-all.js`) starts its own copy of the API on a random port, connected to a separate test database. It creates accounts for each role, an approved technician and the catalogue, and then calls the endpoints over real HTTP. It also checks some services directly. Stripe, Supabase, Gemini and email are all replaced with fakes, so the tests never touch an outside service or any money. To test race conditions, the harness sends competing requests at the same moment, for example two admins assigning the same request, a cancellation arriving together with a payment, or one webhook delivered several times. In every case there must

be exactly one winner, and the data must end up consistent.

Browser tests. Playwright runs a production build of the client against a fake API, at both desktop and mobile (Pixel 7) screen sizes. The tests cover the public pages, the jobs board (the listing, filters, sorting, and the empty and error states), tracking (valid, unknown and invalid codes, and server errors), the sign-in forms and route protection.

Continuous integration. On every push and pull request, GitHub Actions runs the server suites on a throwaway MongoDB 8 replica set, and runs lint, the build and the browser tests for the client. On the final commits (16 October for the server and 14 October 2026 for the client), the server workflow passed and so did the client's lint-and-build job. The client's browser-test job failed once on GitHub, even though the same 58 tests pass locally on that commit (Table 10.1).

10.2 Verification Results

For this report I ran every check between 14 and 16 October 2026 on the final code (client commit `e9caaeb` and server commit `d81fa65`). Just as in CI, the server suites ran against a temporary in-memory MongoDB replica set, so no leftover data in a shared database could affect the results. Table 10.1 shows what happened.

Table 10.1: Verification results (14–16 October 2026)

Check	Passed	Failed
Server full suite (`npm test`)	2,761	0
of which re-run on their own: support and AI suite	164	0
of which re-run on their own: data-migration suite	63	0
of which re-run on their own: feedback suite	177	0
Client browser tests (Playwright, 29 scenarios × 2 viewports)	58	0
Client ESLint	no errors	
Client production build	success	
GitHub Actions, server, latest `main` run	success	
GitHub Actions, client lint and build	success	
GitHub Actions, client browser tests (same tests pass locally)	failed once	

When the earlier report was written in August 2026, the server suite had 1,890 checks. It now has 2,761. The final phase alone added 181 of them: 15 for the final payout on removal, 4 for the AI

fallback, 22 for offer ranking, 45 for the warranty, 72 for the fixes that stop requests getting stuck, 22 for profile editing and the minimum inspection fee, and 1 for the Stripe item name.

10.3 Representative Test Scenarios

Table 10.2: Representative test scenarios

ID	Scenario	Expected and verified result
T-01	Request without a token to a protected endpoint	401; no data returned
T-02	Customer calls an admin endpoint	403; nothing changes
T-03	Request in a region with no matching technician	409 `NO_TECHNICIAN_AVAILABLE`; refusal counted once per day
T-04	Customer picks a pickup slot that is already full	409 `PICKUP_SLOT_FULL`; request not saved
T-05	Unapproved or non-matching technician sends an offer	Refused; offer not stored
T-06	Inspection fee above the catalogue cap	Refused by the server
T-07	Fee payment arrives after another technician was booked	Payment recorded and refunded automatically
T-08	Same Stripe event delivered twice, or webhook and success page both fire	Exactly one payment row and one booking
T-09	Customer rejects an offer	Job reposted; technician may send one revised offer only
T-10	Cancel 3 hours vs 1 hour before pickup	Full refund vs fee kept and split 90/10
T-11	Stripe refund call fails	Refund stays pending, shows on admin home, retry succeeds once
T-12	Quote submitted before inspection	Controlled conflict; state unchanged
T-13	Quote rejected, revised and resubmitted	Old quote archived; new round gets its own notification; no transaction hang
T-14	Approved quote with fee already paid	Amount due = quote total minus fee; settlement on the full quote
T-15	Paid request moves on to repair states	Still reported as paid; no second Pay button
T-16	Repair completed but not yet received	Settlement pending; cannot be withdrawn
T-17	Withdrawal above available balance, or two open withdrawals	Refused; database allows only one open request
T-18	Withdrawal marked paid, then technician asks again	Nothing left to withdraw; balance can never go negative
T-19	Duplicate review, or review before receipt	Refused; one review per repair
T-20	AI draft names an unknown or inactive service	ID dropped by the server

ID	Scenario	Expected and verified result
T-21	AI summary sentence with a number not in the facts	Sentence removed before saving
T-22	Problem text containing a card number	Refused before any AI call
T-23	Public tracking and jobs board responses	No names, emails, addresses or payment data
T-24	Seed or migration script pointed at production	Refused without explicit operator override
T-25	Admin removes a technician with 3,000 BDT available	Removal and a 3,000 BDT final bKash withdrawal commit together; with no bKash number on record, removal is refused and nothing changes
T-26	Visitor searches "Ambarkhana" on the Service Areas page	Map zooms to Ambarkhana in Sylhet, not the district centre; "Sylhet" still opens the district
T-27	Two offers: nearby, quick, reliable technician vs. a cheaper one with a missed pickup	The first is ranked "Best match"; no private details in the response
T-28	Warranty claim covered, re-repair fails	Admin refunds the whole quote once; settlement reversed; already-withdrawn money shown as owed
T-29	Warranty claim after 30 days, or a third claim	Refused
T-30	Approved quote unpaid for 3 days, with the payment page open	Skipped that day (no one charged for a closed request); closed the next day, fee kept 90/10
T-31	Customer neither confirms nor disputes receipt	Confirmed automatically after 3 days; money released exactly once
T-32	Repair payment arrives after the request was cancelled	Payment recorded and refunded automatically
T-33	Offer with an inspection fee below the minimum	Refused (`INSPECTION_FEE_TOO_LOW`); an older low offer is charged at the minimum
T-34	Technician changes service area during an active repair	Refused; allowed once the repair is finished

10.4 Manual End-to-End Acceptance

In August 2026 I tested the core lifecycle by hand with three accounts and Stripe test cards: request, assign, inspect, quote, reject, revise, approve, pay, repair, complete and confirm receipt. I checked that every role saw the same state, and that the money only became available after the customer confirmed receipt. On 14 October 2026 I went through the marketplace flow in the same way, using real customer, technician and admin accounts. This run covered an AI-assisted request,

an offer with an AI note, the inspection-fee payment, the pickup condition, inspection, the quote, approve-and-pay, the repair, the hand-back, a review, a warranty claim through to its resolution, and a bKash withdrawal marked as paid. Chapter 7 is the record of that run.

10.5 Evaluation Against Objectives

Table 10.3: Evaluation against objectives

Obj.	Status	Evidence
O1	Achieved	Firebase identity, server role and state checks (T-01, T-02)
O2	Achieved	Structured V2 form, catalogue, photos, pickup slots, AI helper (T-04, T-20)
O3	Achieved	Region and expertise gate, jobs board, offers, reposting and ranking (T-03, T-05, T-09, T-27)
O4	Achieved	Fee booking, credit, refunds and split (T-06, T-07, T-10, T-11, T-14)
O5	Achieved	Inspection, quote, rejection and revision (T-12, T-13)
O6	Achieved	Single payment truth, idempotent processing, PDF receipts (T-08, T-15)
O7	Achieved	Progress, evidence, receipt confirmation, reviews and reports (T-16, T-19)
O8	Achieved	47 notification types, email and daily reminders (both running in production), sanitized tracking (T-23)
O9	Achieved	90/10 settlement, computed wallet, bKash withdrawals, final payout on removal (T-16 – T-18, T-25)
O10	Achieved	Attention list, reports with checked AI summary, catalogue editor, moderation (T-21)
O11	Achieved	2,761 server checks, 58 browser tests, lint, build, CI, live deployment
O12	Achieved	Warranty, deadlines and stalled-work alerts (T-28 – T-32)

10.6 Validation Boundary

It is worth being clear about what these results do and do not show. The automated results come from an isolated environment with fake providers. Payments run in Stripe test mode, so no real money has changed hands, and the platform has no real customers yet. Load testing and a formal security audit were outside the scope of this project.

Chapter 11

Limitations and Future Work

11.1 Current Limitations

- **Technician payouts are manual.** The wallet keeps exact track of earnings and withdrawals, but the money itself is sent by an admin through bKash Send Money, who then records the TrxID. There is no automatic payout through a bKash, Nagad or bank API.
- **Card payments run in test mode.** Stripe does not offer local accounts to businesses in Bangladesh. A real launch would need a local gateway such as SSLCOMMERZ or the bKash Payment Gateway, plugged in behind the same payment service interface.
- **Pickup capacity is checked but not locked.** If two customers book the last place in a slot in the same second, both bookings could go through. At the current scale this is acceptable, since admins can see every pickup time, but a busy service would need an atomic slot counter.
- **Prices are provisional.** The 45 services and their price ranges are a starting point for demonstration, not prices agreed with a real repair business. Admins can change them at any time.
- **A technician can have only one active job.** This prevents double booking, but it does not fit technicians who handle several small jobs in a day.
- **Some internal values are left over from the old codebase.** Stored values such as `rider` and `parcel_delivered` come from the parcel-delivery codebase. Users never see them, but a migration would make the data model cleaner.

- **The browser tests do not cover signed-in screens.** They only cover the public pages, because signing in needs real Firebase accounts. The signed-in screens are covered instead by server tests of every endpoint behind them, and I checked them by hand (Chapter 7).
- **There are no real customers yet.** The platform is deployed and tested, but it has not yet been used by real customers and technicians with real money, and it has not had a load test or an external security audit.

11.2 Future Enhancements

1. **A local payment gateway and automatic payouts**: SSLCOMMERZ or bKash for both customer payments and technician payouts, with reconciliation.
2. **Live technician location**: showing the technician's approach on a map on the day of the pickup.
3. **A mobile app or Progressive Web App**: a technician app that works well offline and supports push notifications.
4. **SMS notifications**: many customers in Bangladesh rely on SMS more than on email.
5. **Parts inventory**: recording the parts used in each repair, with their cost and supplier.
6. **Technician scheduling**: daily capacity, working hours and assignment that takes travel routes into account.
7. **Cash payment**: letting customers pay in cash at hand-over, with the platform's share settled from the technician's wallet.
8. **A proper domain name**: a `.com` or `.com.bd` address, since some mobile networks could not resolve the free `.vercel.app` address.
9. **A Bangla interface**: translating the whole interface, not just accepting Bangla input for the AI.
10. **Cleaning up the schema**: removing the leftover values through a planned migration.

Chapter 12

Conclusion

In this project I developed Sarabo, a platform that turns informal electronics repair, which today depends almost entirely on trust, into a tracked service with clear prices and a full record of every step. Customers describe the problem in their own language and compare ranked offers from qualified local technicians. They pay an inspection fee that is later taken off the final price, approve an itemised quote, follow every step of the repair, and are protected by a 30-day warranty. Technicians get a steady source of jobs that match their skills and a wallet they can see into. Admins get a single view of everything that needs their attention.

The main engineering result is a workflow in which the server has the final word. An action only succeeds if the caller has the right role, the request is in the right state, and every amount comes from stored data. Transactions, idempotent payments and refunds, and balances that are calculated instead of stored keep money and state consistent even when actions are retried or happen at the same time. Every waiting state now has a deadline, an automatic step or an admin alert, so no repair can get stuck. Generative AI is used only as an assistant, and its output is always validated and reviewed by a person.

Since the earlier report, Sarabo has grown from a workflow in which admins assigned every job into a live two-sided marketplace. Over the same period its automated server checks grew from 1,890 to 2,761, all passing, alongside 58 browser tests. The remaining limitations (Chapter 11) show what still lies between a validated academic system and a live service. The most important of these are a local payment gateway, automatic payouts and use by real customers.

References

[1] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, University of California, Irvine, 2000.

[2] Meta Platforms, Inc., “React documentation,” https://react.dev/, accessed: Oct. 2026.

[3] TanStack, “Tanstack query documentation,” https://tanstack.com/query/latest, accessed: Oct. 2026.

[4] Vite Contributors, “Vite documentation,” https://vite.dev/, accessed: Oct. 2026.

[5] Tailwind Labs, “Tailwind CSS documentation,” https://tailwindcss.com/docs, accessed: Oct. 2026.

[6] OpenJS Foundation, “Express documentation,” https://expressjs.com/, accessed: Oct. 2026.

[7] MongoDB, Inc., “MongoDB manual,” https://www.mongodb.com/docs/manual/, accessed: Oct. 2026.

[8] Google, “Firebase authentication documentation,” https://firebase.google.com/docs/auth, accessed: Oct. 2026.

[9] Stripe, Inc., “Stripe payments and checkout documentation,” https://docs.stripe.com/, accessed: Oct. 2026.

[10] ——, “Receive stripe events in your webhook endpoint,” https://docs.stripe.com/webhooks, accessed: Oct. 2026.

[11] Supabase, “Supabase storage documentation,” https://supabase.com/docs/guides/storage, accessed: Oct. 2026.

[12] Google, “Gemini API documentation,” https://ai.google.dev/gemini-api/docs, accessed: Oct. 2026.

[13] OWASP Foundation, “OWASP top 10 for large language model applications,” https://genai.owasp.org/llm-top-10/, accessed: Oct. 2026.

[14] World Wide Web Consortium, “Web content accessibility guidelines (WCAG) 2.2,” W3C Recommendation, https://www.w3.org/TR/WCAG22/, 2023.

[15] Sheba Platform Limited, “Sheba.xyz,” https://www.sheba.xyz/, accessed: Oct. 2026.

[16] Urban Company, “Urban company,” https://www.urbancompany.com/, accessed: Oct. 2026.

[17] Thumbtack, Inc., “Thumbtack,” https://www.thumbtack.com/, accessed: Oct. 2026.

[18] OWASP Foundation, “REST security cheat sheet,” https://cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html, accessed: Oct. 2026.

Appendices

Appendix A

API Surface

Every route except the public ones needs a Firebase ID token. The "Role" column shows who is allowed to call each route (C = Customer, T = Technician, A = Admin, P = public).

Table A.1: Sarabo API endpoints (main families)

Method	Route	Role	Purpose
Public			
GET	`/service-definitions`, `/service-definitions/:id`	P	Active catalogue
GET	`/service-availability`	P	Can a request be served here?
GET	`/pickup-slots`	P	Available pickup windows
GET	`/jobs`	P	Open jobs board
GET	`/public/trackings/:trackingCode`	P	Sanitized tracking
GET	`/ai/status`	P	Which AI features are on
POST	`/stripe-webhook`	Stripe	Signed payment events
GET	`/health`	P	Health check
Repair requests			
GET/POST	`/repair-requests`	C	List own / create
GET	`/repair-requests/:id`	C,T,A	Authorized detail
PATCH	`/repair-requests/:id/pickup-slot`	C	Reschedule pickup
PATCH	`/repair-requests/:id/cancel`	C	Cancel before booking
GET/POST	`/repair-requests/:id/booking-cancellation`, `/cancel-booking`	C	Preview / cancel a booked visit
POST	`/ai/request-draft`	C	AI request helper
Job portal			
GET	`/technician/job-portal`	T	Matching open jobs with details
POST	`/repair-requests/:id/applications`	T	Send an offer
GET	`/repair-requests/:id/applications/price-guide`	T	Price guide
POST	`/repair-requests/:id/applications/ai-note`	T	AI offer note
POST	`/repair-requests/:id/applications/mine/withdraw`	T	Withdraw own offer

Method	Route	Role	Purpose
POST	`/repair-requests/:id/applications/:applicationId/accept`	C	Choose an offer
POST	`/repair-requests/:id/applications/:applicationId/decline`	C	Reject an offer (repost)
Inspection, quote, payment, repair			
POST	`/repair-requests/:id/inspection`	T	Submit inspection
POST	`/repair-requests/:id/quote`	T	Submit quote
POST	`/repair-requests/:id/quote/decision`	C	Approve / reject
POST	`/repair-requests/:id/quote/revise`	T	Revise rejected quote
POST	`/repair-requests/:id/quote/cancel-request`, `/quote/close`	T, C	End after a declined quote
GET	`/repair-requests/:id/payment-eligibility`	C	Can this be paid now?
POST	`/repair-requests/:id/checkout-session`	C	Start repair payment
GET	`/payments`, `/payments/:transactionId/receipt.pdf`	C,A	History and PDF receipt
POST	`/repair-requests/:id/repair/start`, `/progress`, `/evidence`, `/complete`	T	Repair workflow
POST	`/repair-requests/:id/confirm-receipt`	C	Confirm hand-over
Technicians, wallet and feedback			
GET	`/technicians/me`, `/technicians/me/reviews`	T	Own profile and reviews
PATCH	`/technicians/:id/expertise`	T,A	Edit expertise
GET	`/technician/wallet`	T	Wallet balances
POST	`/technician/withdrawals`	T	Request bKash withdrawal
GET	`/repair-requests/:id/feedback`	C	Review eligibility and own feedback
POST	`/repair-requests/:id/review`, `/technician-reports`	C	Review / private report
GET	`/notifications`, `/notifications/unread-count`	C,T,A	In-app notifications
Administration			
GET	`/admin/repair-requests`	A	Paginated, filterable requests
POST	`/admin/repair-requests/:id/invite`	A	Invite a technician to apply
POST	`/admin/repair-requests/:id/missed-pickup/withdraw`	A	Technician no-show
POST	`/admin/repair-requests/:id/inspection-fee/retry-refund`	A	Retry a pending refund
GET	`/admin/attention`, `/admin/reports`	A	Attention list and reports
GET/POST	`/admin/reports/ai-summary`	A	AI summary
GET/PATCH	`/admin/service-definitions`, `/:id`	A	Catalogue editor
PATCH	`/technicians/:id`	A	Approve / reject technician
GET/POST	`/admin/withdrawals`, `/:id/mark-paid`, `/:id/reject`	A	Withdrawals
GET/PATCH	`/admin/technician-reviews`, `/admin/technician-reports`	A	Moderation
PATCH	`/users/:id/role`	A	Change a user's role
PATCH	`/technicians/me`	T	Edit own profile and service area
GET	`/repair-requests/:id/applications`	C,A	Offers with ranking (`match`)
GET/POST	`/repair-requests/:id/warranty`, `/warranty-claims`, `/warranty/photos`	C	Warranty status, claim, photos
POST	`/warranty-claims/:claimId/confirm`, `/review-request`	C	"It works now" / ask admin review
GET/POST	`/technician/warranty-claims`, `/warranty-claims/:claimId/schedule`, `/decision`, `/rerepair`	T	Re-inspection and re-repair

Method	Route	Role	Purpose
GET/POST	`/admin/warranty-claims`, `/:claimId/review`, `/decision`, `/cannot-fix`, `/refund`	A	Warranty review, take-over, refund
POST	`/repair-requests/:id/cancel-unpaid`, `/close-not-repairable`	C,T	End an unpaid or unrepairable request
POST	`/repair-requests/:id/repair/cannot-complete`, `/mark-returned`, `/customer-not-available`	T	Workflow exceptions
POST	`/repair-requests/:id/receipt-dispute`, `/confirm-return`, `/pickup-condition/dispute`	C	Disputes and return confirmation
POST	`/admin/repair-requests/:id/cancel-paid-repair`, `/confirm-receipt`, `/pickup-condition/reviewed`	A	Admin resolution
GET	`/cron/pickup-reminders`	Cron	Daily reminders, deadlines and stalled-work alerts

Appendix B

Data Model Summary

Table B.1: Selected fields of a repair request

Field	Meaning
`schemaVersion`	2 for modern requests
`product`	Device category, brand and model snapshot
`service`	Chosen service with its price rule snapshot
`trackingId`	Public tracking code (`SRB-...`)
`deliveryStatus`	Lifecycle state (frozen legacy name)
`pickupSlot`	Chosen two-hour window and region
`damage.images`	Up to three private image identifiers
`repostedAt`, `repostCount`	Job-board reposting after a rejected offer
`technicianId`, `technicianEmail`	Booked technician
`assignmentHistory`	Append-only assignment audit trail
`inspectionPayment`	Inspection fee payment and refund state
`inspectionFeeSettlement`	90/10 split of a kept fee
`inspection`	Findings and detected issues
`quote`, `quoteHistory`	Live quote and archived rejected quotes
`payment`	Repair payment state
`repair`	Progress, completion and evidence
`technicianSettlement`	Frozen commission and receivable
`closedBy`	Who ended a declined request
`pickupCondition`	Device condition and accessories recorded at pickup, with any dispute and admin review
`deviceReturn`	Hand-back of the device (after repair or cancellation): marked, disputed, resolved

Appendix C

Financial Rules

$$\text{Repair subtotal} = \text{Labour} + \text{Parts} + \text{Additional charges}$$

$$\text{Amount due} = \max(\text{Quote total} - \text{Inspection fee paid},\ 0)$$

$$\text{Sarabo commission} = \text{round}(10\% \times \text{Repair subtotal})$$

$$\text{Technician receivable} = \text{Repair subtotal} - \text{Sarabo commission}$$

$$\text{Kept inspection fee} : \text{same 90/10 rule, whole taka}$$

$$\text{Minimum inspection fee} = 100 \text{ if the highest estimate} \leq 1{,}000,\ \text{else } 200$$

$$\text{Warranty refund} = \text{Quote total (repair payment + credited fee); settlement reversed}$$

All amounts are in whole Bangladeshi taka (BDT). The settlement is calculated on the full quote, even when the inspection fee already paid for part of it. A repair settlement can only be withdrawn after the customer confirms receipt, but the technician's share of a kept inspection fee is available straight away.

Table C.1: Worked example

Inspection fee paid at booking	BDT 300
Approved quote (labour 1,200 + parts 1,500 + other 0)	BDT 2,700
Customer pays after approval	BDT 2,400
Sarabo commission (10% of 2,700)	BDT 270
Technician receivable	BDT 2,430

Appendix D

Source and Build Notes

Repositories. Client: `https://github.com/jahidjubaer/sarabo-client`; server: `https://github.com/jahidjubaer/sarabo-server`.

Deployment. Client: `https://sarabo-client-six.vercel.app`; API: `https://sarabo-server.vercel.app`.

Table D.1: Verified code baseline

Item	State
Client branch / commit	`main` / `e9caaeb115581050c2887f828384447d5301c8fa`
Server branch / commit	`main` / `d81fa65c40723ad38ee0663a6821a9459798e4bf`
Commits	163 (client), 120 (server), 26 Jul – 16 Oct 2026
Size	about 337 client source files (≈29,000 lines); about 191 server files (≈45,600 lines, including tests)
Server tests	2,761 / 2,761 passed
Browser tests	58 / 58 passed
Lint and build	Passed

Listing D.1: Development commands

```
# client (sarabo-client)
npm install
npm run dev # http://localhost:5173
npm run lint
npm run build
npm run test:e2e # Playwright browser tests
```

```
# server (sarabo-server)
npm install
npm run dev # http://localhost:3000
npm test # full suite, isolated test database
npm run seed:service-definitions -- --confirm-seed
```

Configuration. This report contains no secret values. The credentials for the database, Firebase, Stripe, Supabase, Gemini, Gmail and the cron job are provided through environment variables. Their names (but not their values) are listed in each repository's `.env.example`, `README.md` and `SETUP.md`.

Appendix E

Incident Record: Revised Quote Hang

What went wrong. While I was testing by hand in August 2026, I found that submitting a *revised* quote (after the customer had rejected the first one) left the technician's screen stuck on "Working..." for good.

Cause. The "quote submitted" notification used a de-duplication key that stayed the same for the whole life of a request. So the second round of quoting hit a duplicate-key error inside the database transaction. The code treated this as a harmless duplicate and carried on, but by then MongoDB had already aborted the transaction. The commit failed, the driver retried the same callback, and this repeated until it timed out, without anything ever being saved.

Fix. The key now includes the quote round, which the server works out itself:

Listing E.1: Round-aware quote notification key

```
round = quoteHistory.length + 1
key = "repair:" + requestId + ":quote_submitted:" + round
```

On top of that, when a notification is created inside a transaction, the code now first checks whether the key already exists in the same session. If a race still causes a duplicate-key error, the error is passed on, so the driver restarts the whole transaction cleanly instead of carrying on with one that has already been aborted. Regression tests cover first and revised submissions, repeats within the same round, and races between transactions.

Lesson. Inside a database transaction, an error may already have broken the transaction by the time the application code sees it. Errors like this have to be passed on, not swallowed.